2015

Themida en Visual Basic

Snat
[CrackSLatinoS]

Themida en Visual Basic

CrackSLatinoS

Agosto 2015

1. Contenido

1. Contenido 1
2. Introducción 2
3. Los preparativos 5
4. Tanteando el terreno 6
5. Al atake 8
6. La Tabla de Themida 19
7. Encontrando el camino 43
8. Asomados al abismo 60
9. Notas Finales 69

2. Introducción

Me gustaría comenzar explicando mi historia con este packer: Hace algún tiempo instalé un software en una máquina para gestionar el Visor de Eventos, tanto en local como en remoto, el programa mostraba una nagscreen al iniciarse informándote de que estabas en versión Trial. Hasta aquí todo bien, intenté crackearlo y me topé con un Themida v2.x según RDG Packer Detector.

En un principio Themida trabajaba con un driver 'oreans.sys' en ring0 con el que modificaba la Taba de Descriptores de Interrupción (IDT) cambiando la dirección de las funciones que deben atender a la INT0x1, la INT0x3, la INT0xB y la INT0xE que afectan al uso de Breakpoints de cualquier tipo tal como nos explicaba Akira en su tutorial de Themida en ring0. Además modificaba la Tabla de Servicios de Sistema (SST) de la que interceptaba varias funciones 'Nt' de la librería ntdll.dll. Todo este proceso está muy bien documentado en el tutorial de Akira.

Hay más información al respecto en la Teoría 700-TheMida_defeating_ring0_by_deroko [muy buena por cierto] sobre la protección en ring0 que creo debería ser de obligada lectura.

Pero los tiempos han cambiado y Themida no es una excepción, hoy por hoy ha dejado el ring0 de un lado y trabaja en ring3 como los packers más comunes (ASProtect, Armadillo, Obsidium, VM Protect, etc..), no por eso ha bajado el listón y la protección sigue siendo 'fuerte'.

Dicho esto he de decir que mi primer encontronazo con Themida fue hace mucho más tiempo atrás, todavía utilizaba el ring0 y era demasiado para mí, ni tan siquiera lo intenté a decir verdad cerré el depurador sin más. Hasta hace cuestión de unos meses cuando instalé el programa antes citado y pensé ¿por qué no intentarlo? Y eso hice, armado de valor lo cargué en al debugger y después de varias vueltas sin encontrar nada lo volví a cerrar, no había manera.

Pero hay que ser insistente en este mundo y hace unas semanas atrás decidí volverlo a intentar, tampoco conseguí nada, me topé con la máquina virtual de Themida y me tiró para atrás, no encontré ni una sola API, tampoco.

Como no me rindo, hace unos días estaba mirando de instalar un Kiosko Kodak en una máquina y mientras esperaba a ver si funcionaba me dio por cargar de nuevo el programa en el debugger. Volví a topar con la máquina virtual pero no desistí y seguí, es una función larguísima a la que llamo función virtual, en ella se escribía en ciertas posiciones de la IAT a las que les iba siguiendo la pista pero no me era posible encontrar el patrón. cosas del destino seguí traceando y llegué a un RET, lo pasé y de pronto todo se volvió más claro, aterricé en una función 'normal' con código de toda la vida, todo este tiempo había estado traceando entre tinieblas. Encontré el loop donde escribe la IAT y por fin decubrí el método del packer para rellenarla. Resulta que para algunas entradas de la IAT saltaba a la función virtual [que había traceado antes] y para otras no lo hacía y las escribía desde la función 'normal' vamos a decir.

Reinicié el proceso varias veces para seguirlo desde el principio y recuperar la IAT, este programa además tiene la peculiaridad de que no modifica la tabla de saltos y el desempakado es más sencillo. Logré llegar al OEP con la IAT correcta tras la aplicación de un script, pero el dumpeado no corría. Dudaba de la IAT así que la probé en el proceso original parado en el OEP [binary Copy – binary paste ya sabéis] y el proceso corrió bien, así que la IAT era correcta, descartada.

Me encontraba con un dumpeado que no corría ni a patadas, recuerdo además que al tratar de dumpearlo con LordPE salía el mensaje '*Couldn't grab process Memory* :(', pensé que habría instalado algún gancho en NtReadVirtualMemory o ZwReadVirtualMemory de la NTDLL.dll, lo miré con Kernel Detective y la IDT estaba intacta, la SST tenía modificaciones pero eran del Antivirus y del StrongOD de Olly, nada ¿?

Desinstalé TODOS los ganchos de la SST con este mismo programa, cambié el 'active dump engine' del LorPE a 'IntelliDump' y ahora sí podía dumpearlo pero con limitaciones '*0x4000 of 0x7F5000 bytes could not be dumped and were padded with zeros*' era el nuevo cartel. Continué igual, reparé el dumpeado con ImportReconstructor pero seguía sin funcionar, ni siquiera con la IAT correcta. Estaba claro que algo no iba bien, traceé el dumpeado y descubrí que los retornos de la función virtual no eran correctos y cuando debía retornar EAX==0x1000h por decir algo retornaba EAX==0x00h ¿algún anti-dump?

Encallado aquí decidí buscar otro ejecutable empakado con Themida, pero claro no iba a instalar 50.000 programas buscando el adecuado, podía empakar algún crackme con el propio Themida, entonces recordé que había unpackme's con Themida y recurrí a ellos. Necesitaba saber lo que estaba pasando con esta función y porqué retornaba cero.

Descargué este unpackme 'WinLicense_UnPack Me!.exe' creo recordar de tuts4you, es un unpackme con Themida v2.0.1.0 + WinLicense, perfecto!

Lo estuve mirando y encontré la función virtual, en este caso era muy diferente de la que había visto en el otro soft, a día de hoy he encontrado 2 tipos de funciones virtuales: lineales y cíclicas. Este crackme utiliza una función tipo cíclica, todo se hace en un loop, va leyendo 'bytes' y en función de éstos salta a una función u otra, en ellas se ejecuta el código de la función que está emulando.

En este unpackme sí que se modificaba la tabla de saltos, además se escribían saltos por todo el código tipo 'JMP 03900000 + NOP' o 'NOP + JMP 03900000', a veces no eran saltos sino CALLS tipo 'CALL 03900000 + NOP' o bien 'NOP + CALL 03900000'.

Me pasó lo mismo, logré un dumpeado con la IAT y los saltos reparados pero la función virtual seguía sin trabajar como debía. Algo estaba pasando por alto.

Pero como todo, de lo malo se aprende, hay un dicho que reza "cuando pierdas no pierdas la lección". Para cerciorarme de que la IAT y los saltos eran correctos necesitaba un ejecutable SIN función virtual, pero cuál? Recordé haber leído tutoriales de +NCR para un ejecutable compilado en GCC Windows (MingW) y otros de Ivinson, Karmany entre otros, en el que trabajan sobre un ejecutable compilado en Visual Basic, en ¡Visual Basic! ahí está la cuestión!

Busqué un unpackme para Themida compilado en Visual Basic y lo encontré entre mis carpetas de crackmes, unpackmes, etc.., como digo creo recordar que los descargué de tuts4you con casi total seguridad, se llama ' Themida 2.2.6.0 UnPackMe_VB6.exe' y lo adjuntaré al tutorial.

Con este trabajaremos. La idea principal es obtener un desempakado por el método tradicional de estudiar lo que hace el packer en la rutina donde escribe la IAT y tratar de recuperarla junto con la tabla de saltos con un script.

Además, he podido observar en los otros Themida que éste lee de una tabla en esta rutina, en un principio no le di mucha importancia pero resulta que la encontré en los otros dos ejecutables, eran muy similares. Por lo que he comprobado a partir de ella se obtiene la información de la API tratada, la dirección de la IAT

en la que irá esta API, y por último el/los jmp's o call's que se deberán insertar en el ejecutable. Todo esto lo tiene en esta tabla.

Lo he comprobado en un ejecutable compilado en Visual C++ 6.0, y en otro compilado en Visual C++ 8.0, y en ambos existe esta tabla. Por esta razón decidí investigar en esa dirección, y este Visual Basic nos brinda una oportunidad excelente ya que trabaja únicamente con una librería [MSVBVM60.DLL] y esta se encargará, en última instancia de cargar las restantes [kernel32.dll, user32.dll, gdi32.dll, advapi32.dll, ole32.dll y oleaut32.dll] como se puede ver en su tabla de importación.

Así pues, trataremos de descifrar la tabla del packer y lo desempakaremos de dos modos distintos: el habitual utilizando un script durante la ejecución de la rutina donde escribe la IAT y de un modo diferente utilizando la información extraída de la Tabla, dejaremos que el packer haga sus destrozos y llegaremos al OEP, desde aquí ejecutaremos un script que leerá la tabla y reparará la IAT y la tabla de saltos [sin ejecutar código, parado en el OEP].

Esto se verá mejor durante el desarrollo que en palabras.

Vamos a ello.

3. Los preparativos

Durante el tutorial utilizaremos las siguientes herramientas:

- ✓ OllyDbg 1.10 de Oleh Yuschuk
  http://www.ollydbg.de

  Plugins:

  - StrongOD v0.4..8.892 by °£·çôÂó°
  - PhantOm 1.20 by Hellsp@wn & Archer
  - ODbgScript 1.82.6 by Epsylon

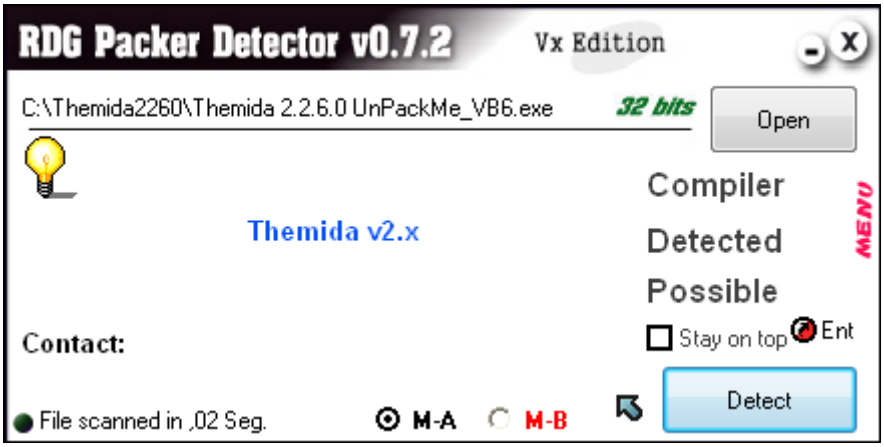
- ✓ ODBGScript Command Reference v0.1 by Cozofdeath [adjunta al texto]
- ✓ RDG Packer Detector v0.7.2 by RDGMax.
- ✓ Scylla x86 v0.9.6b de gRn Team?
- ✓ LordPE Deluxe by Yoda.
- ✓ 32bit Calculator v1.5 by cybult [adjunta al tute, aunque vale cualquier otra]

Todas estas tools son descargables del web de Ricardo Narvaja:

http://www.ricardonarvaja.info/WEB/

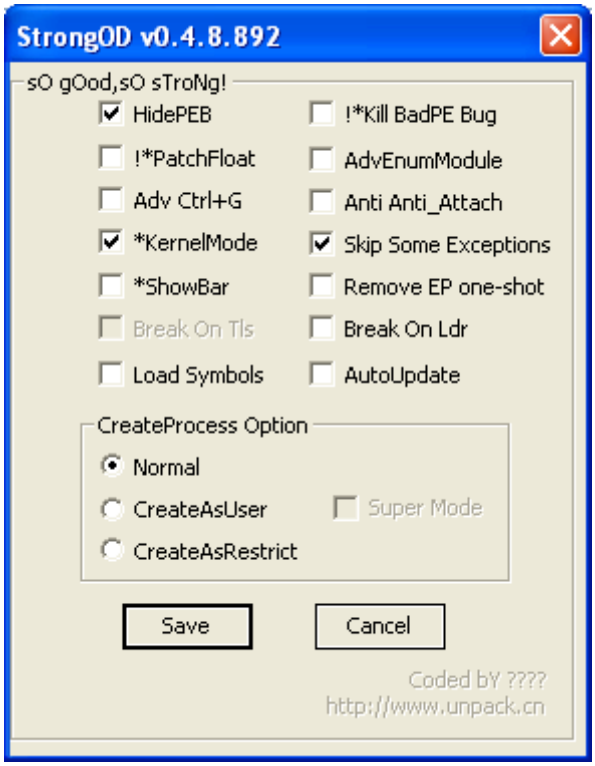
4. Tanteando el terreno

Es un unpackme y no necesita instalación. Sabemos que está protegido con Themida pero le pasamos el RDG Packer Detector igualmente:



Está claro, no nos han mentido.

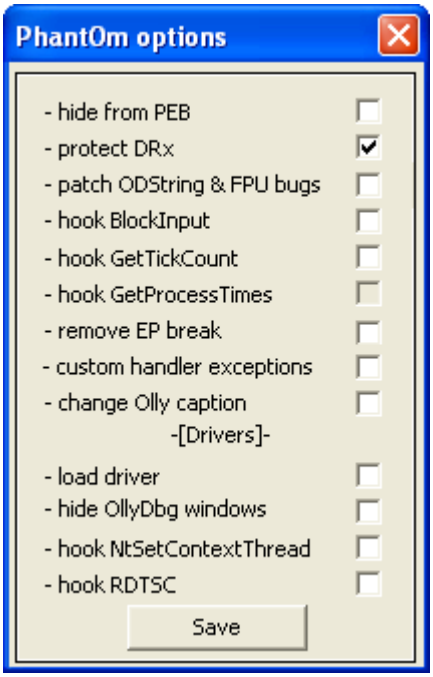
Sabedores del fuerte antidebug que tiene preparamos nuestro Olly para la batalla, utilizaremos dos Plugins para ello, el StrongOD que podemos dejar así:



Todos los Themida no son iguales, algunos con esta configuración llega y otros no llega, es cuestión de afinar probando las opciones.

El siguiente plugin a configurar es el Phant0m.

Lo dejamos así:



Este lo utilizaremos únicamente para preservar los HBP que pongamos, el StrongOD con la opción 'Skip Some exceptions' activa hace que éstos no funcionen bien [al menos a mí me pasa].

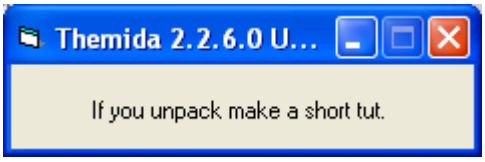
Y ahora sí, estamos listos.

5. Al atake

Cargamos el unpackme en el OllyDbg y paramos en el EntryPoint:

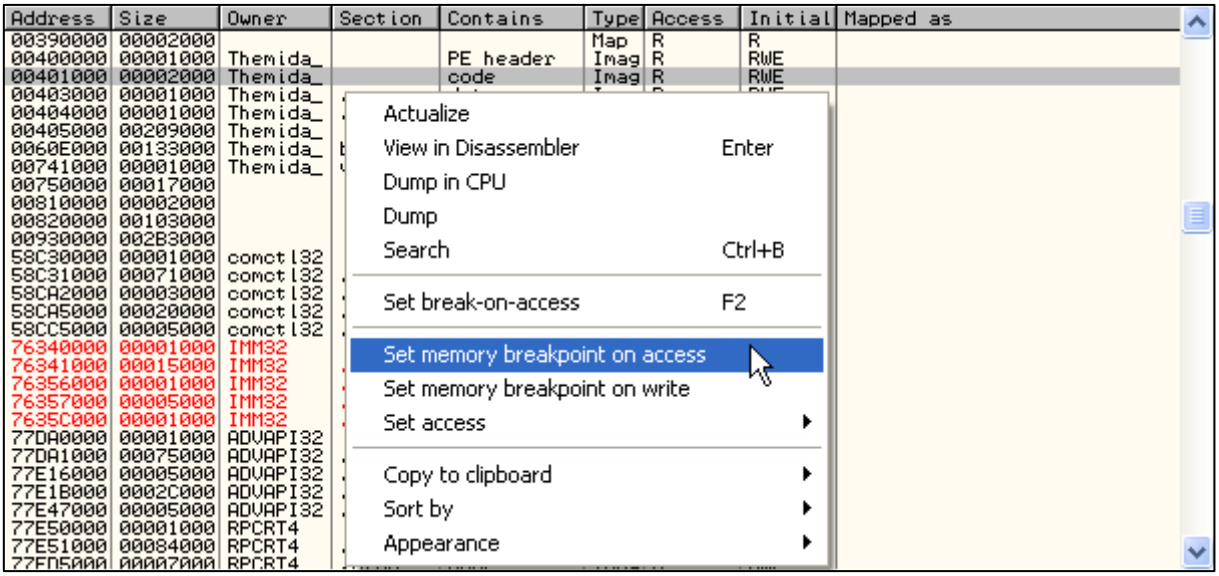
Address	Hex dump	Disassembly	Comment
<ModuleEntryPoint>	83EC 04	sub esp, 0x4	
00741003	50	push eax	
00741004	53	push ebx	
00741005	E8 01000000	call 0074100B	
0074100A	CC	int3	
0074100B	58	pop eax	kernel32.7C816037
0074100C	89C3	mov ebx, eax	
0074100E	40	inc eax	
0074100F	2D 00301300	sub eax, 0x133000	
00741014	2D 8F8E0A10	sub eax, 0x100A8E8F	
00741019	05 848E0A10	add eax, 0x100A8E84	
0074101E	803B CC	cmp byte ptr [ebx], 0xCC	
00741021	75 19	jnz short 0074103C	
00741023	C603 00	mov byte ptr [ebx], 0x0	
00741026	BB 00100000	mov ebx, 0x1000	

Si damos RUN el programa corre:



Cosa poco habitual en Themida que comprueba el breakpoint colocado en el EP y si lo detecta se cierra.

Reiniciamos Olly <Ctrl+F2> y colocamos un 'BPM on Access' en la sección de código:



Lo dejamos correr y para por lectura varias veces, después por escritura en un 'rep movsb' y luego aquí:

Address	Hex dump	Disassembly	Comment
005F7515	8F00	pop dword ptr [eax]	02D10000
005F7517	60	pushad	
005F7518	B1 F5	mov cl, 0xF5	
005F751A	61	popad	
005F751B	AD	lodsd	
005F751C	60	pushad	
005F751D	0F8C 15000000	jl 005F7538	
005F7523	E8 0F000000	call 005F7537	
005F7528	1308	adc ecx, dword ptr [eax]	
005F752A	5C	pop esp	02D10000
005F752B	E2 C7	loopd short 005F74F4	
005F752D	DA4D D4	fimul dword ptr [ebp-0x2C]	

Paró por escritura, está escribiendo la IAT, está en un ciclo, vamos parando en instrucciones tipo *stosd* o *mov* para reajustar la tabla de saltos, si tenemos paciencia con <F9> llegamos al OEP:

```
Address   Hex dump    Disassembly                          Comment
00401111    2C FB     sub     al, 0xFB
00401113    90        nop
00401114    02F8      add     bh, al
00401116  - E9 D6F89002 jmp     02D109F1
0040111B    50        push    eax
0040111C  - E9 8CF99002 jmp     02D10AAD
00401121    D8E9      fsubr   st, st(1)
00401123  v 7D 24     jge     short 00401149
00401125    54        push    esp
00401126  v 72 01     jb      short 00401129
00401128    68 BC124000 push    004012BC                     // ** OEP
0040112D    E8 F0FFFFFF call    00401122
00401132    0000      add     byte ptr [eax], al
00401134    0000      add     byte ptr [eax], al
00401136    0000      add     byte ptr [eax], al
00401138    3000      xor     byte ptr [eax], al
0040113A    0000      add     byte ptr [eax], al
0040113C    40        inc     eax
```

No tiene Stolen bytes. Anotamos este dato:

- ✓ OEP==0x401128h

Veamos la IAT:

```
Address   Hex dump                                         ASCII
00401000  00 00 D1 02 98 00 D1 02 AC 00 D1 02 BD 01 D1 02  ................
00401010  77 03 D1 02 B9 04 D1 02 E8 05 D1 02 45 07 D1 02  w...........E...
00401020  23 08 D1 02 F1 09 D1 02 7B 0A D1 02 AD 0A D1 02  #.......{.......
00401030  83 0B D1 02 41 0C D1 02 00 00 D2 02 BD 0B D2 02  ....A...........
00401040  98 0D D2 02 00 00 D3 02 69 01 D3 02 6E 02 D3 02  ........i...n...
00401050  90 03 D3 02 BB 04 D3 02 BD 05 D3 02 A4 35 94 72  ..............5.
00401060  54 06 D3 02 03 07 D3 02 49 08 D3 02 07 09 D3 02  T.......I.......
00401070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401080  E9 9E F7 90 02 E8 E9 75 EF 91 02 4A E9 6F EF 92  .......u...J.o..
00401090  02 FF E9 22 F4 90 02 10 E9 DA F2 90 02 10 E9 CB  ...".........."
004010A0  F1 92 02 74 E9 03 F0 90 02 4A E9 0E F5 92 02 E8  ...t.....J......
```

Las entradas son del tipo 0x02Dxxxxxh, una sección de memoria que el dumpeado no tendrá. Anotemos los datos de la IAT:

- ✓ Inicio==0x401000h
- ✓ Final==0x401070h
- ✓ Largo==0x70h

Es una mini-iat.

Reiniciamos Olly y colocamos nuevamente el 'BPM on Access' en la sección de código, vamos pulsando <F9>, si para en un 'rep movsb' <F8> + <F9> para ir más rápido, hasta que pare en el 'pop [eax]' de la imagen anterior [dirección 0x5F7515h], donde está escribiendo en la IAT, en los registros hay esto:

EAX 00401000 Themida_.00401000 >> Dirección de la IAT
ECX 02D10000 >> API redirigida
EDX 7C91E514 ntdll.KiFastSystemCallRet
EBX 005FA0DA Themida_.005FA0DA
ESP 0012FF6C
EBP F7015014
ESI 005EA533 Themida_.005EA533
EDI 02D1010E
EIP 005F7515 Themida_.005F7515

Los valores interesantes están en EAX y ECX, el primero tiene la dirección de la IAT y el segundo la dirección de la API redirigida que coincide con el valor de [ESP] que es el que escribirá en [EAX]:

```
0012FF6C  02D10000
0012FF70  FFFFF000
0012FF74  0000006C
0012FF78  00000400
0012FF7C  F7015014
0012FF80  0012FF94
0012FF84  005703FD  Themida_.005703FD
0012FF88  7C91E514  ntdll.KiFastSystemCallRet
```

Ese 0x02D10000h debería contener la dirección de una API. Anotemos la dirección:

- ✓ Escribe en la IAT==0x005F7515h

Tracearemos la rutina del packer para ver qué más hace:

```
Address   Hex dump     Disassembly          Comment
005F7A45  0F8B 00000000  jpo   005F7A4B
005F7A4B  59             pop   ecx
005F7A4C  61             popad
005F7A4D  AA             stosb               // ** inserta Opcode #E9
005F7A4E  E9 05000000    jmp   005F7A58
005F7A53  50             push  eax
005F7A54  ED             in    eax, dx       I/O command
005F7A55  4D             dec   ebp
005F7A56  56             push  esi           Themida_.005EA53B
```

Escribió el Opcode 0x0E9h en la dirección 0x004010E6h:

```
Address   Hex dump                                        ASCII
004010E6  E9 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
004010F6  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401106  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401116  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401126  90 90 68 BC 12 40 00 E8 F0 FF FF FF 00 00 00 00  ..h..@..........
00401136  00 00 30 00 00 00 40 00 00 00 00 00 00 00 AA BF  ..0...@.........
00401146  76 30 60 D2 72 4C 83 FD D4 EF B8 CF 49 CF 00 00  v0'.rL......I...
00401156  00 00 00 00 01 00 00 00 00 00 00 00 00 00 50 72  ..............Pr
00401166  6F 6A 65 63 74 31 00 00 00 00 00 00 00 00 00 00  oject1..........
```

El resto son NOPs, sigamos..

```
Address   Hex dump     Disassembly                  Comment
005F7A96  01F9           add   ecx, edi             Themida_.004010E7
005F7A98  53             push  ebx                  Themida_.005FA0CE
005F7A99  88C3           mov   bl, al
005F7A9B  8819           mov   byte ptr [ecx], bl   // ** inserta byte restante
005F7A9D  5B             pop   ebx                  Themida_.005FA0DA
005F7A9E  59             pop   ecx                  Themida_.005FA0DA
005F7A9F  0F88 07000000  js    005F7AAC
005F7AA5  0F87 01000000  ja    005F7AAC
005F7AAB  F8             clc
005F7AAC  FC             cld
```

En esta instrucción escribe un byte 'basura' en la dirección del JMP+5, debido a que está reemplazando un:

```
JMP DWORD PTR [Direccion_IAT]      >> 0xFF25xxxxxxxx [6 bytes]
```

POR:

```
JMP Direccion                      >> 0xE9xxxxxxxx [5 bytes]
```

Le falta un byte para completar la instrucción reemplazada, de ahí esta parada.

Veamos qué escribió:

```
Address   Hex dump                                          ASCII
004010E6  E9 90 90 90 90 CE 90 90 90 90 90 90 90 90 90 90  ................
004010F6  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401106  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401116  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401126  90 90 68 BC 12 40 00 E8 F0 FF FF FF 00 00 00 00  ..h..@..........
00401136  00 00 30 00 00 00 40 00 00 00 00 00 00 00 AA BF  ..0...@.........
00401146  76 30 60 D2 72 4C 83 FD D4 EF B8 CF 49 CF 00 00  v0`.rL......I...
00401156  00 00 00 00 01 00 00 00 00 00 00 00 00 00 50 72  ..............Pr
00401166  6F 6A 65 63 74 31 00 00 00 00 00 00 00 00 00 00  oject1..........
00401176  00 00 FF CC 31 00 01 7D 47 36 70 61 19 1B 45 88  ....1..}G6pa..E.
```

Va construyendo el salto, sigamos..

```
Address   Hex dump        Disassembly                    Comment
005F7B17    5E            pop     esi                    Themida_.005EA53B
005F7B18    66:B8 BA67    mov     ax, 0x67BA
005F7B1C    61            popad
005F7B1D    AB            stosd                          // ** Completa JMP
005F7B1E  v 0F81 19000000 jno     005F7B3D
005F7B24  v 0F83 13000000 jnb     005F7B3D
005F7B2A  v E9 0E000000   jmp     005F7B3D
005F7B2F    E5 00         in      eax, 0x0               I/O command
005F7B31  v 78 01         js      short 005F7B34
005F7B33    B3 F5         mov     bl, 0xF5
005F7B35    8182 78E15547 add     dword ptr [edx+0x4755E
```

Aquí completa el salto:

```
Address   Hex dump                                          ASCII
004010E6  E9 15 EF 90 02 CE 90 90 90 90 90 90 90 90 90 90  ................
004010F6  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401106  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401116  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00401126  90 90 68 BC 12 40 00 E8 F0 FF FF FF 00 00 00 00  ..h..@..........
00401136  00 00 30 00 00 00 40 00 00 00 00 00 00 00 AA BF  ..0...@.........
00401146  76 30 60 D2 72 4C 83 FD D4 EF B8 CF 49 CF 00 00  v0`.rL......I...
00401156  00 00 00 00 01 00 00 00 00 00 00 00 00 00 50 72  ..............Pr
00401166  6F 6A 65 63 74 31 00 00 00 00 00 00 00 00 00 00  oject1..........
```

Lo podemos ver más claramente en modo desensamblado o en la ventana de código:

```
Address   Hex dump        Disassembly                    Comment
004010E6  - E9 15EF9002   jmp     02D10000
004010EB    CE            into
004010EC    90            nop
004010ED    90            nop
004010EE    90            nop
004010EF    90            nop
004010F0    90            nop
004010F1    90            nop
```

Finalmente quedó 'JMP 02D10000'. Ya ha resuelto una entrada.

Continuamos traceando la rutina hasta llegar aquí:

```
Address   Hex dump        Disassembly                    Comment
005F4A67    BF 8340DE1A   mov     edi, 0x1ADE4083
005F4A6C    29F8          sub     eax, edi
005F4A6E    5F            pop     edi                    // ** API en EAX
005F4A6F    60            pushad
005F4A70    8AC7          mov     al, bh
005F4A72    61            popad
005F4A73  v 0F87 0D000000 ja      005F4A86
005F4A79    60            pushad
005F4A7A  v 0F8D 00000000 jge     005F4A80
005F4A80    BA 1BCE5B1E   mov     edx, 0x1E5BCE1B
```

Si miramos los registros:

EAX 72A309F9 MSVBVM60._adj_fptan
ECX 72940000 offset MSVBVM60.#1374
EDX 76AACDEB
EBX 02700000

En EAX está la API con la que va a trabajar en este ciclo. Comprueba que no se trate de ThunRTMain:

```
Address   Hex dump    Disassembly                                  Comment
005F4D73  3B85 C6303F09 cmp     eax, dword ptr [ebp+0x93F30C6]     MSVBVM60.ThunRTMain
005F4D79  0F84 33000000 je      005F4DB2
005F4D7F  E9 11000000   jmp     005F4D95
005F4D84  51            push    ecx                                MSVBVM60.#1374
005F4D85  5F            pop     edi                                Themida_.004010EB
005F4D86  F7BA F894D4A4 idiv    dword ptr [edx+0xA4D494F8]
005F4D8C  07            pop     es                                 Modification of segment r
005F4D8D  B4 94         mov     ah, 0x94
005F4D8F  19B8 9AECC73E sbb     dword ptr [eax+0x3EC7EC9A], edi    Themida_.004010EB
```

De ser así la tratará diferente a las demás, también comprueba que no sea ExitProcess, LoadLibraryA y unas cuantas apis más.

Lo que hace después es COPIAR la API completa a una zona de memoria dinámica [0x02CE0000h en mi máquina] tal que así:

```
Address   Hex dump   Disassembly
02CE0000  D9F2       fptan
02CE0002  C3         ret
02CE0003  DD1424     fst     qword ptr [esp]
02CE0006  E8 882BD56F call   MSVBVM60.72A32B93
02CE000B  E8 8A93D56F call   MSVBVM60.72A3939A
02CE0010  83C4 08    add     esp, 0x8
02CE0013  C3         ret
02CE0014  0000       add     byte ptr [eax], al
02CE0016  0000       add     byte ptr [eax], al
```

De aquí la pasa a otra sección, 0x2CF0000h en este caso, donde ha copiado la API entera entremezclada con código basura:

```
Address   Hex dump    Disassembly
02CF0000  D9F2        fptan
02CF0002  E9 12000000 jmp     02CF0019
02CF0007  D2A3 A0591EFF shl   byte ptr [ebx+0xFF1E59A0], cl
02CF000D  CC          int3
02CF000E  15 2A1BB891 adc     eax, 0x91B81B2A
02CF0013  F6F7        div     bh
02CF0015  64:CD 82    int     0x82
02CF0018  93          xchg    eax, ebx
02CF0019  C3          ret
```

Las únicas instrucciones válidas son el **fptan** y el ret, el resto las puso de relleno, de aquí pasará a la sección definitiva a continuación de la API anterior:

```
Address   Hex dump    Disassembly                                  Comment
02D1007A  A5          movsd
02D1007B  7A 2B       jpe     short 02D100A8
02D1007D  C3          ret
02D1007E  D9F2        fptan
02D10080  E9 12000000 jmp     02D10097
02D10085  D2A3 A0591EFF shl   byte ptr [ebx+0xFF1E59A0], cl
02D1008B  CC          int3
02D1008C  15 2A1BB891 adc     eax, 0x91B81B2A
02D10091  F6F7        div     bh
02D10093  64:CD 82    int     0x82                                 Superfluous prefix
02D10096  93          xchg    eax, ebx                             Themida_.005FA0DA
02D10097  C3          ret
02D10098  0000        add     byte ptr [eax], al
02D1009A  0000        add     byte ptr [eax], al
02D1009C  0000        add     byte ptr [eax], al
```

Esta dirección 0x02D1007Eh es la que pondrá en la IAT en lugar de la dirección de _adj_fptan.

Con la siguiente API hará lo mismo, la copiará entera a la sección 0x02CE0000h en mi máquina, de aquí la copiará con código polimórfico a la sección 0x02CF0000h de donde la pasará a su lugar definitivo en la sección 0x02D10000h a continuación de la última API que copió. Podemos afirmar que la siguiente entrada de la IAT [no la actual] apuntará a 0x002D10098h que es la dirección siguiente al RET de fptan.

Llegamos a la escritura de la IAT:

```
Address   Hex dump        Disassembly                      Comment
005F7513    61            popad
005F7514    51            push    ecx
005F7515    8F00          pop     dword ptr [eax]          // ** Escribe en la IAT
005F7517    60            pushad
005F7518    B1 F5         mov     cl, 0xF5
005F751A    61            popad
005F751B    AD            lodsd
005F751C    60            pushad
005F751D  v 0F8C 15000000 jl      005F7538
```

Donde escribió la dirección que conocemos 0x02D1007Eh:

```
Address   Hex dump                                         ASCII
00401000  00 00 D1 02 7E 00 D1 02 00 00 00 00 00 00 00 00  ....~...........
00401010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

La siguiente parada es para insertar el Opcode 0x0E9h en 0x5F7A4Dh que ya vimos antes, luego pasa por 0x5F7A9Bh donde inserta el byte basura del JMP y después por 0x5F7B1Dh donde completa el salto:

```
Address   Hex dump        Disassembly                      Comment
004010D9    90            nop
004010DA  - E9 9FEF9002   jmp     02D1007E
004010DF    D090 90909090 rcl     byte ptr [eax+0x90909090]
004010E5    90            nop
004010E6  - E9 15EF9002   jmp     02D10000
004010EB    8890 90909090 mov     byte ptr [eax+0x90909090]
004010F1    90            nop
004010F2    90            nop
004010F3    90            nop
```

Ya tiene dos. Hemos cubierto otro ciclo.

Todo sigue igual hasta que le llega el turno a ThunRTMain, en este caso no pasa por 0x005F4A6Eh donde está la API en EAX sino por aquí:

```
Address   Hex dump        Disassembly                      Comment
005F45FD    2D 8C6B5F69   sub     eax, 0x695F6B8C
005F4602    01C8          add     eax, ecx                 MSVBVM60.#1374
005F4604    05 8C6B5F69   add     eax, 0x695F6B8C
005F4609    05 46394877   add     eax, 0x77483946
005F460E  v E9 14000000   jmp     005F4627                 // ** API caso ESPECIAL
005F4613    3A7D 10       cmp     bh, byte ptr [ebp+0x10]
005F4616    F6BB 1580B713 idiv    byte ptr [ebx+0x13B78015]
005F461C    083F          or      byte ptr [edi], bh
005F461E    8521          test    dword ptr [ecx], esp
```

La API no la encontraremos en el punto anterior sino en este, lo debemos tener en cuenta para el script, además esta API no la redirecciona como las otras:

```
Address   Hex dump                                         ASCII
0040103C  61 0A D2 02 49 0C D2 02 BB 0D D2 02 00 00 D3 02  a...I...........
0040104C  D0 00 D3 02 75 02 D3 02 9B 03 D3 02 A3 04 D3 02  ....u...........
0040105C  A4 35 94 72 00 00 00 00 00 00 00 00 00 00 00 00  .5.r............
0040106C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040107C  00 00 00 00 E9 DC F6 90 02 7C E9 75 EF 91 02 67  .........|.u...g
0040108C  E9 2A FD 91 02 68 E9 D2 F4 90 02 BA E9 F4 F3 90  .*...h..........
0040109C  02 DA E9 2D F0 92 02 AA E9 B3 F0 90 02 E6 E9 F4  ...-............
004010AC  F3 92 02 CA E9 79 F5 90 02 F0 E9 E0 F2 92 02 BF  .....y..........
```

La escribe tal cual en la IAT, su dirección real.

Traceamos el resto de entradas y nada cambia, el resto las trata como en el primer caso. Con esta información ya podemos lanzar un ataque, sólo nos falta saber en qué registros están los datos que necesitamos para reparar la IAT y la tabla de saltos.
Tenemos estos puntos:

- ✓ Para obtener la API:
  - 0x5F4A6Eh para el caso general
  - 0x5F460Eh para los casos especiales
    - La API está en EAX en ambos casos
- ✓ Para reemplazar el valor MALO tipo 0x02Dxxxxxh de la IAT por la API correcta
  - 0x5F7517h tras el 'pop [eax]'
    - La posición de la IAT está en EAX
- ✓ Para reparar el SALTO:
  - 0x5F7B1Eh tras el 'stosd' que completa el SALTO
    - La dirección del salto está en EDI-5

Con estos datos ya podemos escribir, para el punto 1 donde obtenemos la API lo único que hará el script será mover la API de EAX a la variable API:

```
EsAPIenEAX:
mov api, eax
log api
jmp inicio
```

Para el segundo punto lo que haremos será sobrescribir el valor MALO de la IAT por el de la variable api que tendrá la dirección correcta obtenida en el punto 1 y asignaremos a la variable direccionIAT la posición de la IAT en la que se escribe para utilizarla en el punto siguiente [punto3], será algo así:

```
EsSalvaAPI:
mov [eax], api
mov direccionIAT, eax
jmp inicio
```

En el tercer punto obtendremos la dirección del salto en EDI y le restamos 5 para situarnos al inicio de éste, pasamos este valor a la variable `direccionJMP` e insertamos un 'jmp dword ptr [`direccionIAT`]'

```
ProcesaREAJUSTA:
mov direccionJMP, edi
sub direccionJMP, 5
eval "jmp dword ptr [{direccionIAT}]"
Log direccionJMP
Log direccionIAT
Log $RESULT
asm direccionJMP, $RESULT
jmp inicio
```

Estas son las tres etiquetas de importancia para nuestro script, como siempre crearemos un loop para ir parando en los HBP's y en función de cuál sea nos dirigiremos a una etiqueta u otra.

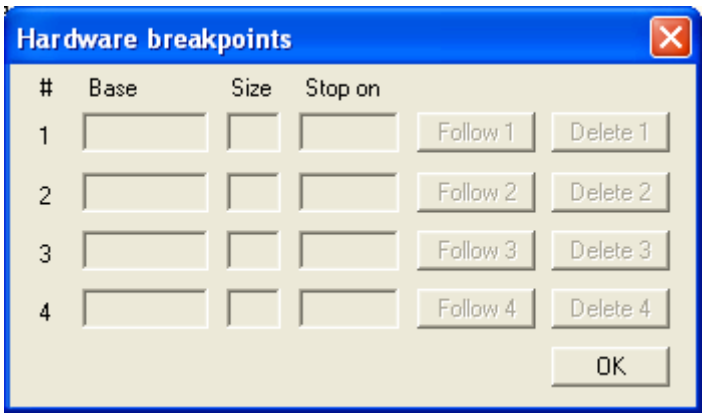
Adicionalmente, para parar en el OEP colocaremos un BPM allí [OEP==0x401128]. El script colocará los HBP's y el BPM al inicio y los borrará cuando llegue al OEP.

El script será el siguiente:

```
1  var api
2  var direccionIAT
3  var direccionJMP
4
5  var APIenEAX
6  var APIenEAX2
7  var salvaAPI
8  var reajustaJMP
9  var OEP
10
11 mov APIenEAX, 005F4A6E
12 mov APIenEAX2, 005F460E      ; caso especial
13 mov salvaAPI, 005F7517
14 mov reajustaJMP, 005F7B1E
15 mov OEP, 00401128
16
17 bphws APIenEAX, "x"
18 bphws APIenEAX2, "x"
19 bphws salvaAPI, "x"
20 bphws reajustaJMP, "x"
21 BPRM OEP, 5
22
23 lclr
24 lc
25
26 inicio:
27 eob rompe
28 run
29
30 rompe:
31 cmp eip, APIenEAX
32 je EsAPIenEAX
33 cmp eip, APIenEAX2
34 je EsAPIenEAX
35 cmp eip, salvaAPI
36 je EsSalvaAPI
37 cmp eip, reajustaJMP
38 je ProcesaREAJUSTA
39 cmp eip, OEP
40 jne inicio
41 jmp Termina
42
```

```
42
43  EsAPIenEAX:
44  log "//////// ***************"
45  mov api, eax
46  log api
47  jmp inicio
48
49  EsSalvaAPI:
50  mov [eax], api
51  mov direccionIAT, eax
52  jmp inicio
53
54  ProcesaREAJUSTA:
55  mov direccionJMP, edi
56  sub direccionJMP, 5
57  eval "jmp dword ptr [{direccionIAT}]"
58  Log direccionJMP
59  Log direccionIAT
60  Log $RESULT
61  asm direccionJMP, $RESULT
62  log ""
63  jmp inicio
64
65  Termina:
66  bpmc
67  bphwc APIenEAX
68  bphwc APIenEAX2
69  bphwc salvaAPI
70  bphwc reajustaJMP
71  ret
72
```

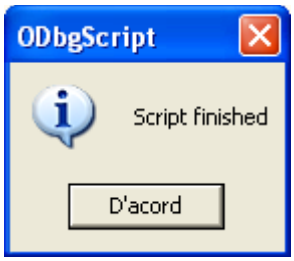
Lo salvamos como 'repara_iat.osc' y reiniciamos Olly, parados en el EP del packer nos aseguramos de no tener Hardware Breakpoints ni BPs comunes [tipo F2]:



Cargamos el script anterior [repara_iat.osc] en ODbgScript:

```
Script Execution
Line  Command                                  Result   EIP      Values <---
      C:\Themida2260\repara_iat.osc
1      var api                                           0056114F
2      var direccionIAT
3      var direccionJMP
4
5      var APIenEAX
6      var APIenEAX2
7      var salvaAPI
8      var reajustaJMP
9      var OEP
10
11     mov APIenEAX, 005F4A6E
```

Lo dejamos correr [RESUME] y en unos segundos termina:



Estamos en el OEP, veamos la tabla de saltos:

```
Address   Hex dump     Disassembly                      Comment
00401081    25 20104000  and     eax, 0x401020
00401086  - FF25 38104000 jmp     dword ptr [0x401038]   MSVBVM60.__vbaExceptHandler
0040108C  - FF25 44104000 jmp     dword ptr [0x401044]   MSVBVM60.__vbaFPException
00401092  - FF25 14104000 jmp     dword ptr [0x401014]   MSVBVM60._adj_fdiv_m16i
00401098  - FF25 10104000 jmp     dword ptr [0x401010]   MSVBVM60._adj_fdiv_m32
0040109E  - FF25 4C104000 jmp     dword ptr [0x40104C]   MSVBVM60._adj_fdiv_m32i
004010A4  - FF25 08104000 jmp     dword ptr [0x401008]   MSVBVM60._adj_fdiv_m64
004010AA  - FF25 58104000 jmp     dword ptr [0x401058]   MSVBVM60._adj_fdiv_r
004010B0  - FF25 18104000 jmp     dword ptr [0x401018]   MSVBVM60._adj_fdivr_m16i
004010B6  - FF25 54104000 jmp     dword ptr [0x401054]   MSVBVM60._adj_fdivr_m32
004010BC  - FF25 50104000 jmp     dword ptr [0x401050]   MSVBVM60._adj_fdivr_m32i
004010C2  - FF25 40104000 jmp     dword ptr [0x401040]   MSVBVM60._adj_fdivr_m64
004010C8  - FF25 28104000 jmp     dword ptr [0x401028]   MSVBVM60._adj_fpatan
004010CE  - FF25 3C104000 jmp     dword ptr [0x40103C]   MSVBVM60._adj_fprem
004010D4  - FF25 0C104000 jmp     dword ptr [0x40100C]   MSVBVM60._adj_fprem1
004010DA  - FF25 04104000 jmp     dword ptr [0x401004]   MSVBVM60._adj_fptan
004010E0  - FF25 60104000 jmp     dword ptr [0x401060]   MSVBVM60._CIatan
004010E6  - FF25 00104000 jmp     dword ptr [0x401000]   MSVBVM60._CIcos
004010EC  - FF25 6C104000 jmp     dword ptr [0x40106C]   MSVBVM60._CIexp
004010F2  - FF25 48104000 jmp     dword ptr [0x401048]   MSVBVM60._CIlog
004010F8  - FF25 1C104000 jmp     dword ptr [0x40101C]   MSVBVM60._CIsin
004010FE  - FF25 30104000 jmp     dword ptr [0x401030]   MSVBVM60._CIsqrt
00401104  - FF25 68104000 jmp     dword ptr [0x401068]   MSVBVM60._CItan
0040110A  - FF25 64104000 jmp     dword ptr [0x401064]   MSVBVM60._allmul
00401110  - FF25 34104000 jmp     dword ptr [0x401034]   MSVBVM60.EVENT_SINK_QueryInterface
00401116  - FF25 24104000 jmp     dword ptr [0x401024]   MSVBVM60.EVENT_SINK_AddRef
0040111C  - FF25 2C104000 jmp     dword ptr [0x40102C]   MSVBVM60.EVENT_SINK_Release
00401122  - FF25 5C104000 jmp     dword ptr [0x40105C]   MSVBVM60.ThunRTMain
00401123    68 BC124000  push    004012BC               // ** OEP
0040112D    E8 F0FFFFFF  call    00401122               jmp to MSVBVM60.ThunRTMain
00401132    0000         add     byte ptr [eax], al
00401134    0000         add     byte ptr [eax], al
```

Si pulsamos F9 el unpackme corre. Podríamos dumpear y fijar la IAT con Scylla. Pero lo haremos después.

Hemos visto que hasta ahora el proceso seguido es muy similar al utilizado con ASProtect y otros packers que destruyen la IAT.

De momento no hemos visto nada de Themida, pues es mucho más que una IAT, estamos en el caso más simple de todos los casos posibles:

- ✓ No hay Stolen OEP.
- ✓ No hay diferenciación entre JMP o CALL [todo son JMP].

- ✓ Sólo se trabaja con una DLL [MSVBVM60.DLL].
- ✓ No se utiliza la Función Virtual.

Si fuera sólo por esto no escribiría el tute, ya que este método está más que explicado y detallado en un sinfín de tutoriales y no aporta nada nuevo.

Si lo escribo es porque vamos a resolver el unpackme de un modo diferente, ya he comentado de la existencia de una Tabla que utiliza Themida para escribir la IAT y la tabla de saltos. Todavía no la hemos visto, es posible hacerle el unpack sin tan siquiera saber de su existencia, pero nosotros vamos a verla, a analizarla, a descifrarla y a valernos de ella tal como hace Themida para obtener la IAT y la tabla de saltos.

Esta tabla además, es extensible en casos de utilización de más de una DLL, así como para casos en que algunas instrucciones son tratadas como JMP y otras como CALL, con o sin función virtual.

Vamos a por ella..

6. La Tabla de Themida

Lo primero que vamos a hacer es encontrar la Tabla, es sencillo, la encontramos en la rutina de tratamiento de la IAT que hemos recorrido antes. Hay que llegar a esta rutina con un BPM en la sección de código hasta que comience a escribir la IAT:

```
Address   Hex dump     Disassembly                    Comment
005F7515    8F00       pop     dword ptr [eax]        // ** Escribe en la IAT
005F7517    60         pushad
005F7518    B1 F5      mov     cl, 0xF5
005F751A    61         popad
005F751B    AD         lodsd                          // ** lee de la tabla dir_JMP
005F751C    60         pushad
005F751D  v 0F8C 15000000 jl      005F7538
005F7523    E8 0F000000 call    005F7537
005F7528    1308       adc     ecx, dword ptr [eax]
ds:[esi]=[005EA533]=C000021C
eax=00401000
```

Escribió la IAT en 0x5F7515h y unas líneas más abajo 0x5F751Bh accede a la tabla citada, la instrucción lods que equivale a 'lods dword ptr [esi]', se puede modificar el comportamiento de Olly para que lo muestre reducido o ampliado desde '*Options -> Debugging options -> Commands -> Use short form of string commands*'. En mi caso lo tengo activado y se muestra en modo reducido.

Hacemos un 'Follow in Dump' sobre el registro ESI en la ventana de registros y caemos aquí:

```
Address   Hex dump                                         ASCII
005EA533  1C 02 00 C0 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA543  07 06 63 2D 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ..c-.......@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF.....5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
005EA593  13 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5A3  38 2E 60 8C 9B 15 BF 95 12 02 00 40 AA AA AA AA  8.'........@....
005EA5B3  FF FF FF FF DD DD DD DD CC 02 78 E0 9B 15 BF B5  ..........x.....
005EA5C3  16 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5D3  2E 3A CF 21 9B 15 BF D5 1F 02 00 00 AA AA AA AA  .:.!............
005EA5E3  FF FF FF FF DD DD DD DD 78 CB DB 73 9B 15 BF F5  ........x..s....
005EA5F3  10 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA603  F4 7D DB BC 9C 15 BF 15 22 02 00 C0 AA AA AA AA  .}......".......
005EA613  FF FF FF FF DD DD DD DD DC 0D A2 79 9C 15 BF 35  ...........y...5
005EA623  19 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA633  87 0F EF EF 9C 15 BF 55 23 02 00 80 AA AA AA AA  .......U#.......
005EA643  FF FF FF FF DD DD DD DD AB D5 5D FC 9C 15 BF 75  ..........]....u
005EA653  1F 02 00 C0 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA663  C9 6B D0 ED 9C 15 BF 95 22 02 00 00 AA AA AA AA  .k......".......
005EA673  FF FF FF FF DD DD DD DD AB 18 F4 CE 9C 15 BF B5  ................
005EA683  10 02 00 C0 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA693  27 55 EB 22 9C 15 BF D5 19 02 00 C0 AA AA AA AA  'U."............
005EA6A3  FF FF FF FF DD DD DD DD B2 25 19 94 9C 15 BF F5  ..........%.....
005EA6B3  18 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA6C3  76 6B 81 CC 9D 15 BF 15 11 02 00 80 AA AA AA AA  vk..............
005EA6D3  FF FF FF FF DD DD DD DD 08 E4 CD 0F 9D 15 BF 35  ................5
005EA6E3  1E 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
```

Allí está la Tabla. Se ven valores tipo FFFFFFFF, DDDDDDDD, AAAAAAAA, BBBBBBBB y otros más normales. En este momento está accediendo al DWORD alojado en 0x5EA533h==0xC000021Ch. Este valor, aunque no lo parezca es la dirección del salto a modificar [donde están los NOPs] de la tabla de saltos [JMP Table].

Para cerciorarnos tracearemos la rutina e iremos desglosando lo que hace con este valor hasta convertirlo en la dirección del salto, seguimos desde aquí con <F7> atentos a lo que haga, sobre todo a aquellas instrucciones que afecten al registro EAX donde está este valor 0xC000021Ch:

```
Address   Hex dump     Disassembly                    Comment
005F7581  v 0F8F 01000000 jg      005F7588
005F7587    F9         stc
005F7588    83F8 FF    cmp     eax, -0x1
005F758B  v 0F85 82000000 jnz     005F7613             // ** tratamos el salto ?
005F7591    60         pushad
005F7592    B8 F5F23973 mov     eax, 0x7339F2F5
005F7597    BF 1D3F2365 mov     edi, 0x65233F1D
005F759C    61         popad
005F759D    813E DDDDDDDD cmp     dword ptr [esi], 0xDDDDDDDD
005F75A3  v 0F85 6A000000 jnz     005F7613
```

Comprueba que el valor no sea 0xFFFFFFFFh, en nuestro caso no lo es y salta a 0x5F7613h donde encontramos la primera operación que afecta a EAX:

```
Address   Hex dump     Disassembly                          Comment
005F7613    C1C0 03    rol     eax, 0x3                     // ** operacion 1
005F7616  v 0F83 01000000 jnb   005F761D
005F761C    F5         cmc
005F761D    0385 09163F09 add   eax, dword ptr [ebp+0x93F160 Themida_.00400000
005F7623  v 0F81 01000000 jno   005F762A
005F7629    F8         clc
005F762A    83BD 5D153F09 cmp   dword ptr [ebp+0x93F155D], 0
005F7631  v 0F84 06050000 je    005F7B3D
005F7637  v E9 0B000000 jmp     005F7647
005F763C    189A FB479381 sbb   byte ptr [edx+0x819347FB], b
```

Hace lo siguiente:

ROL C000021C, 3 == 000010E6

Es un valor que no nos dice nada, pero sigamos:

```
Address   Hex dump     Disassembly                          Comment
005F7613    C1C0 03    rol     eax, 0x3                     // ** operacion 1
005F7616  v 0F83 01000000 jnb   005F761D
005F761C    F5         cmc
005F761D    0385 09163F09 add   eax, dword ptr [ebp+0x93F1609] Themida_.00400000
005F7623  v 0F81 01000000 jno   005F762A
005F7629    F8         clc
005F762A    83BD 5D153F09 cmp   dword ptr [ebp+0x93F155D], 0x1
005F7631  v 0F84 06050000 je    005F7B3D
005F7637  v E9 0B000000 jmp     005F7647
005F763C    189A FB479381 sbb   byte ptr [edx+0x819347FB], bl
```

Tres líneas después le agrega 0x400000h que sí nos suena, y mucho, es el ImageBase de nuestro ejecutable, quedará:

ADD 000010E6, 400000 == 004010E6

Esta es la dirección del salto que deberá apuntar a la dirección de la IAT para esta entrada:

```
Address   Hex dump     Disassembly                          Comment
004010E6    90         nop
004010E7    90         nop
004010E8    90         nop
004010E9    90         nop
004010EA    90         nop
004010EB    90         nop
004010EC    90         nop
004010ED    90         nop
004010EE    90         nop
004010EF    90         nop
```

Es la tabla de saltos.

Anotamos de dónde leyó el ImageBase:

```
Address   Hex dump                                          ASCII
0040661D  00 00 40 00 00 00 00 00 00 00 00 00 98 00 00 00  ..@.............
0040662D  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040663D  20 2C 6B 14 00 00 00 00 00 00 00 00 00 00 00 00   ,k.............
0040664D  00 00 00 00 13 E3 C5 53 00 00 00 00 00 40 00 00  .......S.....@..
0040665D  00 00 00 00 3B 40 40 00 00 00 00 00 00 00 00 00  ....;@@.........
0040666D  00 00 00 00 00 00 CB 00 00 00 00 00 00 00 00 00  ................
0040667D  00 00 00 00 78 00 00 00 00 00 00 00 00 00 00 00  ....x...........
0040668D  00 00 00 00 A4 00 00 00 00 00 00 00 00 00 00 00  ................
0040669D  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004066AD  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Anotamos la dirección:

- ✓ dirección_ImageBase==0x40661Dh

Para obtener la dirección del salto a partir del valor almacenado en la tabla ha aplicado estas operaciones:

```
ror campo_JMP, 3              >> resultado1
add resultado1, ImageBase     >> dirección del salto
```

Quedaría:

Dirección del salto=([campo_JMP ROR 3] ADD ImageBase)

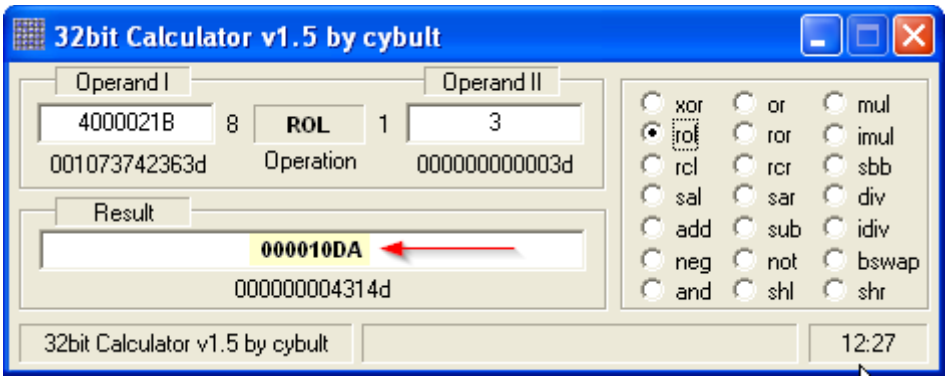
En otros términos:

- ✓ dirección del salto=([campo_JMP ROR 3] ADD [0x40661D]);

Hagamos una prueba, vayamos al siguiente REGISTRO de la Tabla y obtengamos el valor del último CAMPO antes de las AAAAAAAA:

```
Address   Hex dump                                         ASCII
005EA533  00 00 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA543  07 06 63 2D 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ..c-.......@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF......5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
005EA593  13 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5A3  38 2E 60 8C 9B 15 BF 95 12 02 00 40 AA AA AA AA  8.`........@....
005EA5B3  FF FF FF FF DD DD DD DD CC 02 78 E0 9B 15 BF B5  ..........x.....
005EA5C3  16 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
```

Es igual a 0x4000021Bh, abrimos la calculadora de cybult y aplicamos la primera operación a este valor:



El resultado es 0x000010DAh, le agregamos el ImageBase y queda 0x004010DAh, veamos qué hay allí:

```
Address   Hex dump   Disassembly   Comment
004010DA    90       nop
004010DB    90       nop
004010DC    90       nop
004010DD    90       nop
004010DE    90       nop
004010DF    90       nop
004010E0    90       nop
004010E1    90       nop
004010E2    90       nop
004010E3    90       nop
004010E4    90       nop
004010E5    90       nop
004010E6    90       nop
```

Es la dirección del salto siguiente asociado a la entrada de la IAT encriptada en la Tabla del packer.

Anotamos estos datos y seguimos traceando en Olly la rutina a ver qué más hace:

Compara el siguiente campo con AAAAAAAA:

```
Address   Hex dump     Disassembly                          Comment
005F7647    813E AAAAAAAA  cmp     dword ptr [esi], 0xAAAAAAAA
005F764D  v 0F85 B0000000  jnz     005F7703                 // ** caso A's ?
005F7653    60             pushad
005F7654    0FB7D8         movzx   ebx, ax
005F7657  v 0F8F 02000000  jg      005F765F
005F765D    B5 AF          mov     ch, 0xAF
005F765F    61             popad
005F7660    55             push    ebp
005F7661    BD 04000000    mov     ebp, 0x4
ds:[005EA537]=AAAAAAAA
```

En este caso coinciden ambos valores y el salto no se produce, luego borra el valor de la tabla:

```
Address   Hex dump     Disassembly                          Comment
005F7686    D100           rol     dword ptr [eax], 1
005F7688    25 AA14F667    and     eax, 0x67F614AA
005F768D    57             push    edi
005F768E    BF 6F5D8167    mov     edi, 0x67815D6F
005F7693    C746 FC 91A27E mov     dword ptr [esi-0x4], 0x987EA291
005F769A    017E FC        add     dword ptr [esi-0x4], edi  // ** Borra el capo AAAAAAAA
005F769D    8B3C24         mov     edi, dword ptr [esp]
005F76A0    83C4 04        add     esp, 0x4
005F76A3    60             pushad
edi=67815D6F
ds:[005EA537]=00000000

Address   Hex dump                                         ASCII
005EA533  00 00 00 00 00 00 00 00 FF FF FF FF DD DD DD DD  ................
005EA543  07 06 63 2D 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ..c-........@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF......5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
005EA593  13 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5A3  38 2E 60 8C 9B 15 BF 95 12 02 00 40 AA AA AA AA  8.'........@....

0012FF6C  02D100D0
0012FF70  FFFFF000
0012FF74  0000006C
0012FF78  00000400
0012FF7C  F7015014
0012FF80  0012FF94
0012FF84  005703FD  Themida_.005703FD
0012FF88  7C91E514  ntdll.KiFastSyste
0012FF8C  00CBFAD3  RETURN to 00CBFAD
0012FF90  00CBFAD3  RETURN to 00CBFAD

Registers (3DN
EAX 004010E6
ECX 02D10000
EDX 7C91E514
EBX 005FA0DA
ESP 0012FF6C
EBP F7015014
ESI 005EA53B
EDI 67815D6F
EIP 005F769D
C 1  ES 0023
```

A continuación pasa a insertar el Opcode de salto 0x0E9h en 0x4010E6h [dirección de la tabla de saltos], después inserta el byte basura al final y por último lo completa en 0x5F7B1Dh como siempre.

Y pasa a leer el siguiente campo:

```
Address   Hex dump     Disassembly                          Comment
005F7B3D    AD             lodsd                            // ** lee de la tabla
005F7B3E  v E9 0C000000    jmp     005F7B4F
005F7B43    F6B7 8FAB410D  div     byte ptr [edi+0xD41AB8F]
005F7B49    8CE7           mov     di, fs
005F7B4B    3F             aas
005F7B4C    B9 E1D95768    mov     ecx, 0x6857D9E1
005F7B51    41             inc     ecx
005F7B52    4C             dec     esp
005F7B53    8009 5F        or      byte ptr [ecx], 0x5F
ds:[esi]=[005EA53B]=FFFFFFFF
eax=0290EF15

Address   Hex dump                                         ASCII
005EA53B  FF FF FF FF DD DD DD DD 07 06 63 2D 9B 15 BF 15  ..........c-....
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 9B 15 BF 35 14 02 00 80 AA AA AA AA     nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..)....u........
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.'.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............

0012FF70  FFFFF000
0012FF74  0000006C
0012FF78  00000400
0012FF7C  F7015014
0012FF80  0012FF94
0012FF84  005703FD  Themida_.005703FD
0012FF88  7C91E514  ntdll.KiFastSyste
0012FF8C  00CBFAD3  RETURN to 00CBFAD
0012FF90  00CBFAD3  RETURN to 00CBFAD
0012FF94  0012FFBC  Pointer to next S

Registers (3D
EAX 0290EF15
ECX 02D10000
EDX 7C91E514
EBX 005FA0DA
ESP 0012FF70
EBP F7015014
ESI 005EA53B
EDI 004010EB
EIP 005F7B3D
C 1  ES 0023
```

Le toca el turno a la FFFFFFFF, lo borra igual que el anterior. Llegamos a:

```
Address   Hex dump     Disassembly                          Comment
005F7564    89B5 E3545309  mov     dword ptr [ebp+0x95354E3], esi  // ** actualiza puntero Tabla
005F756A  v 0F84 11000000  je      005F7581
005F7570  v E9 0C000000    jmp     005F7581
005F7575    91             xchg    eax, ecx
005F7576    C553 AE        lds     edx, fword ptr [ebx-0x52]   Modification of segment registe
005F7579    E5 0F          in      eax, 0xF                    I/O command
esi=005EA53F (Themida_.005EA53F)
ss:[0054A4F7]=005EA537 (Themida_.005EA537)
```

Donde actualiza el puntero a la Tabla que tiene en 0x54A4F7h:

```
Address   Hex dump                                          ASCII
0054A4F7  3F A5 5E 00 FF FF E9 0E 02 00 00 33 43 61 6E 6E  ?.^........3Cann
0054A507  6F 74 20 77 72 69 74 65 20 6F 72 65 61 6E 73 2E  ot write oreans.
0054A517  76 78 64 0A 0D 4D 61 6B 65 20 73 75 72 65 20 74  vxd..Make sure t
0054A527  68 61 74 20 74 68 69 73 20 66 69 6C 65 20 69 73  hat this file is
0054A537  20 6E 6F 74 20 62 65 69 6E 67 20 75 73 65 64 20   not being used
0054A547  62 79 20 61 6E 6F 74 68 65 72 20 70 72 6F 67 72  by another progr
0054A557  61 6D 2E 00 00 00 00 00 00 00 00 00 00 00 00 00  am..............
0054A567  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0054A577  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0054A587  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lo deja apuntando al siguiente campo que son las DDDDDDDD. Si necesitamos saber con qué campo está trabajando podemos consultar esta dirección. La anotamos:

- ✓ <u>Puntero de tabla</u> == 0x54A4F7h

Continuamos y llega a la comprobación de EAX con 0xFFFFFFFFh, como esta vez sí son iguales llega a la siguiente comparación:

```
Address   Hex dump     Disassembly                          Comment
005F7588    83F8 FF    cmp     eax, -0x1
005F758B  v 0F85 82000000 jnz  005F7613                     // ** tratamos el salto ?
005F7591    60         pushad
005F7592    B8 F5F23973 mov    eax, 0x7339F2F5
005F7597    BF 1D3F2365 mov    edi, 0x65233F1D
005F759C    61         popad
005F759D    813E DDDDDDDD cmp  dword ptr [esi], 0xDDDDDDDD
005F75A3  v 0F85 6A000000 jnz  005F7613
005F75A9  v E9 07000000 jmp    005F75B5
005F75AF    D0FD       sar     ch, 1
ds:[005EA53F]=DDDDDDDD
```

Donde lee el valor DDDDDDDD, valor que borra en 0x5F75E0h y actualiza nuevamente el puntero de tabla en 0x5F75F3h y vuelve a comenzar el ciclo:

```
Address   Hex dump     Disassembly                          Comment
005F4100    C785 022A3F09 mov  dword ptr [ebp+0x93F2A02], 0x0   // ** Inicia Ciclo
005F410A    C785 E22A3F09 mov  dword ptr [ebp+0x93F2AE2], 0x0
005F4114    83BD 0CBD5D09 cmp  dword ptr [ebp+0x95DBD0C], 0x0
005F411B  v 74 06          je  short 005F4123
005F411D    FF95 5B2A5509 call dword ptr [ebp+0x9552A5B]        Themida_.005C66B2
005F4123    FF85 750F3F09 inc  dword ptr [ebp+0x93F0F75]        // ** contador de ciclos
005F4129    60         pushad
005F412A    66:8BC8    mov     cx, ax
005F412D    81EE 8E427B5F sub  esi, 0x5F7B428E
005F4133    61         popad
```

Cada 100 ciclos salta a una función que calcula el CRC de la rutina actual, si tenemos algún BP puesto no coincidirá el valor calculado con el valor semilla, en este caso no hay problema porque la IAT es muy pequeña, pero en otros casos puede ser molesto, para evitar el chequeo habría que JMPear este salto:

```
005F41C7     /0F84 85000000    je       005F4252
```

Y no habría más problemas. Lee el siguiente campo de la Tabla:

```
Address   Hex dump     Disassembly                          Comment
005F42C6    F8         clc
005F42C7    AD         lodsd                                // ** lee TABLA campo_API
005F42C8  v 0F8C 1F000000 jl   005F42ED
005F42CE  v 0F83 19000000 jnb  005F42ED
005F42D4  v E9 14000000 jmp    005F42ED
005F42D9  ^ 74 BC          je  short 005F4297
005F42DB    1187 95ED00DC adc  dword ptr [edi+0xDC00ED95], eax
005F42E1    34 06      xor     al, 0x6
005F42E3    F8         clc
ds:[esi]=[005EA547]=15BF159B
eax=2D63060/
```

En este caso es el primero del REGISTRO:

```
Address   Hex dump                                          ASCII
005EA543  07 06 63 2D 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ..c-.......@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF.....5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
005EA593  13 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5A3  38 2E 60 8C 9B 15 BF 95 12 02 00 40 AA AA AA AA  8.'........@....
005EA5B3  FF FF FF FF DD DD DD DD CC 02 78 E0 9B 15 BF B5  ..........x.....
```

Comienzo de ciclo registro nuevo, este es el campo reservado para la API a tratar, como es el más complejo lo dejamos para el último, seguimos traceando:

```
Address   Hex dump     Disassembly                          Comment
005F7373  61           popad
005F7374  AD           lodsd                                // ** lee TABLA campo_IAT
005F7375  F8           clc
005F7376  50           push    eax
005F7377  B8 6B57247F  mov     eax, 0x7F24576B
005F737C  8946 FC      mov     dword ptr [esi-0x4], eax
005F737F  58           pop     eax
005F7380  FF46 FC      inc     dword ptr [esi-0x4]
005F7383  816E FC 4C25C6 sub   dword ptr [esi-0x4], 0x64C6254C
ds:[esi]=[005EA547]=15BF159B
eax=02D100C2
```

Lee el campo para la dirección de la IAT [encriptada], es el segundo del registro:

```
Address   Hex dump                                          ASCII
005EA543  00 00 00 00 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ............@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF.....5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
005EA593  13 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5A3  38 2E 60 8C 9B 15 BF 95 12 02 00 40 AA AA AA AA  8.'........@....
005EA5B3  FF FF FF FF DD DD DD DD CC 02 78 E0 9B 15 BF B5  ..........x.....
005EA5C3  16 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
```

Allí está 0x15BF159Bh, seguimos traceando atentos a lo que haga con este valor, lo borra en 0x5F738Eh, después le aplica la primera operación:

```
Address   Hex dump     Disassembly                          Comment
005F74AD  C1C0 05      rol     eax, 0x5                     // ** primera operacion campo_IAT
005F74B0  0F82 01000000 jb     005F74B7
005F74B6  FC           cld
005F74B7  F8           clc
005F74B8  56           push    esi                          Themida_.005EA54B
005F74B9  BE A25C1D48  mov     esi, 0x481D5CA2
005F74BE  2D 2971E40A  sub     eax, 0xAE47129
005F74C3  01F0         add     eax, esi                     Themida_.005EA54B
005F74C5  05 2971E40A  add     eax, 0xAE47129
```

Quedando:

ROL 15BF159B, 5 == B7E2B362

Poco después le aplica una nueva operación:

```
Address   Hex dump     Disassembly                          Comment
005F74B6  FC           cld
005F74B7  F8           clc
005F74B8  56           push    esi                          Themida_.005EA54B
005F74B9  BE A25C1D48  mov     esi, 0x481D5CA2
005F74BE  2D 2971E40A  sub     eax, 0xAE47129
005F74C3  01F0         add     eax, esi                     // ** segunda operacion con campo_IAT
005F74C5  05 2971E40A  add     eax, 0xAE47129
005F74CA  5E           pop     esi                          Themida_.005EA54B
005F74CB  60           pushad
```

Le agrega el valor 0x481D5CA2h que movió a ESI primero, entre medio hace un SUB y ADD del mismo valor 0xAE47129h que no alteran el resultado.

Por tanto la operación será:

ADD B7E2B362, 481D5CA2 == 00001004

Ya se aprecia que es el offset de la dirección IAT [sin ImageBase], que agregará más adelante:

```
Address   Hex dump    Disassembly                          Comment
005F74D7  59          pop     ecx
005F74D8  5A          pop     edx
005F74D9  FC          cld
005F74DA  61          popad
005F74DB  0385 09163F09  add    eax, dword ptr [ebp+0x93F1609]  Themida_.00400000
005F74E1  60          pushad
005F74E2  E8 0F000000 call    005F74F6
005F74E7  B1 9C       mov     cl, 0x9C
005F74E9  3E:9B       wait                                 Superfluous prefix
ss:[0040661D]=00400000 (Themida_.00400000)
eax=00001004
```

Quedando así:

ADD 00001004, 400000 == 00401004 >> dirección de la IAT

A todo esto hay que decir que el valor sumado en la segunda operación es una CONSTANTE de las dos que utiliza Themida (en programas con Función Virtual utiliza 6 de estas constantes, dos para la función normal y cuatro para la virtual). La anotamos:

CONSTANTE2==481D5CA2

El procedimiento seguido queda:

rol campo_IAT, 5 >> resultado1
add resultado1, 481D5CA2 >> resultado2
add resultado2, ImageBase >> dirección de la IAT

Quedaría:

Dirección de la IAT=([{campo_IAT ROL 5} ADD 481D5CA2] ADD ImageBase)

En otros términos:

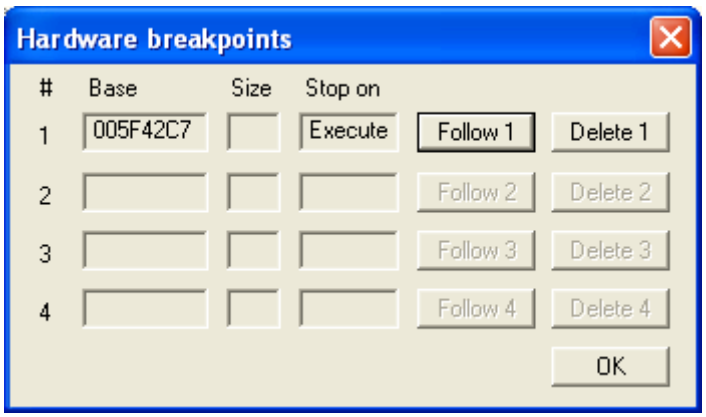
- ✓ dirección de la IAT=([{campo_IAT ROL 5} ADD CONSTANTE2] ADD [0x40661D]);

Después escribe en la IAT:

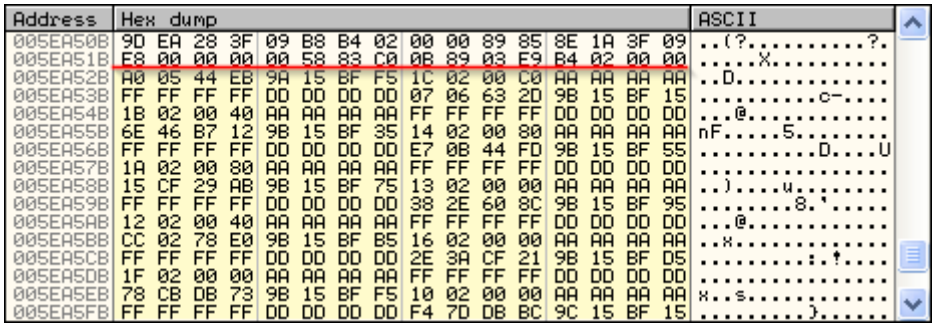
```
005F7514   51          push    ecx
005F7515   8F00        pop     dword ptr [eax]     ; // ** Escribe en la IAT
005F7517   60          pushad
005F7518   B1 F5       mov     cl, 0xF5
005F751A   61          popad
005F751B   AD          lodsd                       ; // ** lee de la tabla campo_JMP
```

Y lee el valor campo_JMP que es el valor encriptado en la Tabla para la dirección del salto, ya sabemos cómo desencriptarlo, así que continuemos.

Reiniciamos Olly <Ctrl+F2>, vamos a comenzar de nuevo la rutina para estudiar el campo_API que parece el más complicado de ellos, antes colocamos un HBP en la dirección del primer lodsb donde lee el campo_API [0x5F42C7h], será el único HBP que tengamos:



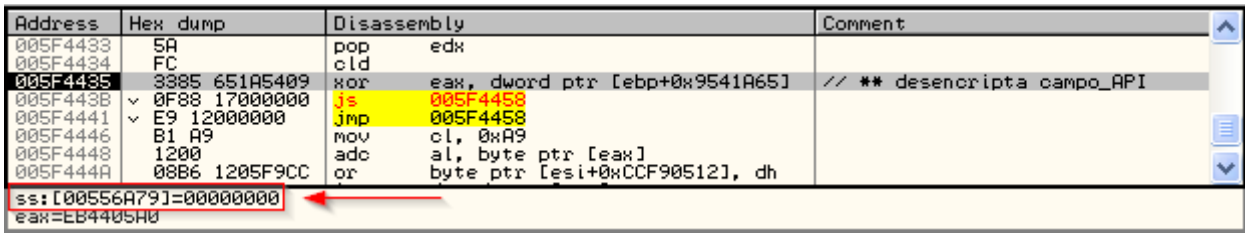
Damos RUN y Olly para en esta dirección, miremos la Tabla:



Está intacta, el primer valor que se lee es el campo_API del primer registro 0xEB4405A0h, quedará EAX==0xEB4405A0h que es el valor a vigilar, actualiza el puntero de tabla y después borra el campo:

```
005F42ED   89B5 E3545309   mov     dword ptr [ebp+0x95354E3], esi     ; // ** actualiza puntero Tabla
005F42F3   60              pushad
005F42F4   60              pushad
005F42F5   81CB AAAE4874   or      ebx, 0x7448AEAA
005F42FB   66:8BD6         mov     dx, si
005F42FE   61              popad
005F42FF   E9 10000000     jmp     005F4314
. . .
005F431C   810424 E336A33A add     dword ptr [esp], 0x3AA336E3
005F4323   8F46 FC         pop     dword ptr [esi-0x4]                ; 02700000
005F4326   816E FC E336A33>sub     dword ptr [esi-0x4], 0x3AA336E3    ; // ** Borra campo_API
005F432D   5B              pop     ebx                                ; 02700000
005F432E   0F82 14000000   jb      005F4348
```

Lo primero que hace es desencriptar en una operación XOR:



La llave de encriptación es [0x556A79h]==0x00h

XOR EB4405A0, 0 == EB4405A0

El valor de EAX no varía.

La siguiente operación que hace es esta:

```
Address   Hex dump     Disassembly                          Comment
005F4458    C1C8 03    ror     eax, 0x3                     // ** segunda operacion campo_API
005F445B  v 0F89 0B000000 jns   005F446C
005F4461  v E9 06000000 jmp     005F446C
005F4466  v 70 0C      jo      short 005F4474
005F4468    1AA5 2EE42DEF sbb   ah, byte ptr [ebp+0xEF2DE42E]
005F446E  ^ 74 F2      je      short 005F4462
005F4470    49         dec     ecx
005F4471    05 643BD44E add     eax, 0x4ED43B64
```

Quedando:

ROR EB4405A0, 3 == 1D6880B4

Después llegamos a:

```
Address   Hex dump     Disassembly                          Comment
005F446C    2D EF74F249 sub     eax, 0x49F274EF
005F4471    05 643BD44E add     eax, 0x4ED43B64
005F4476    29D0       sub     eax, edx                     // ** EDX==76AACDEB >> CONSTANTE1
005F4478    2D 643BD44E sub     eax, 0x4ED43B64
005F447D    05 EF74F249 add     eax, 0x49F274EF
005F4482    FC         cld
005F4483    C1C0 10    rol     eax, 0x10
005F4486  v E9 05000000 jmp     005F4490
edx=76AACDEB
eax=224A4729
```

Este es un punto importante porque en EDX está la CONSTANTE1 que anotamos

CONSTANTE1==76AACDEB

Esta constante y la anterior son fundamentales para la resolución de Themida. Hizo una resta:

SUB 1D6880B4, 76AACDEB == A6BDB2C9

Valor que utiliza justo después:

```
Address   Hex dump     Disassembly                          Comment
005F446C    2D EF74F249 sub     eax, 0x49F274EF
005F4471    05 643BD44E add     eax, 0x4ED43B64
005F4476    29D0       sub     eax, edx                     // ** EDX==76AACDEB >> CONSTANTE1
005F4478    2D 643BD44E sub     eax, 0x4ED43B64
005F447D    05 EF74F249 add     eax, 0x49F274EF
005F4482    FC         cld
005F4483    C1C0 10    rol     eax, 0x10                    // ** cuarta operacion campo_API
005F4486  v E9 05000000 jmp     005F4490
eax=A6BDB2C9
```

Hace un 'rol eax, 10' quedando:

ROL A6BDB2C9, 10 == B2C9A6BD

Continuamos traceando y llegamos a:

```
Address   Hex dump     Disassembly                                  Comment
005F4490  31C8         xor     eax, ecx
005F4492  60           pushad
005F4493  B7 B4        mov     bh, 0xB4
005F4495  E9 09000000  jmp     005F44A3
005F449A  D2A3 48A20672 shl    byte ptr [ebx+0x7206A248], cl
005F44A0  86F1         xchg    cl, dh
005F44A2  6D           insd                                         I/O command
005F44A3  61           popad
ecx=481D5CA2   <------ CONSTANTE2
eax=B2C9A6BD
```

Utiliza la CONSTANTE2 para desencriptar:

XOR B2C9A6BD, 481D5CA2 == FAD4FA1F

Y muy sutilmente ejecuta esta aparente inofensiva instrucción:

```
Address   Hex dump     Disassembly                                  Comment
005F4493  B7 B4        mov     bh, 0xB4
005F4495  E9 09000000  jmp     005F44A3
005F449A  D2A3 48A20672 shl    byte ptr [ebx+0x7206A248], cl
005F44A0  86F1         xchg    cl, dh
005F44A2  6D           insd                                         I/O command
005F44A3  61           popad
005F44A4  899D 651A5409 mov    dword ptr [ebp+0x9541A65], ebx       // ** cambia llave encriptacion
005F44AA  E9 0F000000  jmp     005F44BE
ebx=EB4405A0
ss:[00556A79]=00000000
```

Lo que acaba de hacer es cambiar la antigua llave de encriptación 0x00h que utilizó en la dirección 0x5F4435h para aplicar la primera operación al campo_API:

```
005F4433   5A              pop   edx
005F4434   FC              cld
005F4435   3385 651A5409   xor   eax, dword ptr [ebp+0x9541A65]    ; // ** desencripta campo_API
005F443B   0F88 17000000   js    005F4458
005F4441   E9 12000000     jmp   005F4458
```

Así que modifica la llave, la siguiente vez que pase por aquí (SEGUNDO REGISTRO de la Tabla) ésta no valdrá 0x00h sino este nuevo valor 0xEB4405A0h, si nos fijamos detenidamente vemos que es el campo_IAT del PRIMER REGISTRO de la Tabla con el que estamos trabajando. El packer ha borrado las huellas y lo ha puesto a cero:

```
Address   Hex dump                                          ASCII
005EA52B  00 00 00 00 9A 15 BF F5 1C 02 00 C0 AA AA AA AA  ................
005EA53B  FF FF FF FF DD DD DD DD 07 06 63 2D 9B 15 BF 15  ..........c-....
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA  nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..)....u........
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.`.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA5BB  CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA  ..x.............
005EA5CB  FF FF FF FF DD DD DD DD 2E 3A CF 21 9B 15 BF D5  ..........:.!....
```

Pero si miramos la captura inicial:

```
Address   Hex dump                                          ASCII
005EA50B  9D EA 28 3F 09 B8 B4 02 00 00 89 85 8E 1A 3F 09  ..(?..........?.
005EA51B  E9 00 00 00 00 58 83 C0 0B 89 03 E9 B4 02 00 00  .....X..........
005EA52B  A0 05 44 EB 9A 15 BF F5 1C 02 00 C0 AA AA AA AA  ..D.............
005EA53B  FF FF FF FF DD DD DD DD 07 06 63 2D 9B 15 BF 15  ..........c-....
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA  nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..)....u........
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.`.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA5BB  CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA  ..x.............
005EA5CB  FF FF FF FF DD DD DD DD 2E 3A CF 21 9B 15 BF D5  ..........:.!....
005EA5DB  1F 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5EB  78 CB DB 73 9B 15 BF F5 10 02 00 00 AA AA AA AA  x..s............
005EA5FB  FF FF FF FF DD DD DD DD F4 7D DB BC 9C 15 BF 15  ..........}.....
```

Allí vemos que el valor era 0xEB4405A0h, se ha convertido en la nueva llave de encriptación:

```
Address   Hex dump                                          ASCII
00556A79  A0 05 44 EB 3E 74 E9 D6 00 00 00 C1 E5 07 81 ED  ..D.>t..........
00556A89  D3 E3 C9 F0 31 EE 5D E9 F6 04 00 00 01 F3 E9 19  ....1.].........
00556A99  00 00 00 58 81 C7 6D 0B DB 58 5A 81 C7 04 00 00  ...X..m..XZ.....
00556AA9  00 33 3C 24 31 3C 24 E9 56 08 00 00 56 BE 48 2C  .3<$1<$.V...V.H,
00556AB9  72 63 57 50 B8 C0 61 FB 0F E9 BF FD FF FF 8B 3C  rcWP..a........<
00556AC9  24 E9 3A 04 00 00 BB 04 00 00 00 E9 2D 01 00 00  $.:.........-...
00556AD9  51 E9 DD FE FF FF F7 DF 87 3C 24 5F 51 55 68 79  Q........<$_QUhy
00556AE9  18 FB 59 8B 2C 24 81 C4 04 00 22 B8 58 00 04 00  ..Y.,$....".X...
00556AF9  00 0D EA 60 EF 55 C1 E0 07 E9 39 05 00 00 8B 24  ...`.U....9....$
00556B09  24 83 E9 01 0F 85 C2 FD FF FF FB 57 E9 15 03 00  $..........W....
00556B19  00 4E F7 D6 81 EE B8 5E 87 52 81 F6 3F FA 65 23  .N.....^.R..?.e#
```

Así que es variable, muy interesante.

Llegamos a una comparación:

```
005F44BE    3D 00500000    cmp    eax, 0x5000
005F44C3    0F83 11020000  jnb    005F46DA
005F44C9    FC             cld
005F44CA    813E BBBBBBBB  cmp    dword ptr [esi], 0xBBBBBBBB
005F44D0    0F85 04020000  jnz    005F46DA
```

Y después a un loop donde compara el valor actual de EAX==0xFAD4FA1Fh:

```
Address   Hex dump     Disassembly                          Comment
005F4740    3B02       cmp    eax, dword ptr [edx]
005F4742  v 0F84 D4000000 je   005F481C
005F4748  v 0F8F 0E000000 jg   005F475C
005F474E  v E9 09000000  jmp   005F475C
005F4753    A6         cmpsb
005F4754    F1         int1
005F4755    92         xchg   eax, edx
005F4756    F1         int1
005F4757    A1 E53F5E71 mov   eax, dword ptr [0x715E3FE5]
005F475C    FC         cld
005F475D    56         push   esi                          Themida_.005EA52F
005F475E    BE 04000000 mov   esi, 0x4
005F4763    81EA 486B426D sub edx, 0x6D426B48
005F4769    01F2       add    edx, esi                     Themida_.005EA52F
ds:[02D00000]=40FC5417
eax=FAD4FA1F
```

Está leyendo de la dirección 0x02D00000h donde hay estos valores:

```
Address   Hex dump                                          ASCII
02D00000  17 54 FC 40 71 11 10 56 41 A3 5A EB 69 17 8B 7A  .T.@q..VA.Z.i..z
02D00010  B7 24 85 77 5E 04 2B D7 5F 22 CD 7A B3 46 3D 40  .$.w^.+._".z.F=@
02D00020  6D 91 B4 1B 19 01 3C B2 85 BC 07 C0 8B 66 43 17  m.....<......fC.
02D00030  4F E7 49 6C C6 47 0E F2 89 F0 A6 77 82 BE 8D 97  O.Il.G.....w....
02D00040  96 32 37 F7 97 7F FB 00 5B 8A 59 9A C1 88 5D AB  .27.....[.Y...].
02D00050  24 97 FC A1 BC A0 7D 9A CB 6F A0 84 F7 17 C6 95  $.....}..o......
02D00060  61 7B 6A 2E B9 EF 98 5C 6F DE 55 E7 65 35 76 F5  a{j....\o.U.e5v.
02D00070  38 6E D7 BB 1B 82 E0 92 35 EF 56 B5 29 DB 74 FB  8n......5.V.).t.
02D00080  B5 53 B1 6E CD F3 64 44 4D CE 66 C3 00 FC 8E 25  .S.n..dDM.f....%
02D00090  C3 AF A3 0E 45 08 F9 58 49 47 4C F4 7B 33 5D 90  ....E..XIGL.{3].
02D000A0  93 6E 17 17 8A C2 73 AA 36 6D 0C 2A 9F FC 3F 07  .n....s.6m.*..?.
02D000B0  00 B7 F3 9E 3A 6E E5 E2 A2 A8 AA 7B FC 58 2E 34  ....:n.....{.X.4
02D000C0  3F 0B 03 1F B9 AC 59 49 B5 E3 EC E5 C3 F3 2B 46  ?.....YI......+F
02D000D0  ED 23 7B D2 6A 1A C1 C3 D6 B5 BE 43 7F 24 8D 6E  .#{.j......C.$.n
02D000E0  F8 E5 43 9F DF A2 A9 63 18 E3 36 84 0D DD 7A 29  ..C...@c..6...z)
02D000F0  BA 67 51 A2 A8 34 91 CB ED EC E5 ED 07 83 F1 07  .gQ..4..........
```

La tabla continúa hasta 0x02D009F0h. Lo que hará será buscar el valor de EAX entre los DWORDS de la Tabla, valor que a ojo que podemos encontrar aquí:

```
Address   Hex dump                                          ASCII
02D00120  39 6F 61 F2 F0 9B F2 59 F9 B8 2F 18 41 CF 55 05  9oa....Y../.A.U.
02D00130  94 7E 49 B0 63 8B 9E B7 17 F4 B4 B9 17 38 F3 AE  .~I.c........8.«
02D00140  F1 58 0D AA ED 79 C8 F5 4B 84 F4 46 1F FA D4 FA  .X...y..K..F....
02D00150  F5 43 FB ED 1C 3D 19 8C 2E BD 2C 51 69 57 47 C5  .C...=.....QiWG.
02D00160  2F D4 E8 C2 45 A2 E7 9C 56 1C 73 73 21 3C 09 BD  /...E...V.ss!<..
02D00170  FA 88 0B 83 7E 3A 1A 16 D1 72 CF 67 74 17 C1 DE  ....~:....r.gt...
02D00180  88 A6 9F 46 0F 17 61 90 81 88 AB CD 55 5C 41 9F  ...F..a.....U\A.
02D00190  8C 6C D1 A1 A8 68 F4 B7 D1 DF 0B 7C 4A 1D 93 C3  .l...h.....|J...
02D001A0  E3 8C A0 EE D1 00 B3 D7 E9 8E 2B 49 96 D7 FD FB  ..........+I....
```

Recapitulemos un segundo y anotemos qué ha hecho a partir del campo_API:

Inicialmente EAX==0xEB4405A0h

XOR EB4405A0, [0x556A79h]==0x00h == EB4405A0
ROR EB4405A0, 3 == 1D6880B4
SUB 1D6880B4, 76AACDEB == A6BDB2C9 >> Utiliza CONSTANTE1
ROL A6BDB2C9, 10 == B2C9A6BD
XOR B2C9A6BD, 481D5CA2 == FAD4FA1F >> Utiliza CONSTANTE2

Este es el valor que busca en la tabla, para referirnos a éste lo haremos como Semilla_IAT y a la tabla de 0x02D00000h como Tabla_Semillas. Lo que hace en el loop es buscar la Semilla_IAT [0xFAD4FA1Fh en este caso concreto] en la Tabla_Semillas, quedó ECX==0x53h y lo salva:

```
Address   Hex dump     Disassembly                          Comment
005F481C  898D 0A2A3F09 mov     dword ptr [ebp+0x93F2A0A], ecx  // ** salva posicion Semilla_IAT
005F4822  0F85 15000000 jnz     005F483D
005F4828  60            pushad
005F4829  E8 07000000   call    005F4835
005F482E  90            nop
005F482F  CA D583       retf    0x83D5                          Far return
005F4832  88E8          mov     al, ch
005F4834  305F 81       xor     byte ptr [edi-0x7F], bl
ecx=00000053
ss:[00407A1E]=0000027B
```

De dónde sale el 0x53h ? es la posición de la semilla en la tabla, tenemos:

Inicio Tabla_Semillas == 0x02D00000h
Dirección Semilla_IAT == 0x02D0014Ch
Desplazamiento == 0x02D0014Ch - 0x02D00000h >> 0x14Ch
En DWORDS == 0x14Ch DIV 4 >> 0x53h

Después hace unas comparaciones:

```
Address   Hex dump     Disassembly                          Comment
005F4857  8B8D D92B3F09 mov     ecx, dword ptr [ebp+0x93F2BD9]  // ** puntero a BASE de DLL
005F485D  8B09          mov     ecx, dword ptr [ecx]            // ** obtiene BASE DLL [MSVBVM
005F485F  3B8D B21C3F09 cmp     ecx, dword ptr [ebp+0x93F1CB2]  USER32.7E390000
005F4865  74 10         je      short 005F4877
005F4867  3B8D F6273F09 cmp     ecx, dword ptr [ebp+0x93F27F6]  ADVAPI32.77DA0000
005F486D  74 08         je      short 005F4877
005F486F  3B8D 5D0F3F09 cmp     ecx, dword ptr [ebp+0x93F0F5D]  kernel32.7C800000
005F4875  75 31         jnz     short 005F48A8
005F4877  C685 F0BD5D09 mov     byte ptr [ebp+0x95DBDF0], 0x1   // ** Virtualizamos..
005F487E  8D95 FDBD5D09 lea     edx, dword ptr [ebp+0x95DBDFD]
005F4884  8D8D D5BE5D09 lea     ecx, dword ptr [ebp+0x95DBED5]
005F488A  39CA          cmp     edx, ecx                        MSVBVM60.#1374
005F488C  73 1A         jnb     short 005F48A8
005F488E  833A 00       cmp     dword ptr [edx], 0x0
005F4891  74 15         je      short 005F48A8
005F4893  3902          cmp     dword ptr [edx], eax
005F4895  75 0C         jnz     short 005F48A3
005F4897  C785 F9BD5D09 mov     dword ptr [ebp+0x95DBDF9], 0x1  // ** Caso excepcional
005F48A1  EB 05         jmp     short 005F48A8
```

Obtiene la base de MSVBVM60.DLL [0x72940000h en mi máquina] y la compara con la base del kernel32.dll, user32.dll y advapi32.dll, si es la misma base activará el flag de Virtualización

```
005F4877  C685 F0BD5D09 0>mov     byte ptr [ebp+0x95DBDF0], 0x1      ; // ** Virtualizamos..
```

En este ejecutable no se utiliza pero este es el flag que indica que hay que tratar la API en la Función Virtual de Themida. En este caso [ebp+0x95DBDF0h] equivale a [0x5F0E04h] que lógicamente está a cero.

Tampoco se utiliza el flag de excepción:

```
005F4897  C785 F9BD5D09 0>mov     dword ptr [ebp+0x95DBDF9], 0x1      ; // ** Caso excepcional
```

Teniendo que [ebp+0x95DBDF9h] equivale a [0x5F0E0Dh] y también está a cero:

```
Address   Hex dump                                          ASCII
005F0DF1  DE 81 7C 2D FA 80 7C D0 27 83 7C 00 00 00 00 08  ..|-..|.'.|.....
005F0E01  7C 3A 7E 00 00 00 00 00 00 00 00 00 00 00 00 00  |:~..............
005F0E11  81 71 F5 40 C3 C5 FE 5B AF 06 8A 0D 93 D1 86 56  .q.@...[.......V
005F0E21  F7 19 7B 25 20 AC E3 39 80 24 57 6C 29 02 B8 7D  ..{% ..9.$Wl)..}
005F0E31  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E41  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E51  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E61  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E71  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E81  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E91  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Allí se ven los dos flags resaltados en verde a 0x00h, y lo que hay a continuación resaltado en color amarillo es una tablita con semillas_API. Lo que hace el packer es mirar si la semilla actual (que está en EAX) existe en esta tabla, si existe entonces activa el flag de excepción para que la API no se virtualice, será tratada aquí, en la función normal. Porque pasa esto?

- ✓ Si la DLL actual corresponde a kernel, user o advapi automáticamente se virtualizan sus funciones, las APIs de estas librerías serán tratadas en la Función Virtual, esto lo controla el primer flag [0x5F0E04h] que se pone a 0x01h cuando se trata de alguna de estas tres librerías.

- ✓ Si la DLL actual corresponde a kernel, user o advapi pero su Semilla_API existe en la tabla de excepciones 0x5F0E11h – 0x5F0E30h el flag de Virtualización [0x5F0E04h] se mantiene a 0x01h pero se activará el de excepción [0x5F0E0Dh] poniéndose a 0x01h y la API no se virtualiza.

Esta es la manera que tiene el packer de diferenciar las APIs que va a trabajar en la Función Virtual de las que lo harán aquí, en la Función Normal.

Como digo, en este ejecutable esto no sucede y TODAS las APIs serán tratadas aquí, en la Función Normal, pero conviene conocer el método del protector para futuros usos.

Continuamos traceando la rutina y llegamos a otra interesante zona:

```
Address   Hex dump     Disassembly                          Comment
005F4974    F7D2       not     edx
005F4976    81EA 4428FEFF  sub     edx, 0xFFFE2844
005F497C    01DA       add     edx, ebx
005F497E    8B0A       mov     ecx, dword ptr [edx]         // ** obtiene BASE DLL
005F4980    5A         pop     edx
005F4981  v 0F87 0B000000  ja      005F4992
005F4987  v E9 06000000    jmp     005F4992
005F498C    313E       xor     dword ptr [esi], edi
```

lo que hace es obtener la BASE de la DLL actual de una TABLA:

```
Address   Hex dump                                          ASCII
02700000  00 00 94 72 00 00 00 00 00 00 00 00 00 00 00 00  ...r............
02700010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

En este caso sólo está la de MSVBVM60.DLL pero suele haber más para ejecutables que importen más librerías.

Esta dirección la anotamos también para después:

- ✓ dirección Base_DLL == 0x02700000h

Vuelve a obtener la posición de la Semilla_IAT en la Tabla_Semillas que guardó antes:

```
Address   Hex dump        Disassembly                          Comment
005F4992    8B85 0A2A3F09 mov     eax, dword ptr [ebp+0x93F2A0A] // ** lee posicion semilla_IAT
005F4998  v 0F82 0D000000 jb      005F49AB
005F499E  v E9 08000000   jmp     005F49AB
005F49A3    3F            aas
005F49A4    66:6B91 4FE02A imul   dx, word ptr [ecx+0x12AE04F], 0x
005F49AC  v E0 0F         loopdne short 005F49BD
005F49AE    870C00        xchg    dword ptr [eax+eax], ecx       MSVBVM60.#1374
ss:[00407A1E]=00000053
eax=FAD4FA1F
```

Dejando EAX==0x00000053h y comienza a hacer operaciones con este valor:

```
Address   Hex dump        Disassembly                          Comment
005F49AB    D1E0          shl     eax, 1                       // ** opera con posicion_semilla
005F49AD  v 0F87 0C000000 ja      005F49BF
005F49B3  v 0F81 06000000 jno     005F49BF
005F49B9  v 0F85 00000000 jnz     005F49BF
005F49BF  v E9 07000000   jmp     005F49CB
005F49C4    1A90 BED521AD sbb     dl, byte ptr [eax+0xAD21D5BE]
005F49CA    D7            xlat    byte ptr [ebx+al]
eax=00000053
```

Anotémoslo también:

SHL 53, 1 == 000000A6

A este resultado le agrega un valor que en un principio puede costar de identificar:

```
Address   Hex dump        Disassembly                          Comment
005F49CA    D7            xlat    byte ptr [ebx+al]
005F49CB    0385 1F545D09 add     eax, dword ptr [ebp+0x95D541F] // ** suma AddressOfNameOrdinals
005F49D1    60            pushad
005F49D2  v E9 09000000   jmp     005F49E0
005F49D7    8D5D 14       lea     ebx, dword ptr [ebp+0x14]
005F49DA    9B            wait
005F49DB    93            xchg    eax, ebx
ss:[005EA433]=729655A0 (MSVBVM60.729655A0)
eax=000000A6
```

Le añade 0x729655A0h.

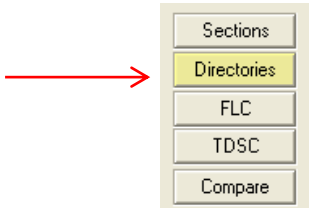
Si a este valor le restamos la ImageBase de la DLL actual [MSVBVM60.DLL] queda:

0x729655A0h - 0x72940000 == 0x255A0h

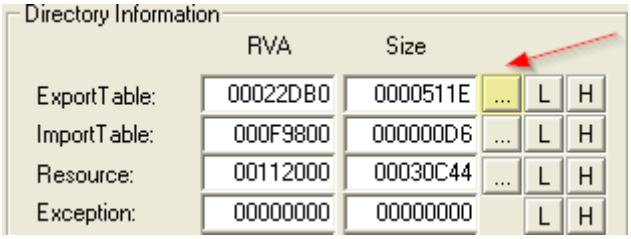
Abramos LordPE Deluxe (o cualquier otro programa que permita acceder a la cabecera de un archivo). Cargamos la librería MSVBVM60.DLL que en mi máquina está en esta ruta:

`c:\WINDOWS\system32\MSVBVM60.DLL`

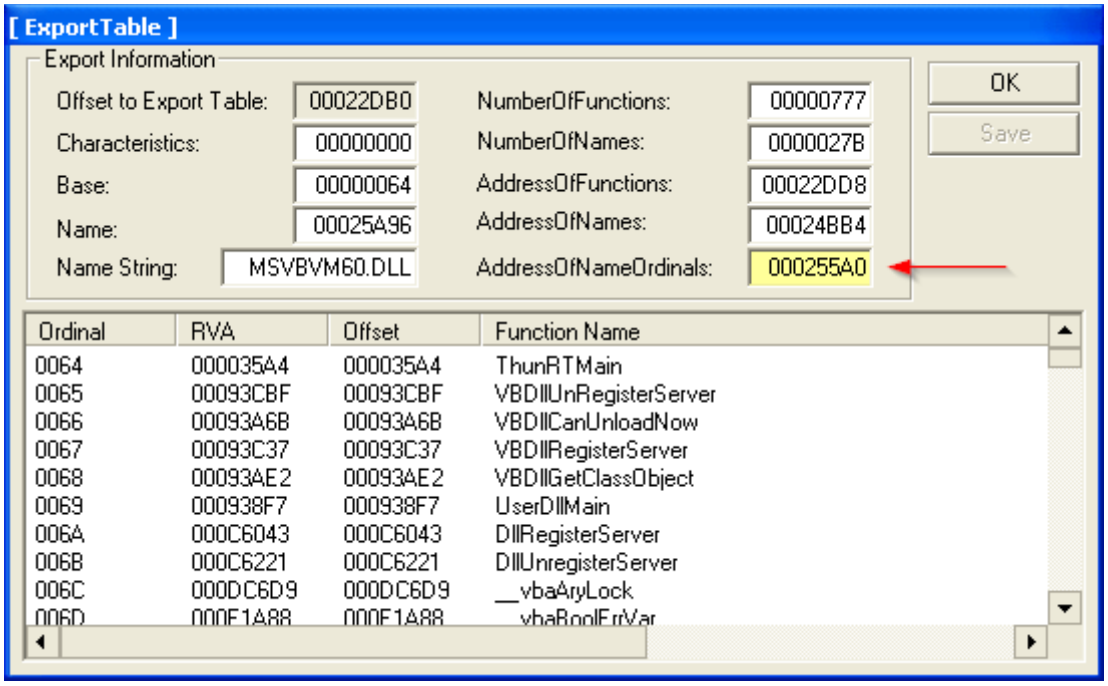
Una vez cargada desde el botón 'PE Editor' tenemos que pulsar el botón 'Directories':



Nos dirigimos a la Tabla de Exportación desde este botoncito:



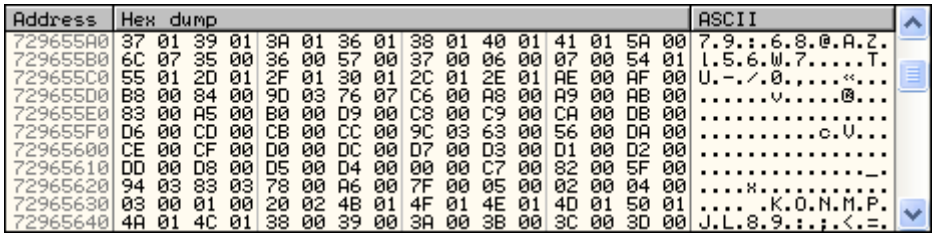
Y se abrirá este cuadro con TODA la información acerca de esta TABLA:



Encontramos el valor que buscamos en el campo AddressOfNameOrdinals:

0x729655A0h - 0x72940000 == 0x255A0h

Para llegar al inicio de ella hay que agregarle el ImageBase de la DLL y este resultado está en la resta anterior 0x729655A0h, vamos allí en el Olly:



A este valor se le suma el 0x000000A6 obtenido:

0x729655A0h + 0x000000A6 == 72965646

Miramos en Olly y encontramos esto:

```
Address   Hex dump                                          ASCII
72965646  39 00 3A 00 3B 00 3C 00 3D 00 3E 00 3F 00 41 00  9.:.;.<.=.>.?.A.
72965656  40 00 42 00 43 00 08 00 44 00 45 00 46 00 47 00  @.B.C...D.E.F.G.
72965666  48 00 49 00 11 00 09 00 4A 00 4B 00 4C 00 4D 00  H.I.....J.K.L.M.
72965676  4E 00 4F 00 50 00 51 00 52 00 53 00 54 00 55 00  N.O.P.Q.R.S.T.U.
72965686  58 00 59 00 5B 00 5C 00 5D 00 5E 00 60 00 61 00  X.Y.[.\.].^.`.a.
72965696  62 00 64 00 65 00 66 00 67 00 68 00 69 00 6A 00  b.d.e.f.g.h.i.j.
729656A6  6B 00 6C 00 6D 00 6E 00 6F 00 70 00 71 00 72 00  k.l.m.n.o.p.q.r.
729656B6  73 00 74 00 75 00 76 00 77 00 79 00 7A 00 7B 00  s.t.u.v.w.y.z.{.
729656C6  7C 00 7D 00 7E 00 80 00 81 00 85 00 86 00 87 00  |.}.~............
729656D6  88 00 89 00 8A 00 8B 00 8C 00 8D 00 8E 00 12 00  ................
729656E6  13 00 14 00 15 00 16 00 17 00 18 00 19 00 1A 00  ................
```

Valor que lee justo después:

```
Address   Hex dump   Disassembly                  Comment
005F4A20  31F0       xor     eax, esi             // ** intercambia
005F4A22  31C6       xor     esi, eax             // ** ESI y
005F4A24  31F0       xor     eax, esi             // ** EAX
005F4A26  F5         cmc
005F4A27  66:AD      lodsw                        // ** lee un WORD
005F4A29  60         pushad
ds:[esi]=[72965646]=0039
ax=0000
```

Quedando EAX==0x00000039h

Hace una operación con este valor:

```
Address   Hex dump   Disassembly                  Comment
005F4A2B  80DB 24    sbb     bl, 0x24
005F4A2E  61         popad
005F4A2F  C1E0 02    shl     eax, 0x2             // ** operacion logica
005F4A32  E9 05000000 jmp    005F4A3C
005F4A37  F9         stc
005F4A38  1F         pop     ds                   Modification of segment register
```

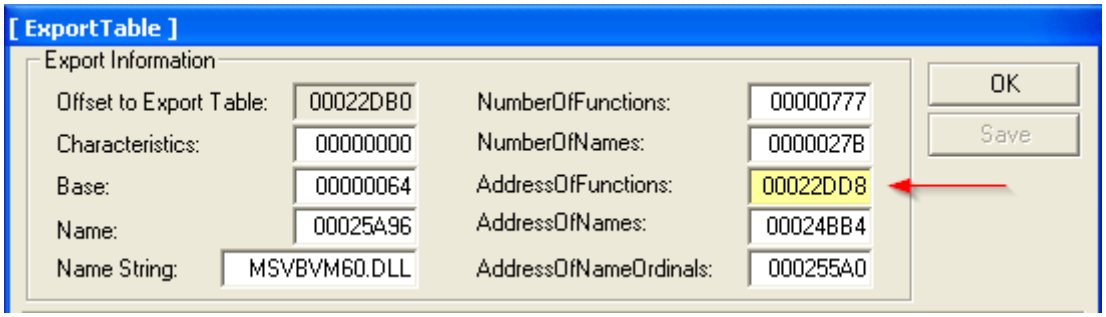
Quedando:

SHL 39, 2 == 000000E4

A este valor se le agrega el AddressOfFunctions de la DLL después:

```
Address   Hex dump   Disassembly                  Comment
005F4A3C  0385 03F85209 add  eax, dword ptr [ebp+0x952F803]  // ** agrega AddressOfFunctions
005F4A42  FC         cld
005F4A43  31F0       xor     eax, esi             MSVBVM60.72965648
005F4A45  31C6       xor     esi, eax
005F4A47  31F0       xor     eax, esi             MSVBVM60.72965648
005F4A49  0F88 08000000 js   005F4A57
ss:[00544817]=72962DD8 (MSVBVM60.72962DD8)
eax=000000E4
```

Lo podemos comprobar con LordPE:



Quedando EAX==0x72962EBCh

Vayamos allí con el Olly:

```
Address   Hex dump                                          ASCII
72962EBC  86 93 0F 00 11 DD 0E 00 2B 94 0F 00 EE 94 0F 00  ........+.......
72962ECC  93 95 0F 00 44 96 0F 00 6A 68 0D 00 AE C6 0D 00  ....D...jh..«...
72962EDC  94 C6 0D 00 0D C2 0C 00 FE C1 0D 00 44 C2 0C 00  ............D...
72962EEC  83 C0 0D 00 7B C2 0C 00 F8 C2 0C 00 FC C6 0D 00  ....{...........
72962EFC  40 39 10 00 ED 0F 0E 00 73 0D 0E 00 85 71 10 00  @9......s....q..
72962F0C  46 C1 0D 00 ED 5D 10 00 7B A1 0D 00 09 96 0D 00  F....]..{.......
72962F1C  56 96 0D 00 EA 62 0F 00 F3 A0 0D 00 18 A1 0D 00  V....b..........
72962F2C  33 98 0E 00 5D D0 0F 00 FD A0 00 00 6A 97 0E 00  3...]....."...j...
72962F3C  36 4E 0E 00 49 B7 0D 00 C9 96 0E 00 10 94 10 00  6N..I...........
72962F4C  38 94 10 00 9F 19 0E 00 97 EA 0C 00 B1 19 0E 00  8...............
72962F5C  F9 96 0E 00 94 97 0E 00 68 3B 10 00 74 96 0E 00  ........h;..t...
```

Podemos encontrar el desplazamiento [offset] de la función dentro de la librería, valor que lee enseguida:

```
Address   Hex dump     Disassembly                          Comment
005F4A54    60         pushad
005F4A55    61         popad
005F4A56    61         popad
005F4A57    AD         lodsd                                // ** lee offset API en la DLL
005F4A58    0F8A 01000000  jpe     005F4A5F
005F4A5E    F8         clc
005F4A5F    05 8340DE1A add     eax, 0x1ADE4083
005F4A64    01C8       add     eax, ecx                     MSVBVM60.#1374
ds:[esi]=[72962EBC]=000F9386
eax=72965648
```

Quedando EAX==0x000F9386h.

A este valor le agrega el ImageBase de MSVBVM60.DLL aquí:

```
Address   Hex dump     Disassembly                          Comment
005F4A5F    05 8340DE1A add     eax, 0x1ADE4083
005F4A64    01C8       add     eax, ecx                     // ** ImageBase DLL
005F4A66    57         push    edi
005F4A67    BF 8340DE1A mov     edi, 0x1ADE4083
005F4A6C    29F8       sub     eax, edi
005F4A6E    5F         pop     edi                          // ** API en EAX
005F4A6F    60         pushad
005F4A70    8AC7       mov     al, bh
```

En este momento los registros están así:

EAX 000F9386
ECX 72940000 offset MSVBVM60.#1374
EDX 76AACDEB
EBX 02700000

Quedando:

000F9386 + 72940000 == 72A39386 [MSVBVM60._CIcos]

Veamos los registros:

EAX 72A39386 MSVBVM60._CIcos
ECX 72940000 offset MSVBVM60.#1374

Lo que hizo fue obtener la dirección de la API a través de la tabla de exportación de la DLL [sin saltar a GetProcAddress], muy agudo !

Desde que leyó el campo_API hasta aquí se cumple un nuevo ciclo, a partir de aquí pasará a escribir la API en 0x02CE0000h, agregarle código polimórfico hacia 0x02CF0000h y finalmente pasarla a la sección definitiva 0x02Dxxxxxh.

Hecho esto pasará a escribir en la IAT el valor tipo 0x02Dxxxxxh obtenido y después pasará a reajustar el 'JMP 02Dxxxxx' en la tabla de saltos.

Pero lo que realmente nos interesa saber es si la siguiente API utilizará como llave encriptadora el primer campo de este registro 0xEB4405A0h que salvó en la dirección 0x556a79h:

```
Address   Hex dump                                          ASCII
00556A79  A0 05 44 EB 3E 74 E9 D6 00 00 00 C1 E5 07 81 ED  ..D.>t..........
00556A89  D3 E3 C9 F0 31 EE 5D E9 F6 04 00 00 01 F3 E9 19  ....1.].........
00556A99  00 00 00 58 81 C7 6D 0B DB 58 5A 81 C7 04 00 00  ...X..m..XZ.....
00556AA9  00 33 3C 24 31 3C 24 E9 56 08 00 00 56 BE 48 2C  .3<$1<$.V...V.H,
00556AB9  72 63 57 50 B8 C0 61 FB 0F E9 BF FD FF FF 8B 3C  rcWP..a........<
00556AC9  24 E9 3A 04 00 00 BB 04 00 00 00 E9 2D 01 00 00  $.:.........-...
00556AD9  51 E9 DD FE FF FF F7 DF 87 3C 24 5F 51 55 68 79  Q........<$_QUhy
00556AE9  18 FB 59 8B 2C 24 81 C4 04 00 22 B8 58 00 04 00  ..Y.,$....".X...
00556AF9  00 0D EA 60 EF 55 C1 E0 07 E9 39 05 00 00 8B 24  ...`.U....9....$
00556B09  24 83 E9 01 0F 85 C2 FD FF FF FB 57 E9 15 03 00  $..........W....
```

Lleguemos nuevamente al punto donde se lee el campo_API para el siguiente REGISTRO:

```
Address   Hex dump     Disassembly                          Comment
005F42C7    AD         lodsd                                // ** lee TABLA campo_API
005F42C8  v 0F8C 1F000000 jl      005F42ED
005F42CE  v 0F83 19000000 jnb     005F42ED
005F42D4  v E9 14000000   jmp     005F42ED
005F42D9  ^ 74 BC         je      short 005F4297
005F42DB    1187 95ED00DC adc     dword ptr [edi+0xDC00ED95], eax
005F42E1    34 06         xor     al, 0x6
ds:[esi]=[005EA543]=2D630607
eax=FFFFFFFF
```

Veamos cómo está la Tabla:

```
Address   Hex dump                                          ASCII
005EA52B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005EA53B  00 00 00 00 00 00 00 00 07 06 63 2D 9B 15 BF 15  ..........c-....
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA  nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..)....u........
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.`.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA5BB  CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA  ..x.............
005EA5CB  FF FF FF FF DD DD DD DD 2E 3A CF 21 9B 15 BF D5  ..........:.!....
005EA5DB  1F 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA5EB  78 CB DB 73 9B 15 BF F5 10 02 00 00 AA AA AA AA  x..s............
```

Borró todo el registro anterior, el siguiente comienza en 0x5EA543h.

Por si lo necesitáramos recordemos que el puntero de tabla lo tenemos en [0x54A4F7h]

Llegamos a la desencriptación, veamos qué llave utiliza:

```
Address   Hex dump     Disassembly                          Comment
005F4433    5A         pop     edx
005F4434    FC         cld
005F4435    3385 651A5409 xor     eax, dword ptr [ebp+0x9541A65]  // ** desencripta campo_API
005F443B  v 0F88 17000000 js      005F4458
005F4441  v E9 12000000   jmp     005F4458
005F4446    B1 A9         mov     cl, 0xA9
005F4448    1200          adc     al, byte ptr [eax]
005F444A    08B6 1205F9CC or      byte ptr [esi+0xCCF90512], dh
ss:[00556A79]=EB4405A0
eax=2D630607
```

Bingo!

Realiza la siguiente operación:

XOR 2D630607, EB4405A0 == C62703A7

Después de las restantes operaciones de rigor:

XOR 2D630607, [0x556A79h]==0xEB4405A0h == C62703A7
ROR C62703A7, 3 == F8C4E074
SUB F8C4E074, 76AACDEB == 821A1289 >> Utiliza CONSTANTE1
ROL 821A1289, 10 == 1289821A
XOR 1289821A, 481D5CA2 == 5A94DEB8 >> Utiliza CONSTANTE2

Queda: Semilla_IAT == 0x5A94DEB8h

Instala la nueva llave de encriptación 0x2D630607:

```
Address   Hex dump     Disassembly                          Comment
005F4493    B7 B4      mov     bh, 0xB4
005F4495  v E9 09000000 jmp    005F44A3
005F449A    D2A3 48A20672 shl  byte ptr [ebx+0x7206A248], cl
005F44A0    86F1       xchg    cl, dh
005F44A2    6D         insd                                 I/O command
005F44A3    61         popad
005F44A4    899D 651A5409 mov  dword ptr [ebp+0x9541A65], ebx  // ** cambia llave encriptacion
005F44AA  v E9 0F000000 jmp    005F44BE
ebx=2D630607
ss:[00556A79]=EB4405A0
```

Continúa buscando la Semilla_API en la Tabla_Semillas obteniendo ECX==0x000001B3h y le aplica las operaciones que ya conocemos:

SHL 1B3, 1 == 00000366
ADD 00000366, 729655A0 == 72965906 >> AddressOfNameOrdinals
MOV EAX, WORD [72965906] == 00000273
SHL 273, 2 == 000009CC
ADD 000009CC, 72962DD8 == 729637A4 >> AddressOfFunctions
MOV EAX, DWORD [729637A4] == 000F09F9
ADD 0F09F9, 72940000 [Base DLL] == 72A309F9 [**MSVBVM60._adj_fptan**]

En los registros queda la API en EAX:

EAX 72A309F9 MSVBVM60._adj_fptan
ECX 72940000 offset MSVBVM60.#1374
EDX 76AACDEB
EBX 02700000

Como hicimos con la campo_IAT y campo_JMP vamos a comprobar la teoría, busquemos el siguiente registro de la tabla:

```
Address   Hex dump                                         ASCII
005EA52B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005EA53B  00 00 00 00 00 00 00 00 00 00 00 00 9B 15 BF 15  ................
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA  nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..)....u........
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.`.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA5BB  CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA  ..x.............
005EA5CB  FF FF FF FF DD DD DD DD 2E 3A CF 21 9B 15 BF D5  .........:.!....
005EA5DB  1F 02 00 00 AA AA AA AA FF FF FF FF DD DD DD DD  ................
```

El primer campo es el que nos interesa, sería el siguiente en ser procesado, nos adelantaremos al packer.

Llave actual de encriptación:

```
00556A79  07 06 63 2D 3E 74 E9 D6 00 00 00 C1 E5 07 81 ED  ..c->t..........
00556A89  D3 E3 C9 F0 31 EE 5D E9 F6 04 00 00 01 F3 E9 19  ....1.].........
```

Que es el primer campo del registro anterior [con el que estábamos trabajando].

Trataremos de seguir los pasos del packer a ver si obtenemos la dirección de la API cuyo valor es 0x12B746E6h encriptado en la Tabla. Como sabemos las operaciones que se seguirán es cuestión de irlas aplicando una por una:

- **campo_API** == 0x12B746E6

OPERACIÓN	RESULTADO
XOR 12B7466E, [0x556A79h]==0x2D630607h	== 3FD44069
ROR 3FD44069, 3	== 27FA880D
SUB 27FA880D, 76AACDEB	== B14FBA22 >> Utiliza CONSTANTE1
ROL B14FBA22, 10	== BA22B14F
XOR BA22B14F, 481D5CA2	== F23FEDED >> Utiliza CONSTANTE2
Posición_Semilla en Tabla_Semillas	== 1AA
SHL 1AA, 1	== 00000354
ADD 00000354, [0x5EA433h]==0x729655A0h	== 729658F4 >> AddressOfNameOrdinals
EAX == WORD [729658F4]	== 0000026A
SHL 26A, 2	== 000009A8
ADD 000009A8, [0x544817h]==0x72962DD8h	== 72963780 >> AddressOfFunctions
EAX == DWORD [72963780]	== 000F02BA
ADD 0F02BA, [0x02700000]==0x72940000h	== 72A302BA [MSVBVM60._adj_fdiv_m64

Genial!. Hemos pasado de 0x12B746E6h a 0x72A302BAh [MSVBVM60._adj_fdiv_m64]

Si seguimos traceando la rutina llegaremos al punto donde carga el campo_API:

```
005F42C7    AD              lodsd                   ; // ** lee TABLA campo_API
005F42C8    0F8C 1F000000   jl      005F42ED
005F42CE    0F83 19000000   jnb     005F42ED
005F42D4    E9 14000000     jmp     005F42ED
```

para este REGISTRO:

```
Address   Hex dump                                         ASCII
005EA753  10 A8 05 60 BB BB BB BB 9D 15 BF D5 24 02 00 40  ...`........$..@
005EA763  AA AA AA AA FF FF FF FF DD DD DD DD B2 4E 9B 1C  .............N..
005EA773  9D 15 BF F5 1C 02 00 00 AA AA AA AA FF FF FF FF  ................
005EA783  DD DD DD DD 58 16 6D 23 9E 15 BF 15 21 02 00 40  ....X.m#....!..@
005EA793  AA AA AA AA FF FF FF FF DD DD DD DD 5F D1 AF DA  ............_...
005EA7A3  9E 15 BF 35 20 02 00 80 AA AA AA AA FF FF FF FF  ...5 ...........
005EA7B3  DD DD DD DD D3 4F BC 6A 9E 15 BF 55 1D 02 00 80  .....O.j...U....
005EA7C3  AA AA AA AA FF FF FF FF DD DD DD DD EE EE EE EE  ................
005EA7D3  DD DD DD DD 00 00 00 00 00 00 00 00 EB 0C D1 2E  ................
005EA7E3  CA 91 04 88 C1 A8 FC 40 17 CB E9 12 12 00 00 E9  ........@.......
005EA7F3  91 06 00 00 00 00 E8 01 3E 00 6B 00 B5 03 5D 00  ........>.k...].
005EA803  1F 00 96 00 2F 00 3E 00 DD BC 54 DF 5D 00 5C 2A  ..../.>...T.].\*
```

Vemos una diferencia respecto a los otros y es que tiene un campo de más [7 campos], la razón son esas BBBBBBBB del segundo campo, recordemos el orden:

```
campo_API, campo_IAT, campo_JMP
```

Este registro es distinto:

```
campo_API, BBBBBBBB, campo_IAT, campo_JMP
```

Averigüemos por qué estas B's.

Traceamos la rutina y vemos que al llegar al punto donde aplica la 4ª operación al campo_API:

```
005F4482    FC              cld
005F4483    C1C0 10         rol     eax, 0x10       ; // ** cuarta operación campo_API
005F4486    E9 05000000     jmp     005F4490
```

Queda este resultado:

EAX==0x00000064h

Muy inferior para los valores de EAX vistos hasta ahora en este punto, seguimos traceando y encontramos una segunda diferencia, ahora NO SALTA el JNB que antes sí lo hacía:

```
Address   Hex dump     Disassembly                          Comment
005F44BE    3D 00500000   cmp     eax, 0x5000
005F44C3  v 0F83 11020000 jnb    005F46DA
005F44C9    FC            cld
005F44CA    813E BBBBBBBB cmp     dword ptr [esi], 0xBBBBBBBB
005F44D0  v 0F85 04020000 jnz    005F46DA
005F44D6    60            pushad
005F44D7    57            push    edi                      Themida_.004010AF
005F44D8  v 0F82 00000000 jb     005F44DE
005F44DE    5E            pop     esi                      Themida_.005EA757
ds:[005EA757]=BBBBBBBB
```

A continuación compara el siguiente campo con 0xBBBBBBBBh, en este caso coinciden y el JNZ siguiente NO SE PRODUCE.

A continuación borra las B's de la Tabla y actualiza el puntero a ésta [0x5EA75Bh], después le resta el valor 0x64h en esta instrucción:

```
Address   Hex dump     Disassembly                          Comment
005F45A2    60            pushad
005F45A3    BB D2EA3B17   mov     ebx, 0x173BEAD2
005F45A8    61            popad
005F45A9    2B85 7BE55609 sub     eax, dword ptr [ebp+0x956E57B]  // ** resta [0x58358Fh]==0x64h
005F45AF  v E9 0B000000   jmp     005F45BF
005F45B4    47            inc     edi                      Themida_.004010AF
005F45B5    52            push    edx
005F45B6    19C5          sbb     ebp, eax
ss:[0058358F]=00000064
eax=00000064
```

Lee de [0x58358Fh], quedando EAX==0x00h:

SUB 64, 64 == 0

Después le aplica otra operación al resultado:

```
Address   Hex dump     Disassembly                          Comment
005F45BF    C1E0 02       shl     eax, 0x2                 // ** segunda operacion Semilla_API
005F45C2  v 0F8C 10000000 jl     005F45D8
005F45C8  v E9 0B000000   jmp     005F45D8
005F45CD    006D 86       add     byte ptr [ebp-0x7A], ch
005F45D0    42            inc     edx
005F45D1    0016          add     byte ptr [esi], dl
005F45D3    1A31          sbb     dh, byte ptr [ecx]
005F45D5    8B46 05       mov     eax, dword ptr [esi+0x5]
```

Donde hace:

SHL 0, 2 == 0

A este resultado le agrega el AddressOfFunctions de la DLL:

```
Address   Hex dump     Disassembly                          Comment
005F45D0    42         inc     edx
005F45D1    0016       add     byte ptr [esi], dl
005F45D3    1A31       sbb     dh, byte ptr [ecx]
005F45D5    8B46 05    mov     eax, dword ptr [esi+0x5]
005F45D8    0385 03F85209  add  eax, dword ptr [ebp+0x952F803]  // ** agrega AddressOfFunctions
005F45DE    FC         cld
005F45DF    31F0       xor     eax, esi                         Themida_.005EA75B
005F45E1    31C6       xor     esi, eax
ss:[00544817]=72962DD8 (MSVBVM60.72962DD8)
eax=00000000
```

Lee de [0x544817h] y quedará:

ADD 0, 72962DD8

Estos resultados van quedando en EAX, en este momento EAX==0x72962DD8h

Leerá este DWORD justo después:

```
Address   Hex dump     Disassembly                          Comment
005F45E6  v 0F82 06000000  jb      005F45F2
005F45EC    0F8A 00000000  jpe     005F45F2
005F45F2    8BC3       mov     eax, ebx
005F45F4    61         popad
005F45F5    AD         lodsd                                // ** obtiene AddressOfFunction
005F45F6    F5         cmc
005F45F7    FC         cld
005F45F8    2D 46394877    sub     eax, 0x77483946
ds:[esi]=[72962DD8]=000035A4
eax=005EA75B
```

Quedando EAX==0x000035A4h, hizo:

EAX == DWORD [72962DD8] == 35A4

A este valor le agrega el ImageBase de la DLL:

```
Address   Hex dump     Disassembly                          Comment
005F45F7    FC         cld
005F45F8    2D 46394877    sub     eax, 0x77483946
005F45FD    2D 8C6B5F69    sub     eax, 0x695F6B8C
005F4602    01C8       add     eax, ecx                     // ** agrega ImageBase DLL
005F4604    05 8C6B5F69    add     eax, 0x695F6B8C
005F4609    05 46394877    add     eax, 0x77483946
005F460E  v E9 14000000    jmp     005F4627                 // ** API caso ESPECIAL
005F4613    3A7D 10    cmp     bh, byte ptr [ebp+0x10]
```

En este momento los registros están así:

EAX 000035A4
ECX 72940000 offset MSVBVM60.#1374
EDX 00000064
EBX 02700000

Tras la suma queda en EAX la API:

```
EAX 729435A4 MSVBVM60.ThunRTMain
ECX 72940000 offset MSVBVM60.#1374
EDX 00000064
EBX 02700000
```

Si miramos en la tabla de exportación de MSVBVM60.DLL encontramos la API:

Ordinal	RVA	Offset	Function Name
0064	000035A4	000035A4	ThunRTMain
0065	00093CBF	00093CBF	VBDllUnRegisterServer
0066	00093A6B	00093A6B	VBDllCanUnloadNow
0067	00093C37	00093C37	VBDllRegisterServer
0068	00093AE2	00093AE2	VBDllGetClassObject
0069	000938F7	000938F7	UserDllMain

Cuyo Ordinal es igual a 0x64h, su Offset 0x35A4h que es el valor obtenido en Olly en 0x5F45F5h

A partir de aquí se va a leer el campo_IAT para obtener la dirección de la IAT donde salvar la API. Este caso es especial y de ahí las BBBBBBBB en la Tabla.

Para resolver estas API el procedimiento es exactamente el mismo que para el caso general hasta la obtención de la Semilla_API, a partir de aquí cambia si esta tiene un valor inferior a 0x5000h [este caso Semilla_API==0x64h] y además el siguiente campo de la Tabla es BBBBBBBB se tratará de manera especial. Esto hay que tenerlo en cuenta para el script.

En caso de encontrar un caso especial habrá que hacer:

```
SUB Semilla_API, [0x58358Fh]   == valor1
SHL valor1, 2                  == valor2
ADD valor2, [0x544817h]        == valor3    >> Agrega AddressOfFunctions
MOV EAX, DWORD [valor3]        == valor4
ADD EAX, [0x02700000]          == API       >> Agrega ImageBase
```

La razón de utilizar las direcciones de memoria en lugar del valor contenido en ella directamente es para no perder generalidad en el procedimiento, el script seguramente lo queramos reutilizar con otros Themida y quizá necesitemos tratar con más de una DLL, por esto accedemos a su valor a través de la dirección de memoria y no directamente a su valor. Podríamos hacer esto:

```
SUB Semilla_API, 64            == valor1
SHL valor1, 2                  == valor2
ADD valor2, 0x72962DD8h        == valor3    >> Agrega AddressOfFunctions
MOV EAX, DWORD [valor3]        == valor4
ADD EAX, 0x72940000h           == API       >> Agrega ImageBase
```

Pero qué pasará cuando cambie de DLL ? nuestro script seguiría trabajando con el AddressOfFunctions y el ImageBase de MSVBVM60.DLL y no es correcto. Lo mismo sucedería en caso de trabajar en una máquina distinta donde no tienen por qué coincidir las direcciones de las API's.

Accediendo a la dirección de memoria nos aseguramos de estar actualizados respecto a estos cambios y ampliar el umbral de compatibilidad.

El procedimiento para descifrar una API tipo especial [2º campo BBBBBBBB] será el siguiente:

OPERACIÓN	RESULTADO
XOR campo_API, [0x556A79h]	== valor1
ROR valor1, 3	== valor2
SUB valor2, 76AACDEB	== valor3 >> Utiliza CONSTANTE1
ROL valor3, 10	== valor4
XOR valor4, 481D5CA2	== **Semilla_API** >> Utiliza CONSTANTE2
SUB Semilla_API, [0x58358Fh]	== valor5
SHL valor5, 2	== valor6
ADD valor6, [0x544817h]	== valor7 >> AddressOfFunctions
MOV EAX, DWORD [valor7]	== valor8
ADD valor8, [0x02700000]	== API

El comportamiento varía a partir de la obtención de la Semilla_API, en lugar de buscarla en la Tabla_Semillas le resta el valor de la dirección [0x58358Fh] que coincide con el del primer Ordinal de la tabla de Exportación de la Librería (0x64h en este caso), tampoco utiliza el campo AddressOfNameOrdinals. Todo esto lo tendremos en cuenta.

Si seguimos traceando la rutina del packer no encontramos más diferencias.

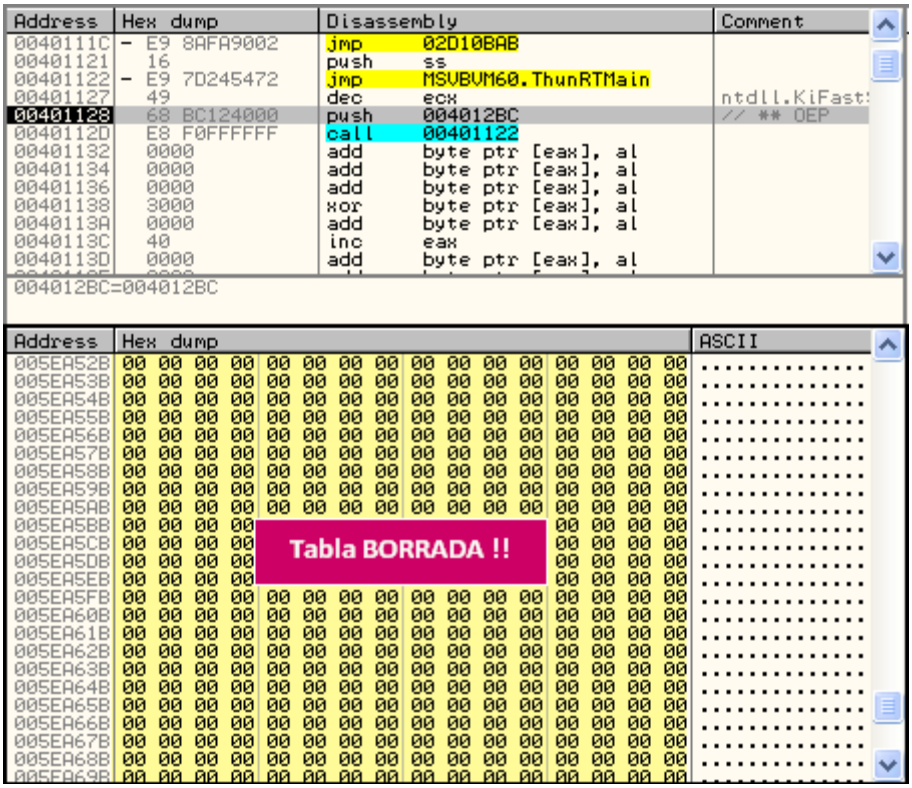
Podemos, pues pasar a escribir el script.

7. Encontrando el camino

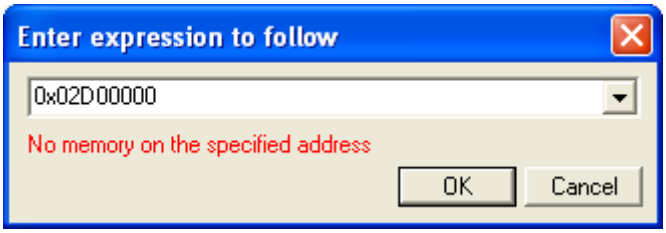
Antes de ponernos como locos a escribir el script vamos a pulir unos detalles, recordemos a qué direcciones tendremos que acceder:

[0x5EA52Bh]	- 0x2B0h bytes	>> Tabla Themida
[0x556A79h]		>> Llave encriptación [no accederemos]
[0x02D00000h]	- 0x9ECh bytes	>> Tabla_Semillas
[0x02700000h]	- 0x04 bytes	>> Base DLL
[0x5EA433h]		>> AddressOfNameOrdinals
[0x544817h]		>> AddressOfFunctions
[0x40661Dh]		>> Base EXE
[0x58358Fh]		>> valor 0x64h
[0x407BA2h]		>> valor de reajuste CASO ESPECIAL [no utilizado]

Estos son los puntos de acceso para nuestro script, de aquí deberá consultar los valores que vaya necesitando, todo perfecto, pero lleguemos al OEP miremos por ejemplo la Tabla del packer:



La borró conforme fue leyendo de ella, intentemos por ejemplo ir a la Tabla_Semillas:



El bloque de memoria fue liberado y ni tan si quiera existe, no hay tabla de Semillas tampoco.

Miremos la dirección donde se guarda la BASE de la DLL:

```
Address   Hex dump                                          ASCII
02700000  00 00 00 00 81 3A 4B 27 00 00 00 00 00 00 00 00  .....:K'......
02700010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700060  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700070  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
02700090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
027000A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
027000B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

También la borró.

La información restante está en su sitio. No es tan grave, lo que haremos será salvaguardar esta información ANTES de que la destruya y ponerla a buen recaudo para utilizarla después.

Una primera aproximación podría ser llegar a la rutina de la IAT y parar justo cuando va a leer el primer valor de la Tabla:

```
005F42C7    AD              lodsd                   ; // ** lee TABLA campo_API
005F42C8    0F8C 1F000000   jl      005F42ED
005F42CE    0F83 19000000   jnb     005F42ED
005F42D4    E9 14000000     jmp     005F42ED
```

Aquí tendríamos la Tabla intacta así como el resto de información.

La idea es salvar esta información:

[0x5EA52Bh]	- 0x2B0h bytes	>> Tabla Themida
[0x02D00000h]	- 0x9ECh bytes	>> Tabla_Semillas
[0x02700000h]	- 0x04 bytes	>> Base DLL

El mismo script podría hacerlo con el comando dm (dump Memory):

ODbgScript > Command Groups > OUTPUT >

DM

DM addr, size, file

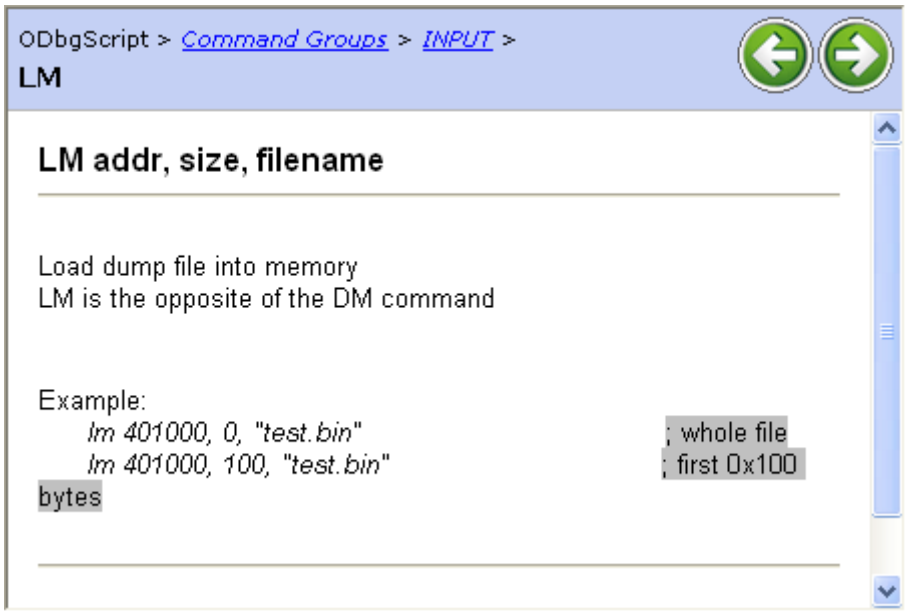
Dumps memory of specified size from specified address to specified file (default path set from opened app.)

Example:
dm 401000, 1F, "c:\dump.bin"

CHM by Cozofdeath

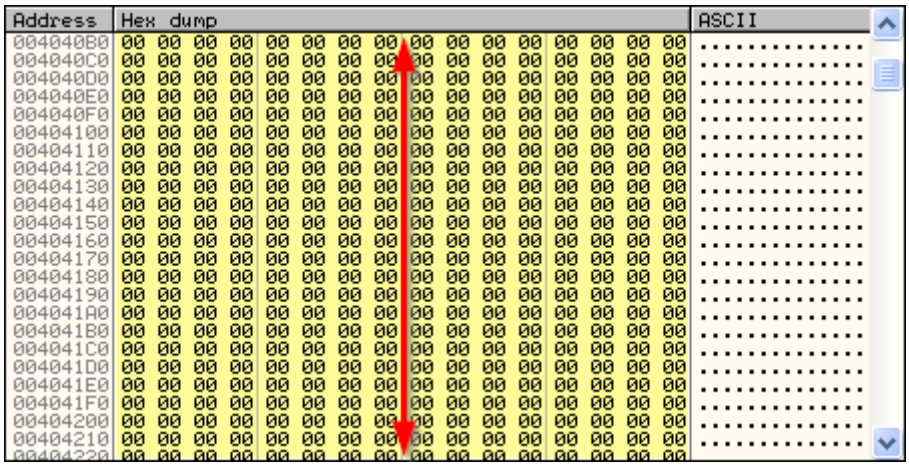
Podríamos dumpear los bloques de memoria que nos interesan al disco, dejar que el packer haga sus destrozos, llegar al OEP, recuperar la información (sobre todo las tablas) y recuperar el código.

Con el script podremos también cargar el contenido de los archivos salvados en memoria con el comando lm (load Memory) como podemos comprobar:



Nos va como anillo al dedo.

Ya que estamos la tabla de semillas la vamos a reubicar, la cargaremos en memoria del propio ejecutable y nuestro script leerá de allí, busquemos espacio libre en el EXE:



Aquí habrá de sobra y ubicaremos la Tabla_Semillas [0x9ECh], el comando para salvar la tabla será:

```
dm 02D00000, 9EC, "Tabla_Semillas.bin"
```

Y para recuperarla:

```
lm 4040B0, 0, "Tabla_Semillas.bin"
```

Para comprobar que el método puede funcionar haremos una prueba antes de continuar, escribiremos un pequeño script que lanzaremos desde el EP, colocará dos HBP's, uno en la dirección del lodsb [0x5F42C7h] donde tendremos la tabla intacta y otro en el OEP.

Cuando pare revisará si está en el lodsb, si es así salvará la Tabla como "Tabla_Semillas.bin" y borrará el HBP en esta instrucción [ya no queremos volver a parar aquí], la siguiente vez que pare será en el OEP donde restaurará la Tabla a partir del archivo salvado "Tabla_Semillas.bin" a partir de 0x4040B0h:

```
1   var oep
2   var lodsb
3
4   mov oep, 401128
5   mov lodsb, 5F42C7
6
7   bphws oep, "x"
8   bphws lodsb, "x"
9
10  inicio:
11  eob rompe
12  run
13
14  rompe:
15  cmp eip, lodsb
16  je SalvaTabla
17  cmp eip, oep
18  je Termina
19  jmp inicio
20
21  SalvaTabla:
22  dm 02D00000, 9EC, "Tabla_Semillas.bin"
23  bphwc lodsb
24  jmp inicio
25
26  Termina:
27  lm 4040B0, 0, "Tabla_Semillas.bin"
28  ret
```

Lo ejecutamos desde el EP del packer sin HBP's ni BP's puestos y cuando termine miramos si funcionó:

```
Address   Hex dump                                          ASCII
00404090  00 3B 40 00 00 00 00 00 00 00 00 00 00 00 00 00  .;@.............
004040A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004040B0  17 54 FC 40 71 11 10 56 41 A3 5A EB 69 17 8B 7A  .T.@q..VA.Z.i..
004040C0  B7 24 85 77 5E 04 2B D7 5F 22 CD 7A B3 46 3D 40  .$.w^.+._".z.F=
004040D0  6D 91 B4 1B 19 01 3C B2 85 BC 07 C0 8B 66 43 17  m.....<.......f
004040E0  4F E7 49 6C C6 47 0E F2 89 F0 A6 77 82 BE 8D 97  O.Il.G.....w...
004040F0  96 32 37 F7 97 7F FB 00 5B 8A 59 9A C1 88 5D AB  .27.....[.Y...
00404100  24 97 FC A1 BC A0 7D 9A CB 6F A0 84 F7 17 C6 95  $.....}..o....
00404110  61 7B 6A 2E B9 EF 98 5C 6F DE 55 E7 65 35 76 F5  a{j....\o.U.e5
00404120  38 6E D7 BB 1B 82 E0 92 35 EF 56 B5 29 DB 74 FB  8n......5.V.).
00404130  B5 53 B1 6E CD F3 64 44 4D CE 66 C3 00 FC 8E 25  .S.n..dDM.f...
00404140  C3 AF A3 0E 45 08 F9 58 49 47 4C F4 7B 33 5D 90  ....E..XIGL.{3
00404150  93 6E 17 17 8A C2 73 AA 36 6D 0C 2A 9F FC 3F 07  .n....s.6m.*..
00404160  00 B7 F3 9E 3A 6E E5 E2 A2 A8 AA 7B FC 58 2E 34  ....:n.....{.X
```

Ahí está la tabla copiada donde esperábamos. Perfecto !

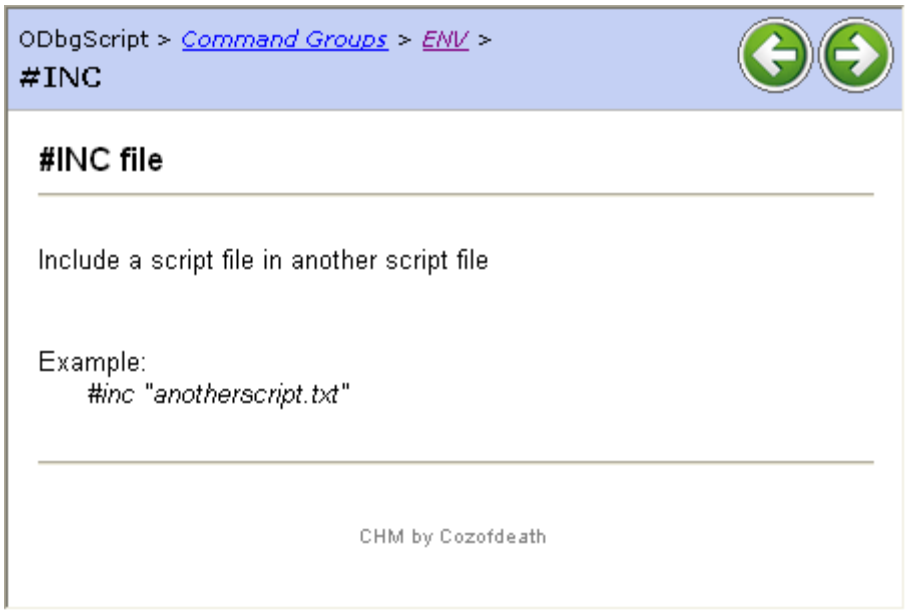
Ampliaremos el script para que salve también la Tabla de la IAT y la base de la DLL (esto último se podría hacer a mano pero para esto precisamente están los scripts):

```
21  SalvaTabla:
22  dm 02D00000, 9EC, "Tabla_Semillas.bin"
23  dm 5EA52B, 2B0, "Tabla_IAT.bin"
24  dm 02700000, 4, "Base_DLL.bin"
25  bphwc lodsb
26  jmp inicio
27
28  Termina:
29  lm 4040B0, 0, "Tabla_Semillas.bin"
30  lm 5EA52B, 0, "Tabla_IAT.bin"
31  lm 02700000, 4, "Base_DLL.bin"
32  ret
```

Lo probamos y funciona. Lo salvamos como "Al_OEP.osc"

Tenemos una parte hecha. Lo siguiente es una vez llegados al OEP con las tablas en su sitio comenzar el descifrado de los REGISTROS (Campo por Campo) y actuar en función de los valores encontrados. Como es una estructura no habrá problemas al ir recorriéndola y creo que funcionará bien.

Como me gusta hacer las cosas por el lado 'largo' vamos a crear dos scripts, el primero ya lo tenemos y es el antes mostrado "Al_OEP.osc", con él llegamos al OEP con las tablas en su sitio, no necesita nada más, pero este script será cargado desde el script principal, el comando es el siguiente:



Desde el primer script haremos:

```
#inc "Al_OEP.osc"
```

Lo he comprobado y funciona, lo que ocurre es que al llegar al RET del script "Al_OEP.osc" finalizan ambos scripts y no nos interesa, una manera que he encontrado de solucionarlo es quitando el RET en "Al_OEP.osc" y funciona bien.

El proceso se hace más largo pero a su vez el código queda más claro, este script recibirá un parámetro que será la dirección del OEP, el paso de parámetros habitualmente se realiza de la siguiente manera:

```
// ** Código principal
push parametro2
push parametro1
call Rutina
```

Y el código de la función 'Rutina' debe recoger éstos en el propio cuerpo de la función:

```
// ** Código de Rutina
var local1
var local2

pop local1   ; parametro1
pop local2   ; parametro2
```

El paso de parámetros se hace invertido al trabajar con la pila [push + pop) tipo LIFO (*Last-In First-Out*, el último en llegar será el primero en salir), por esto se recogen en orden inverso:

```
push parametroN
push parametroN-1
. . .
push parametro2
push parametro1
```

Y se recogerán en este orden:

```
pop local1   ; parametro1
pop local2   ; parametro2
. . .
pop localN-1 ; parametroN-1
pop localN   ; parametroN
```

Normalmente las variables locales NO SON VISIBLES desde el EXTERIOR del bloque en el que se declaran, por lo que no podríamos hacer esto:

```
// ** Código principal
push parametro2
push parametro1
call Rutina
mov eax, Local1    ; técnicamente ilegal (aceptado en OllyScript)
```

La variable Local1 sólo debería ser visible dentro del bloque en el que se declara (ámbito de la variable o *scope*) así como en sus bloques interiores (o sub-funciones), pero esta regla no parece afectar a los scripts de Olly por lo que he podido comprobar ya que las variables son visibles en ambos sentidos (de dentro a fuera y de fuera a dentro).

En cualquier caso le pasaremos el valor del OEP como parámetro a "Al_OEP.osc" aunque en realidad se podría acceder por el nombre de la variable que le hayamos dado en el script principal. Resumiendo, los scripts interiores pueden utilizar las variables declaradas en scripts exteriores y viceversa.

Finalmente dejamos el script así:

```
1  var lodsb
2  mov lodsb, 5F42C7
3
4  var Local1
5  pop Local1 ; OEP
6
7  bphws Local1, "x"
8  bphws lodsb, "x"
9
10 inicio:
11 eob rompe
12 run
13
14 rompe:
15 cmp eip, lodsb
16 je SalvaTabla
17 cmp eip, oep
18 je Termina
19 jmp inicio
20
21 SalvaTabla:
22 dm 02D00000, 9EC, "Tabla_Semillas.bin"
23 dm 5EA52B, 2B0, "Tabla_IAT.bin"
24 dm 02700000, 4, "Base_DLL.bin"
25 bphwc lodsb
26 jmp inicio
27
28 Termina:
29 lm 4040B0, 0, "Tabla_Semillas.bin"
30 lm 5EA52B, 0, "Tabla_IAT.bin"
31 lm 02700000, 4, "Base_DLL.bin"
32 ;ret
```

Nos falta el script principal, ´que irá recorriendo la Tabla de la IAT, utilizaremos una variable para la dirección de inicio de la tabla y una auxiliar que actuará como puntero de tabla, que mantendrá en todo momento la dirección del campo de la tabla que esté siendo tratado.

El tema de la llave de encriptación la manejará el script también, declararemos dos variables, una para la llave de encriptación actual y otra para la llave anterior, inicialmente ambas tendrán valor 0 y a medida que vayamos avanzando se actualizarán.

Se obtendrá el valor SEMILLA para el campo_API, se revisará el campo siguiente y en función si hay BBBBBBBB o no se ejecutará el caso general o el especial.

Las operaciones para obtener esta SEMILLA las tenemos del punto anterior 'La Tabla de Themida', así como las operaciones para el caso General y especial.

Asimismo se irán desencriptando el resto de campos del REGISTRO, cuando los tengamos todos (campo_API, campo_IAT y campo_JMP) saltaremos a la rutina Repara que recibe estos tres parámetros, como ya se ha explicado anteriormente sería posible utilizar las variables globales sin paso de parámetros, pero prefiero hacerlo así por motivos de estilo, pues si siempre lo hago así cuando esté escribiendo un programa en otro lenguaje no habré perdido el hábito, es muy recomendable crear hábitos y sobretodo mantenerlos (no perderlos).

La rutina Repara sencillamente toma los parámetros, salva la API en la IAT y ensambla un JMP en la dirección que toque (primer parámetro) a la entrada de la IAT que corresponda.

Entremedio haremos llamadas a la rutina BuscaSemilla que recibe una semilla como parámetro y retornará la posición de ésta en la Tabla_Semillas, si sucede algún error y no la encuentra activa el flag de error ($RESULT a -0x1h) y el script detendrá su ejecución (no tiene sentido continuar)

Cada campo tiene su rutina de desencriptación excepto campo_API que se resuelve en el cuerpo del loop:

- ✓ campo_API >> Se resuelve en el cuerpo principal del loop.
- ✓ campo_IAT >> Rutina desencriptaIAT
- ✓ campo_JMP >> Rutina desencriptaJMP

El script sería el siguiente:

```
1  var inicioTabla
2  var punteroTabla
3  var valorTabla
4  var llave_encriptacion
5  var antigua_llave
6  var addressOfOrdinals
7  var addressOfFunctions
8  var constante1
9  var constante2
10 var imageBase
11 var base_dll
12 var temp
13 var api
14 var direccionIAT
15 var direccionJMP
16 var sub64
17 var tabla_semillas
18 var contador_registros
19
20 mov inicioTabla, 5EA52B
21 mov llave_encriptacion, 0
22 mov antigua_llave, 0
23 mov base_dll, 02700000
24 mov addressOfOrdinals, 5EA433
25 mov addressOfFunctions, 544817
26 mov constante1, 76AACDEB
27 mov constante2, 481D5CA2
28 mov sub64, 0058358F
29 mov imageBase, 40661d
30 mov tabla_semillas, 4040B0
```

```
31 mov contador_registros, 0
32
33 lc
34 lclr
35
36 var oep
37 mov oep, 401128
38
39 //////// ****** LLEGAMOS AL OEP
40 push oep
41 #inc "Al_OEP.osc"
42
43 mov punteroTabla, inicioTabla
44 Log "///////////// **********                ****************"
45 Log "///////////// ********** Inicio SCRIPT ****************"
46 Log ""
47
48 loop:
49 mov valorTabla, [punteroTabla]
50 cmp valorTabla, EEEEEEEE
51 je termina
52 cmp valorTabla, AAAAAAAA
53 je sigue
54 cmp valorTabla, FFFFFFFF
55 je sigue
56 cmp valorTabla, DDDDDDDD
57 je sigue
58
59
60 inc contador_registros
61 eval "///////////// ********** Informacion REGISTRO 0x{contador_registros}h"
62 Log $RESULT
63
64
65 mov llave_encriptacion, antigua_llave
66 mov antigua_llave, valorTabla
67
68 xor valorTabla, llave_encriptacion
69 ror valorTabla, 3
70 sub valorTabla, constante1
71 rol valorTabla, 10
72 xor valorTabla, constante2
73
74 cmp [punteroTabla+4], BBBBBBBB
75 je casoEspecial
76
77 push valorTabla
78 call BuscaSemilla
79 cmp $RESULT, -1
80 je abortar        ; problemas
81
82 mov temp, $RESULT
83
84 shl temp, 1
85 add temp, [addressOfOrdinals]
86 mov temp, [temp]
```

```
87  and temp, 0000ffff
88  shl temp, 2
89  add temp, [addressOfFunctions]
90  mov temp, [temp]
91  add temp, [base_dll]
92  mov api, temp
93
94  retornoEspecial:
95  Log api
96
97  add punteroTabla, 4
98  mov valorTabla, [punteroTabla]
99  cmp valorTabla, BBBBBBBB
100 jne continua
101 add punteroTabla, 4
102 mov valorTabla, [punteroTabla]
103
104 continua:
105 push valorTabla
106 call desencriptaIAT
107 mov temp, $RESULT
108 mov direccionIAT, temp
109 Log direccionIAT
110
111 add punteroTabla, 4
112 mov valorTabla, [punteroTabla]
113 push valorTabla
114 call desencriptaJMP
115 mov temp, $RESULT
116 mov direccionJMP, temp
117 Log direccionJMP
118 Log ""
119
120 push api
121 push direccionIAT
122 push direccionJMP
123 call Repara
124
125 sigue:
126 add punteroTabla, 4
127 jmp loop
128
129 casoEspecial:
130 sub valorTabla, [sub64]
131 shl valorTabla, 2
132 add valorTabla, [addressOfFunctions]
133 mov valorTabla, [valorTabla]
134 add valorTabla, [base_dll]
135 mov api, valorTabla
136 gn api
137 mov nombreAPI, $RESULT
138 eval "Caso ESPECIAL {nombreAPI} en [0x{punteroTabla}h]"
139 Log $RESULT
140 jmp retornoEspecial
141
142
```

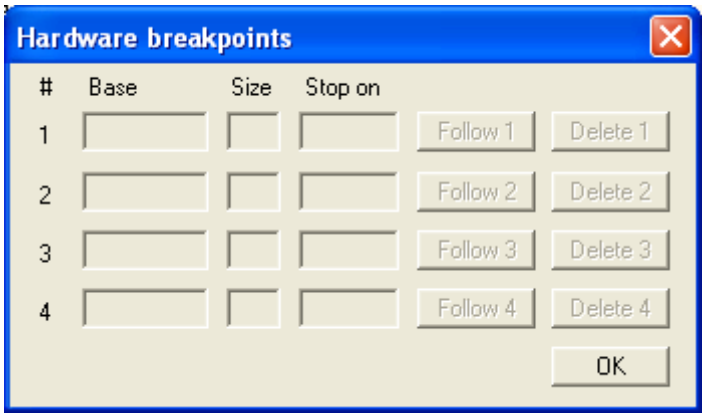
```
143 termina:
144 fill 4040B0, 9EC, 0
145 eval "Se Repararon 0x{contador_registros}h REGISTROS"
146 Log $RESULT
147 Log ""
148 MSG $RESULT
149 ret
150
151 abortar:
152 Log "Error de ejecución del script, abortando.."
153 ret
154
155 ////////////////// ******************
156 BuscaSemilla:
157 var Local1
158 var Local2
159 var Local3
160 var contador
161
162 pop Local1  ; semilla
163
164 mov contador, -1
165 var puntoInicio
166 mov puntoInicio, tabla_semillas
167 mov Local3, puntoInicio
168
169 loop_bs:
170 inc contador
171 mov Local2, [Local3]
172 cmp Local2, 0
173 je problemas
174 add Local3, 4
175 cmp Local2, Local1
176 jne loop_bs
177 jmp finaliza
178
179 problemas:
180 MSG "Problemas Reparando.."
181 mov $RESULT, -1
182 ret
183
184 finaliza:
185 mov $RESULT, contador
186 ret
187
188 ////////////////// ******************
189 desencriptaIAT:
190 var Local1
191 pop Local1  ; campo_IAT
192
193 rol Local1, 5
194 add Local1, constante2
195 add Local1, [imageBase]  ; add imageBase
196 mov $RESULT, Local1
197 ret
198
```

```
199 ////////////////// *****************
200  desencriptaJMP:
201  var Local1
202  pop Local1  ; campo_JMP
203
204  rol Local1, 3
205  add Local1, [imageBase] ; add imageBase
206  mov $RESULT, Local1
207  ret
208
209 ////////////////// *****************
210  Repara:
211  var Local1
212  pop Local1  ; direccionJMP
213  var Local2
214  pop Local2  ; direccionIAT
215  var Local3
216  pop Local3  ; api
217  mov [Local2], Local3
218  eval "jmp dword ptr [{Local2}]"
219  asm Local1, $RESULT
220  ret
```

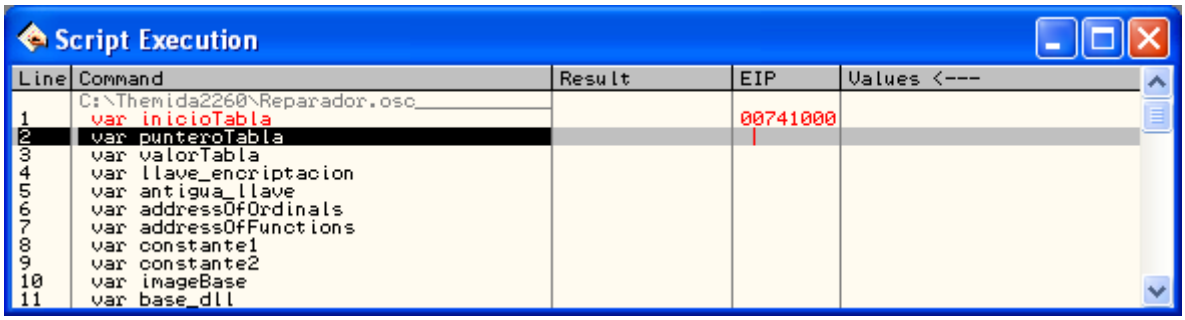
Lo salvamos como "Reparador.osc".

Sólo nos queda probarlo.

Reiniciamos Olly <Ctrl+F2> y deshabilitamos cualquier BP/HBP/BPM que hayamos puesto:



Vamos a la ventana de ODbgScript y cargamos Reparador.osc:



Si lo dejamos correr directamente todo pasa tan rápido que casi no se ve lo que hace, pararemos antes. Podríamos insertar un comando pause, pero así también funciona:

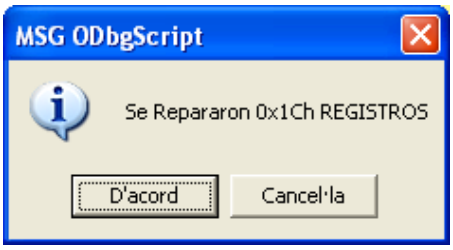
```
Line  Command                                                    Result   EIP      Values <---
73
74    mov punteroTabla, inicioTabla                                                  5EA52B
75    Log "////////////// **********                 ****************"
76    Log "////////////// ********** Inicio SCRIPT ****************"
77    Log ""
78
79   loop:
80    mov valorTabla, [punteroTabla]                             4     00401128   EB4405A0<5EA52B
81    cmp valorTabla, EEEEEEEE                                                      EB4405A0
82    je termina
83    cmp valorTabla, AAAAAAAA                                                      EB4405A0
84    je sigue
85    cmp valorTabla, FFFFFFFF                                                      EB4405A0
86    je sigue
87    cmp valorTabla, DDDDDDDD                                                      EB4405A0
88    je sigue
89
90    mov cnt, contador_registros
91    add cnt, 1
92    eval "////////////// ********** Informacion REGISTRO 0x{cnt}"
93    Log $RESULT
```

Nos situamos sobre la línea 90 y clicamos con el botón derecho del ratón, en el menú emergente elegimos la opción 'Run Until Cursor' (abreviadamente F4) y Olly parará ahí. Estamos en el OEP:

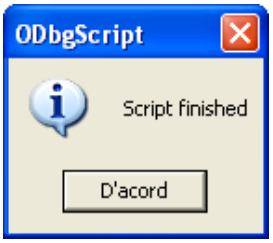
```
Address   Hex dump    Disassembly                          Comment
00401114    025E E9   add     bl, byte ptr [esi-0x17]
00401117    EE        out     dx, al                       I/O command
00401118    F8        clc
00401119    90        nop
0040111A    02D1      add     dl, cl
0040111C  - E9 99F99002 jmp   02D10ABA
00401121    08E9      or      cl, ch
00401123  v 7D 24     jge     short 00401149
00401125    54        push    esp
00401126  ^ 72 E9     jb      short 00401111
00401128    68 BC124000 push  004012BC                     // ** OEP
0040112D    E8 F0FFFFFF call  00401122
00401132    0000      add     byte ptr [eax], al
00401134    0000      add     byte ptr [eax], al
00401136    0000      add     byte ptr [eax], al
00401138    3000      xor     byte ptr [eax], al
0040113A    0000      add     byte ptr [eax], al
004012BC=004012BC

Address   Hex dump                                         ASCII
005EA52B  A0 05 44 EB 9A 15 BF F5 1C 02 00 C0 AA AA AA AA  ..D.............
005EA53B  FF FF FF FF DD DD DD DD 07 06 63 2D 9B 15 BF 15  ..........c-....
005EA54B  1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA55B  6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA  nF.....5........
005EA56B  FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55  ..........D....U
005EA57B  1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA58B  15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA  ..).....u.......
005EA59B  FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95  ........8.`.....
005EA5AB  12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD  ...@............
005EA5BB  CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA  ..x.............
005EA5CB  FF FF FF FF DD DD DD DD 2F 3A CF 21 9B 15 BF D5  ........:.+.....
```

La Tabla de la IAT está en su lugar nuevamente, también la Tabla_Semillas y la Base de la DLL, todo en orden, quitamos el BP con F2, dejamos el script correr libre y en unos segundos para:



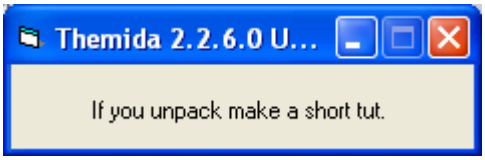
Después:



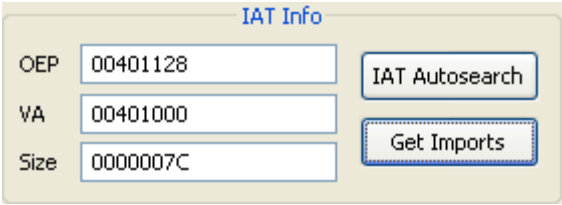
Veamos cómo quedó el código:

Address	Hex dump	Disassembly	Comment
00401081	25 20104000	and eax, 0x401020	
00401086	- FF25 38104000	jmp dword ptr [0x401038]	MSVBVM60.__vbaExceptHandler
0040108C	- FF25 44104000	jmp dword ptr [0x401044]	MSVBVM60.__vbaFPException
00401092	- FF25 14104000	jmp dword ptr [0x401014]	MSVBVM60._adj_fdiv_m16i
00401098	- FF25 10104000	jmp dword ptr [0x401010]	MSVBVM60._adj_fdiv_m32
0040109E	- FF25 4C104000	jmp dword ptr [0x40104C]	MSVBVM60._adj_fdiv_m32i
004010A4	- FF25 08104000	jmp dword ptr [0x401008]	MSVBVM60._adj_fdiv_m64
004010AA	- FF25 58104000	jmp dword ptr [0x401058]	MSVBVM60._adj_fdiv_r
004010B0	- FF25 18104000	jmp dword ptr [0x401018]	MSVBVM60._adj_fdivr_m16i
004010B6	- FF25 54104000	jmp dword ptr [0x401054]	MSVBVM60._adj_fdivr_m32
004010BC	- FF25 50104000	jmp dword ptr [0x401050]	MSVBVM60._adj_fdivr_m32i
004010C2	- FF25 40104000	jmp dword ptr [0x401040]	MSVBVM60._adj_fdivr_m64
004010C8	- FF25 28104000	jmp dword ptr [0x401028]	MSVBVM60._adj_fpatan
004010CE	- FF25 3C104000	jmp dword ptr [0x40103C]	MSVBVM60._adj_fprem
004010D4	- FF25 0C104000	jmp dword ptr [0x40100C]	MSVBVM60._adj_fprem1
004010DA	- FF25 04104000	jmp dword ptr [0x401004]	MSVBVM60._adj_fptan
004010E0	- FF25 60104000	jmp dword ptr [0x401060]	MSVBVM60._CIatan
004010E6	- FF25 00104000	jmp dword ptr [0x401000]	MSVBVM60._CIcos
004010EC	- FF25 6C104000	jmp dword ptr [0x40106C]	MSVBVM60._CIexp
004010F2	- FF25 48104000	jmp dword ptr [0x401048]	MSVBVM60._CIlog
004010F8	- FF25 1C104000	jmp dword ptr [0x40101C]	MSVBVM60._CIsin
004010FE	- FF25 30104000	jmp dword ptr [0x401030]	MSVBVM60._CIsqrt
00401104	- FF25 68104000	jmp dword ptr [0x401068]	MSVBVM60._CItan
0040110A	- FF25 64104000	jmp dword ptr [0x401064]	MSVBVM60._allmul
00401110	- FF25 34104000	jmp dword ptr [0x401034]	MSVBVM60.EVENT_SINK_QueryInterface
00401116	- FF25 24104000	jmp dword ptr [0x401024]	MSVBVM60.EVENT_SINK_AddRef
0040111C	- FF25 2C104000	jmp dword ptr [0x40102C]	MSVBVM60.EVENT_SINK_Release
00401122	- FF25 5C104000	jmp dword ptr [0x40105C]	MSVBVM60.ThunRTMain
00401128	68 BC124000	push 004012BC	// ** OEP
0040112D	E8 F0FFFFFF	call 00401122	jmp to MSVBVM60.ThunRTMain
00401132	0000	add byte ptr [eax], al	
00401134	0000	add byte ptr [eax], al	
00401136	0000	add byte ptr [eax], al	

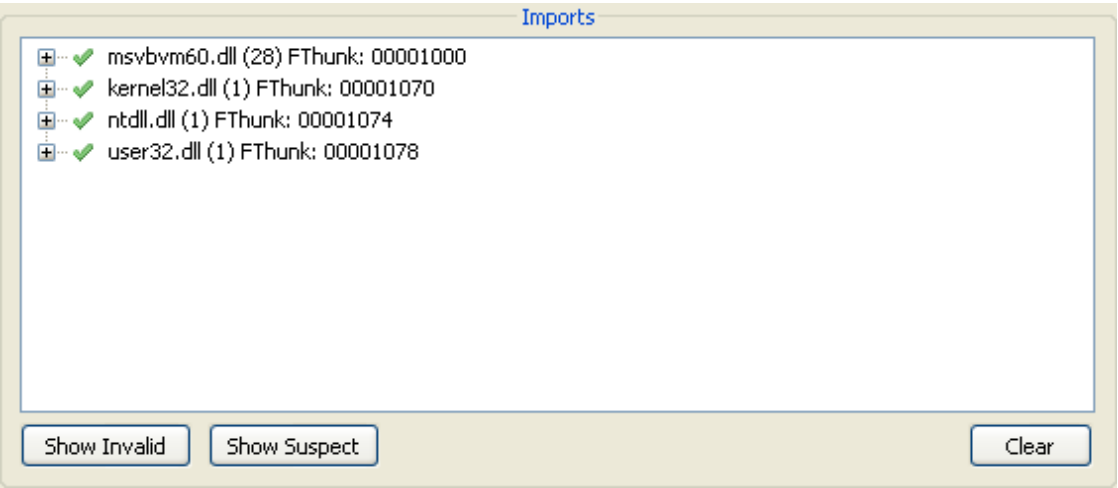
Daremos RUN <F9> a ver si corre..



Nos queda fijar los cambios. Reiniciemos Olly y volvemos a aplicar el script que nos dejará en el OEP, abrimos Scylla_x64 y atacheamos el proceso del unpackme. Completamos los valores para la IAT:

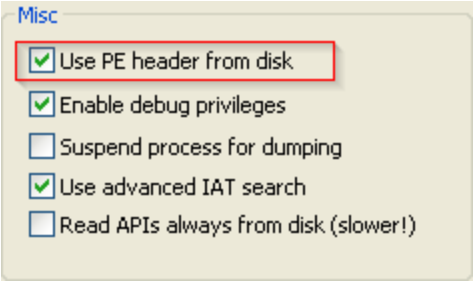


Pulsamos 'Get Imports' y se completa el listado de importaciones:



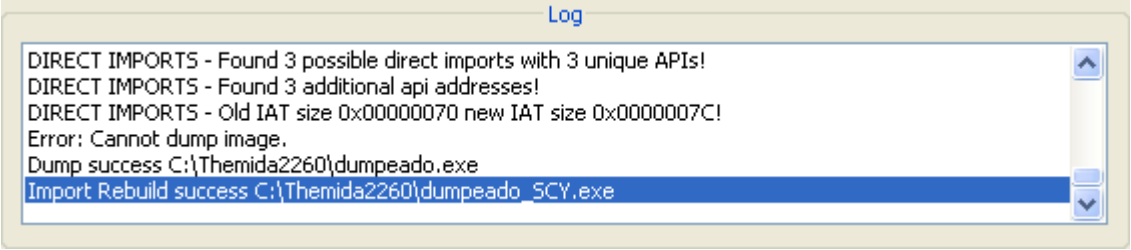
Pulsamos el botón 'Dump' y en lo salvamos como 'dumpeado.exe':

No puede! Vamos a 'Misc –> Options ' y activamos el checkbox 'Use PE header from disk':

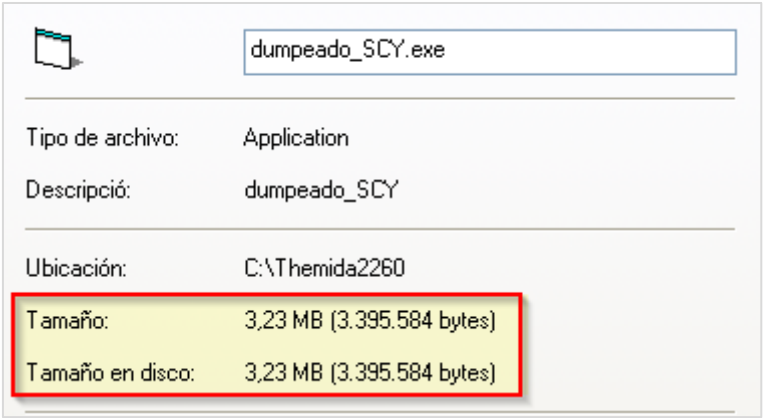


Y lo volvemos a intentar, esta vez nos da un dumpeado.

Desde el botón 'Fix Dump' elegimos el dumpeado.exe recién creado y lo repara:



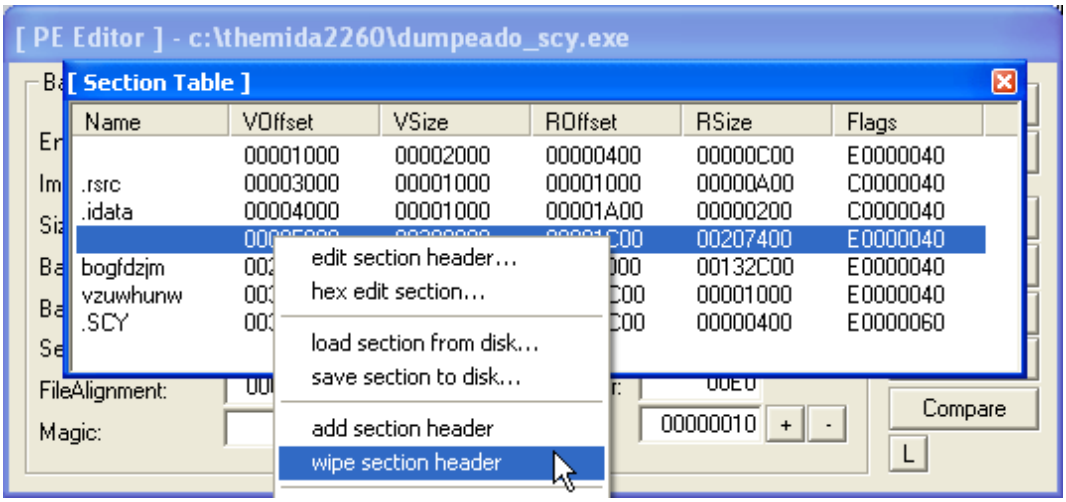
Si nos finamos vemos que el tamaño del archivo es excesivo para un unpackme:



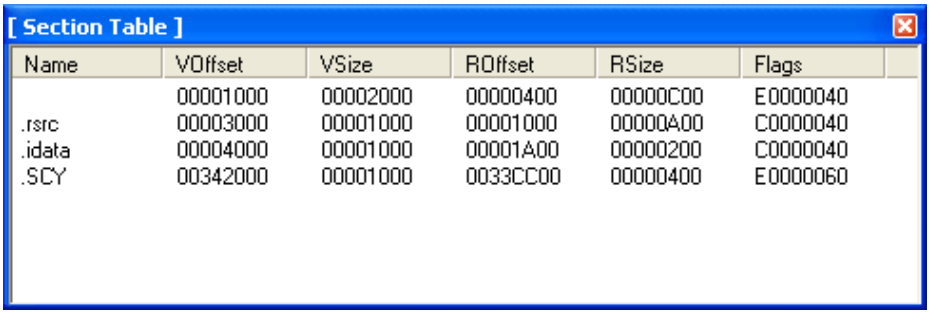
3,23 MB !! un tamaño gigantesco para un crackme y/o unpackme.

Esto es cosa del packer, para que funcione inserta una ingente cantidad de código den el ejecutable protegido que aumenta su tamaño descomunalmente.

Abramos LordPE si lo habíamos cerrado y cargamos el exe 'dumpeado_SCY.exe' que nos dejó Scylla_x86, vamos a Sections y hacemos un '*wipe section header*' de las tres secciones siguientes a .idata:

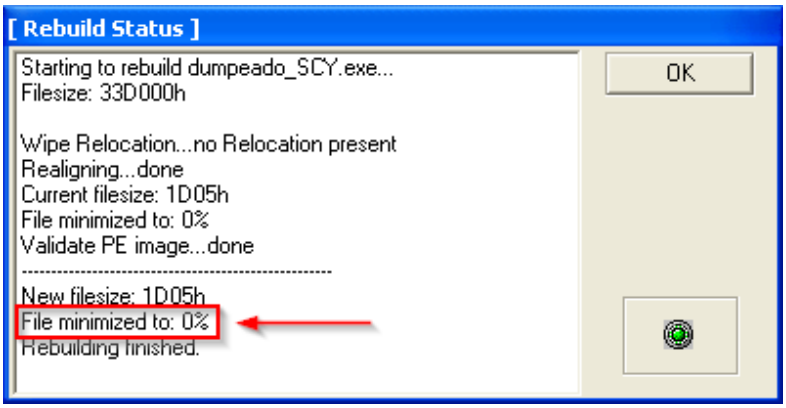


Nos debe quedar como en la siguiente imagen:

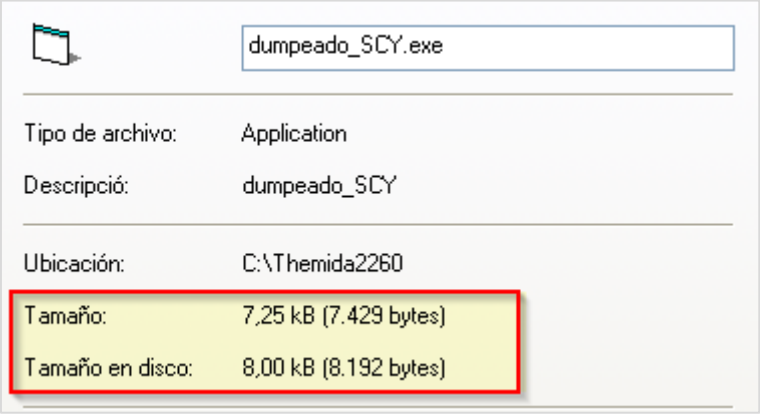


Después pulsamos el botón '*Save*' y por último '*Ok*'. Si tratamos de ejecutarlo nos funcionará:

No nos alarmemos, es cosa del encabezado (header) y el mismo LordPE nos da la opción de reconstruirlo con '*Rebuild PE*', elegimos el mismo archivo 'dumpeado_SCY.exe' y le pasamos el Rebuild:

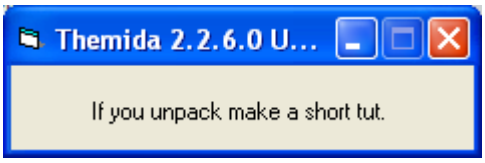


Veamos su nuevo tamaño:



Esto es otra cosa.

Hacemos doble clic sobre el nuevo archivo ejecutable 'dumpeado_SCY.exe' y...



Corre bien. Tenemos el unpack terminado.

Misión Cumplida !

8. Asomados al abismo

No me quiero despedir sin antes echar un vistazo al infierno: La función Virtual de Themida. Es la causante de que esté escribiendo este texto, de lo contrario no me hubiera detenido a desempakar este unpackme. Ha sido la curiosidad de saber cómo suceden las cosas, cómo se desencadenan como un serpentín de fichas concatenadas, una cosa lleva a otra, que a su vez lleva a otra, un sin parar de acontecimientos.

A estas alturas todavía no tengo resuelta esta función, bajo mi punto de vista es el fuerte de la protección, como ya digo en la Introducción creo que es cuestión de algún antidump, tengo una Demo del Themida 2.3.2.0 y he visto la opción de Antidump en las Opciones de Protección. De ahí el mensaje del LordPE '*0x4000 of 0x7F5000 bytes could not be dumped and were padded with zeros*' donde dice que no puede dumpear los siguientes 0x4000 bytes a partir de la dirección 0x7F5000 y que completará esta sección con ceros, todavía no lo sé, lo que sí sé es que no voy a parar hasta que lo descubra, es nuestra naturaleza, está en nuestros genes.

La semana que viene comienzo el trabajo nuevamente y de tiempo casi no voy a disponer pero siempre que pueda le echaré un vistazo. Además tengo los análisis de la rutina casi completados, tengo las cuatro constantes que utiliza esta función identificadas y el REPORTE fijado. Todo esto, si lo consigo, lo escribiré.

Pero como no lo tengo nos conformaremos con echarle un vistazo.

¿Cómo encontrar la Función Virtual? muy fácil, vayamos a la rutina de la IAT:

```
005F42C7    AD              lodsd                   ; // ** lee TABLA campo_API
005F42C8    0F8C 1F000000   jl      005F42ED
005F42CE    0F83 19000000   jnb     005F42ED
005F42D4    E9 14000000     jmp     005F42ED
```

Este es un buen lugar.

Reiniciamos Olly y colocamos un HBP en la dirección del lodsd 0x5F42C7h, damos RUN y estamos allí en un santiamén, recordemos además que tenemos los flags de virtualización:

```
005F4877  C685 F0BD5D09 0>mov     byte ptr [ebp+0x95DBDF0], 0x1       ; // ** Virtualizamos..
005F4897  C785 F9BD5D09 0>mov     dword ptr [ebp+0x95DBDF9], 0x1      ; // ** Caso excepcional
```

Estas banderas conservan el estado de 'virtualización' para la API actual y se inicializan aquí:

```
005F483D  C685 F0BD5D09 0>mov     byte ptr [ebp+0x95DBDF0], 0x0      ; // ** flag de virtualización
005F4844  C785 F9BD5D09 0>mov     dword ptr [ebp+0x95DBDF9], 0x0     ; // ** flag de excepción
```

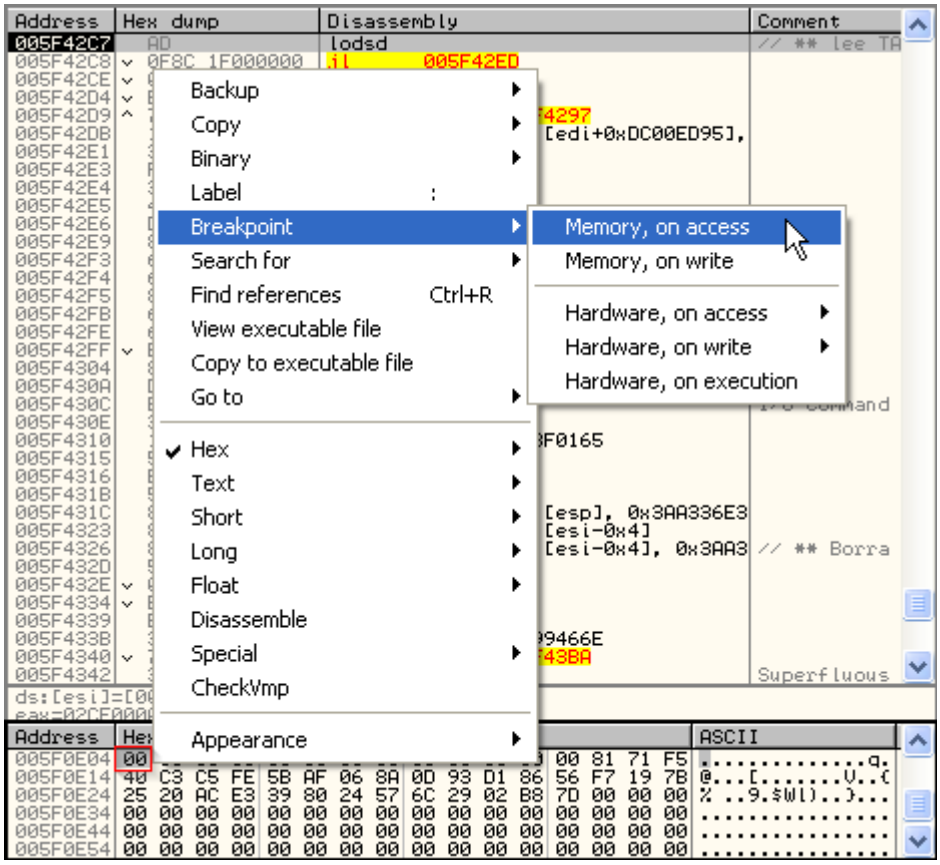
Y los encontramos junto a la tabla de semillas de excepción como vimos en el Apartado '6.La Tabla de Themida' justo en esta imagen:

```
Address   Hex dump                                         ASCII
005F0DF1  DE 81 7C 2D FA 80 7C D0 27 83 7C 00 00 00 00 08  ..|-..|.'.|.....
005F0E01  7C 3A 7E 00 00 00 00 00 00 00 00 00 00 00 00 00  |:~..............
005F0E11  81 71 F5 40 C3 C5 FE 5B AF 06 8A 0D 93 D1 86 56  .q.@...[.......V
005F0E21  F7 19 7B 25 20 AC E3 39 80 24 57 6C 29 02 B8 7D  ..{% ..9.$Wl)..}
005F0E31  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E41  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E51  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E61  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E71  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E81  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
005F0E91  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

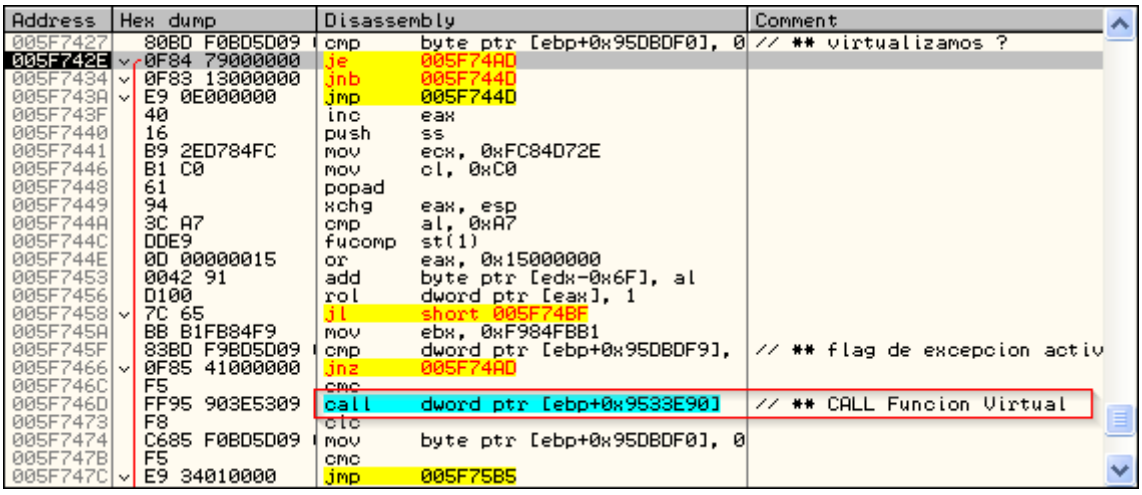
El primero de ellos [0x5F0E04h] corresponde al flag indicador de DLL (kernel, user o advapi) y el segundo [0x5F0E0Dh] se activa para informar de una excepción para la API actual (no será virtualizada).

Cuando digo 'virtualizada' me refiero a que el tratamiento se le dará en la función virtual y no en la que llamamos función normal (o rutina de la IAT), al fin y al cabo se le dará el mismo tratamiento tanto en una rutina como en la otra, la diferencia está en que la función virtual en lugar de ejecutar las instrucciones 'visibles' las 'emula', es decir, puede ejecutar el mismo código de manera distinta. Para complicar las cosas utiliza 4 constantes de encriptación totalmente distintas a las de la función normal. Aun así el resultado es el mismo en ambas funciones.

Para llegar a esta función podemos colocar un BPM en el primero de los flags (indicador de DLL):



Damos RUN y la segunda es la BUENA:



El salto va a pasar por encima de dicha función, si miramos los registros vemos esto:

EAX F5BF159A >> campo_IAT del REGISTRO
ECX 00000002
EDX 7C91E514 ntdll.KiFastSystemCallRet
EBX 005FA0DA Themida_.005FA0DA
ESP 0012FF70
EBP F7015014
ESI 005EA533 Themida_.005EA533
EDI 02D10119
EIP 005F742E Themida_.005F742E

Si miramos la imagen de la Tabla Completa vemos que efectivamente es el campo_IAT:

Address	Hex dump	ASCII
005EA52B	A0 05 44 EB 9A 15 BF F5 1C 02 00 C0 AA AA AA AA	..D.....
005EA53B	FF FF FF FF DD DD DD DD 07 06 63 2D 9B 15 BF 15	.....c-....
005EA54B	1B 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD	...@.....
005EA55B	6E 46 B7 12 9B 15 BF 35 14 02 00 80 AA AA AA AA	nF.....5.....
005EA56B	FF FF FF FF DD DD DD DD E7 0B 44 FD 9B 15 BF 55	.....D....U
005EA57B	1A 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD	.....
005EA58B	15 CF 29 AB 9B 15 BF 75 13 02 00 00 AA AA AA AA	..)....u.....
005EA59B	FF FF FF FF DD DD DD DD 38 2E 60 8C 9B 15 BF 95	.....8.`.....
005EA5AB	12 02 00 40 AA AA AA AA FF FF FF FF DD DD DD DD	...@.....
005EA5BB	CC 02 78 E0 9B 15 BF B5 16 02 00 00 AA AA AA AA	..x.....
005EA5CB	FF FF FF FF DD DD DD DD 2F 3A CF 21 9B 15 BF D5	...../:.!.....

Normalmente la operación para obtener la dirección de la IAT sería:

✓ dirección de la IAT=([{campo_IAT ROL 5} ADD CONSTANTE2] ADD [0x40661D]);

ROL F5BF159A, 5 == B7E2B35E
ADD B7E2B35E, 481D5CA2 == 00001000
ADD 00001000, 400000 == 00401000 >> dirección IAT desencriptada

Pero esto NO ES VÁLIDO en la función virtual porque la CONSTANTE2 no es 0x481D5CA2h, tiene otro valor.

Hagamos una pequeña incursión en esta función, cambiamos el Flag Zero para que no salte en 0x005F742Eh y llegamos a la entrada del call:

Address	Hex dump	Disassembly		Comment
005F7458	7C 65	jl	short 005F74BF	
005F745A	BB B1FB84F9	mov	ebx, 0xF984FBB1	
005F745F	83BD F9BD5D09	cmp	dword ptr [ebp+0x95DBDF9], 0x0	// ** flag de excepcion activc
005F7466	0F85 41000000	jnz	005F74AD	
005F746C	F5	cmc		
005F746D	FF95 903E5309	call	dword ptr [ebp+0x9533E90]	// ** CALL Funcion Virtual
005F7473	F8	clc		
005F7474	C685 F0BD5D09	mov	byte ptr [ebp+0x95DBDF0], 0x0	
005F747B	F5	cmc		
005F747C	E9 34010000	jmp	005F75B5	

Entramos con F7 y vemos dos PUSH y un salto:

Address	Hex dump	Disassembly		Comment
005EB9EA	68 98AE1E00	push	0x1EAE98	
005EB9EF	68 1D020000	push	0x21D	
005EB9F4	E9 EE8CF5FF	jmp	005446E7	
005EB9F9	E8 00000000	call	005EB9FE	
005EB9FE	58	pop	eax	Themida_.005F7473
005EB9FF	2D 14000000	sub	eax, 0x14	
005EBA04	8BD3	mov	edx, ebx	Themida_.005FA0DA
005EBA06	8D85 D6695D09	lea	eax, dword ptr [ebp+0x95D69D6]	

Pulsamos <F7> hasta superar el JMP que nos pone aquí:

```
Address   Hex dump     Disassembly                          Comment
005446E7  9C           pushfd
005446E8  60           pushad
005446E9  E8 00000000  call    005446EE
005446EE  59           pop     ecx
005446EF  83E9 07      sub     ecx, 0x7
005446F2  81E9 E7461400 sub    ecx, 0x1446E7
005446F8  BD 6FED0000  mov     ebp, 0xED6F
005446FD  01CD         add     ebp, ecx
005446FF  51           push    ecx
00544700  B9 01000000  mov     ecx, 0x1
00544705  BB 46000000  mov     ebx, 0x46
0054470A  31C0         xor     eax, eax
0054470C  F0:0FB14C1D 00 lock cmpxchg dword ptr [ebp+ebx], ecx   LOCK prefix
00544712  74 04        je      short 00544718
```

traceamos hasta encontrar un 'jmp [eax]':

```
Address   Hex dump     Disassembly                          Comment
00544766  EB F4        jmp     short 0054475C
00544768  58           pop     eax                          02D10075
00544769  5B           pop     ebx                          02D10075
0054476A  89441D 00    mov     dword ptr [ebp+ebx], eax     Themida_.00543B87
0054476E  8B5C24 24    mov     ebx, dword ptr [esp+0x24]
00544772  C1E3 02      shl     ebx, 0x2
00544775  01D8         add     eax, ebx
00544777  FF20         jmp     dword ptr [eax]              Themida_.00493AF9
00544779  E9 04000000  jmp     00544782
0054477E  A1 9E807CC6  mov     eax, dword ptr [0xC67C809E]
00544783  85C2         test    edx, eax                     Themida_.00543B87
00544785  383F         cmp     byte ptr [edi], bh
00544787  0949 B8      or      dword ptr [ecx-0x48], ecx    Themida_.00400000
0054478A  E9 29158C50  jmp     50E05CB8
```

Lo seguimos y nos pone aquí:

```
Address   Hex dump     Disassembly                          Comment
00493AF9  89F6         mov     esi, esi                     Themida_.005EA533
00493AFB  BA 00000000  mov     edx, 0x0
00493B00  89EB         mov     ebx, ebp                     Themida_.0040ED6F
00493B02  81C3 4C000000 add    ebx, 0x4C
00493B08  8B1B         mov     ebx, dword ptr [ebx]
00493B0A  81C3 02000000 add    ebx, 0x2
00493B10  66:8B13      mov     dx, word ptr [ebx]
00493B13  C1E2 02      shl     edx, 0x2
00493B16  89EF         mov     edi, ebp                     Themida_.0040ED6F
00493B18  81C7 42000000 add    edi, 0x42
00493B1E  8B1F         mov     ebx, dword ptr [edi]
00493B20  01D3         add     ebx, edx                     Themida_.00543313
00493B22  8B3B         mov     edi, dword ptr [ebx]
00493B24  89EA         mov     edx, ebp                     Themida_.0040ED6F
00493B26  81C2 4C000000 add    edx, 0x4C
00493B2C  8102 04000000 add    dword ptr [edx], 0x4
00493B32  FFE7         jmp     edi
00493B34  81EE 40000000 sub    esi, 0x40
00493B3A  81CF 00080000 or     edi, 0x800
00493B40  81F2 01000000 xor    edx, 0x1
00493B46  89E9         mov     ecx, ebp                     Themida_.0040ED6F
00493B48  BB 00040000  mov     ebx, 0x400
00493B4D  01C7         add     edi, eax                     Themida_.00543B87
00493B4F  31D3         xor     ebx, edx                     Themida_.00543313
00493B51  BF 00040000  mov     edi, 0x400
00493B56  BF 00020000  mov     edi, 0x200
```

Se trata de una función tipo lineal, todo el código se va a ejecutar secuencialmente llegando a un salto tipo 'jmp registro' que nos pone en el inicio de otra función similar, seguimos el 'jmp edi' y nos pone aquí:

```
Address   Hex dump     Disassembly                          Comment
00486ED9  89E8         mov     eax, ebp                     Themida_.0040ED6F
00486EDB  05 4C000000  add     eax, 0x4C
00486EE0  8B00         mov     eax, dword ptr [eax]         Themida_.00493AF9
00486EE2  05 00000000  add     eax, 0x0
00486EE7  0FB730       movzx   esi, word ptr [eax]
00486EEA  01EE         add     esi, ebp                     Themida_.0040ED6F
00486EEC  8926         mov     dword ptr [esi], esp
00486EEE  B9 00000000  mov     ecx, 0x0
00486EF3  89E8         mov     eax, ebp                     Themida_.0040ED6F
00486EF5  05 4C000000  add     eax, 0x4C
00486EFA  8B00         mov     eax, dword ptr [eax]         Themida_.00493AF9
00486EFC  05 02000000  add     eax, 0x2
00486F01  66:8B08      mov     cx, word ptr [eax]
00486F04  C1E1 02      shl     ecx, 0x2
00486F07  89EA         mov     edx, ebp                     Themida_.0040ED6F
00486F09  81C2 42000000 add    edx, 0x42
00486F0F  8B02         mov     eax, dword ptr [edx]         Themida_.005EAE9C
00486F11  01C8         add     eax, ecx                     Themida_.00400000
00486F13  8B30         mov     esi, dword ptr [eax]         Themida_.00493AF9
00486F15  89EA         mov     edx, ebp                     Themida_.0040ED6F
00486F17  81C2 4C000000 add    edx, 0x4C
00486F1D  8102 04000000 add    dword ptr [edx], 0x4
00486F23  FFE6         jmp     esi                          Themida_.005EA533
00486F25  BF 00020000  mov     edi, 0x200
00486F2A  89F7         mov     edi, esi                     Themida_.005EA533
00486F2C  81E6 04000000 and    esi, 0x4
```

Si seguimos el 'jmp esi' nos deja aquí:

```
Address   Hex dump     Disassembly                      Comment
00429957  89EB         mov     ebx, ebp                 Themida_.0040ED6F
00429959  81C3 04000000 add    ebx, 0x4
0042995F  C703 00000000 mov    dword ptr [ebx], 0x0
00429965  89EA         mov     edx, ebp                 Themida_.0040ED6F
00429967  81C2 0E000000 add    edx, 0xE
0042996D  C702 00000000 mov    dword ptr [edx], 0x0
00429973  89EE         mov     esi, ebp                 Themida_.0040ED6F
00429975  81C6 61000000 add    esi, 0x61
0042997B  C706 00000000 mov    dword ptr [esi], 0x0
00429981  89EB         mov     ebx, ebp                 Themida_.0040ED6F
00429983  81C3 6E000000 add    ebx, 0x6E
00429989  C703 00000000 mov    dword ptr [ebx], 0x0
0042998F  89EE         mov     esi, ebp                 Themida_.0040ED6F
00429991  81C6 7D000000 add    esi, 0x7D
00429997  C706 00000000 mov    dword ptr [esi], 0x0
0042999D  89EF         mov     edi, ebp                 Themida_.0040ED6F
0042999F  81C7 08000000 add    edi, 0x8
004299A5  66:C707 0000  mov    word ptr [edi], 0x0
004299AA  89EA         mov     edx, ebp                 Themida_.0040ED6F
004299AC  81C2 A5000000 add    edx, 0xA5
004299B2  66:C702 0000  mov    word ptr [edx], 0x0
004299B7  89EA         mov     edx, ebp                 Themida_.0040ED6F
004299B9  81C2 37000000 add    edx, 0x37
004299BF  66:C702 0000  mov    word ptr [edx], 0x0
004299C4  89EE         mov     esi, ebp                 Themida_.0040ED6F
004299C6  81C6 A1000000 add    esi, 0xA1
004299CC  C706 00000000 mov    dword ptr [esi], 0x0
004299D2  89EE         mov     esi, ebp                 Themida_.0040ED6F
004299D4  81C6 59000000 add    esi, 0x59
004299DA  C706 00000000 mov    dword ptr [esi], 0x0
004299E0  B9 00000000  mov     ecx, 0x0
004299E5  89EF         mov     edi, ebp                 Themida_.0040ED6F
004299E7  81C7 4C000000 add    edi, 0x4C
004299ED  8B3F         mov     edi, dword ptr [edi]
004299EF  81C7 00000000 add    edi, 0x0
004299F5  66:8B0F      mov     cx, word ptr [edi]
004299F8  C1E1 02      shl     ecx, 0x2
004299FB  89EB         mov     ebx, ebp                 Themida_.0040ED6F
004299FD  81C3 42000000 add    ebx, 0x42
00429A03  8B33         mov     esi, dword ptr [ebx]     Themida_.00486ED9
00429A05  01CE         add     esi, ecx
00429A07  8B3E         mov     edi, dword ptr [esi]
00429A09  89EE         mov     esi, ebp                 Themida_.0040ED6F
00429A0B  81C6 4C000000 add    esi, 0x4C
00429A11  8106 02000000 add    dword ptr [esi], 0x2
00429A17  FFE7         jmp     edi                      Themida_.00486ED9
00429A19  81E6 00080000 and    esi, 0x800
00429A1F  29F7         sub     edi, esi                 Themida_.00429957
00429A21  BB 00000000  mov     ebx, 0x0
```

Y así hasta casi el infinito, si nos detenemos en el 'mov edi, dword ptr [esi]' de 0x429A07h y hacemos un follow in dump encontramos la tabla de funciones que comienza aquí:

```
Address   Hex dump                                         ASCII
00543313  1A EE 40 00 C2 F3 40 00 8A F7 40 00 AB FB 40 00  ..@...@...@...
00543323  09 00 41 00 86 03 41 00 16 08 41 00 98 0B 41 00  ..A...A...A...
00543333  7B 0F 41 00 DB 15 41 00 D4 19 41 00 07 1F 41 00  {.A...A...A...
00543343  1C 22 41 00 3F 26 41 00 64 28 41 00 BF 2C 41 00  ."A.?&A.d(A..,
00543353  40 30 41 00 88 33 41 00 B7 37 41 00 2B 38 41 00  @0A..3A..7A.+8
00543363  42 3C 41 00 F9 41 41 00 F4 46 41 00 82 49 41 00  B<A..AA..FA..I
00543373  EB 4C 41 00 72 50 41 00 DA 55 41 00 D6 59 41 00  .LA.rPA..UA..Y
00543383  86 5D 41 00 90 61 41 00 8F 64 41 00 E9 68 41 00  .]A..aA..dA..h
00543393  AF 6D 41 00 0C 70 41 00 17 75 41 00 1E 79 41 00  .mA..pA..uA..y
005433A3  18 7E 41 00 AB 82 41 00 2A 87 41 00 F4 8A 41 00  .~A...A.*.A...
005433B3  18 8F 41 00 DD 92 41 00 FC 96 41 00 DD 9C 41 00  ..A...A...A...
005433C3  3D A0 41 00 48 A4 41 00 72 A6 41 00 13 AB 41 00  =.A.H.A.r.A...
005433D3  A3 AF 41 00 6A B4 41 00 DB B7 41 00 35 BC 41 00  ..A.j.A...A.5.
```

y no termina hasta 0x5446EBh, si hacemos las cuentas:

SUB 5446EB, 543313 == 13D8
DIV 13D8, 4 == 4F6

Hay 0x04F6h, o lo que es lo mismo: 1270 funciones

Tracear todas ellas parece un imposible.

El modo en que podemos encontrar lo que queremos es a través de los datos que recibe cada función. Volvamos al inicio de la Función virtual.

Reiniciamos Olly y volvemos al salto previo al CALL a la Función Virtual:

```
005F7427   80BD F0BD5D09 0>cmp     byte ptr [ebp+0x95DBDF0], 0x0    ; // ** virtualizamos ?
005F742E   0F84 79000000   je      005F74AD
005F7434   0F83 13000000   jnb     005F744D
```

Invertimos el Flag Zero para que NO salte JE y llegamos nuevamente al CALL:

```
005F746C   F5              cmc
005F746D   FF95 903E5309   call    dword ptr [ebp+0x9533E90]     ; // ** CALL Función Virtual
005F7473   F8              clc
```

Entremos con <F7>, recordando que el vaor del campo_IAT está en EAX, después de los pushfd y pushad del inicio de la función queda el valor en la pila:

```
0012FF40  02D100B5
0012FF44  005EA533  Themida_.005EA533
0012FF48  F7015014
0012FF4C  0012FF60
0012FF50  005FA0DA  Themida_.005FA0DA
0012FF54  7C91E514  ntdll.KiFastSystemCallRet
0012FF58  00000002
0012FF5C  F5BF159A
0012FF60  00000347
0012FF64  0000021D
0012FF68  001EAE98
0012FF6C  005F7473  RETURN to Themida_.005F7473
0012FF70  FFFFF000
0012FF74  0000006C
0012FF78  00000400
0012FF7C  F7015014
```

Colocamos un BPM on Access en esta dirección 0x12FF5Ch damos RUN y paramos aquí:

```
Address   Hex dump     Disassembly                    Comment
004F66E9  011E         add     dword ptr [esi], ebx
004F66EB  BF 00040000  mov     edi, 0x400
004F66F0  8F01         pop     dword ptr [ecx]        Themida_.005F0CF3
004F66F2  81CF 00080000 or     edi, 0x800
004F66F8  81E7 00000080 and    edi, 0x80000000
004F66FE  81C7 00040000 add    edi, 0x400
004F6704  89C8         mov     eax, ecx               Themida_.0040EDA9
004F6706  81F7 80000000 xor    edi, 0x80
004F670C  31F7         xor     edi, esi               Themida_.0040EE10
004F670E  81C7 80000000 add    edi, 0x80
004F6714  B9 00000000  mov     ecx, 0x0
004F6719  81EF 40000000 sub    edi, 0x40
004F671F  89EF         mov     edi, ebp               Themida_.0040ED6F
004F6721  BA 00040000  mov     edx, 0x400
004F6726  BA 00040000  mov     edx, 0x400
004F672B  89C2         mov     edx, eax
004F672D  BA 0A000000  mov     edx, 0xA
004F6732  81C7 4C000000 add    edi, 0x4C
004F6738  81CA 04000000 or     edx, 0x4
004F673E  21D2         and     edx, edx               Themida_.0040ED7D
004F6740  81F2 28000000 xor    edx, 0x28
004F6746  01F2         add     edx, esi               Themida_.0040EE10
004F6748  8B3F         mov     edi, dword ptr [edi]
004F674A  81CA 00080000 or     edx, 0x800
004F6750  81C7 08000000 add    edi, 0x8
004F6756  66:8B0F      mov     cx, word ptr [edi]
```

Va a cambiar el valor a ECX que vale 0x0040EDA9h, pasamos la instrucción y colocamos un 'BPM on Access' en esta nueva posición, paramos aquí:

```
Address   Hex dump     Disassembly                    Comment
0045A7B8  FF37         push    dword ptr [edi]
0045A7BA  FF31         push    dword ptr [ecx]
0045A7BC  8F07         pop     dword ptr [edi]        Themida_.005FA0DA
0045A7BE  8F01         pop     dword ptr [ecx]        Themida_.005FA0DA
0045A7C0  89EE         mov     esi, ebp               Themida_.0040ED6F
0045A7C2  81F7 80000000 xor    edi, 0x80
0045A7C8  29DA         sub     edx, ebx               Themida_.005EAF51
0045A7CA  B9 01000000  mov     ecx, 0x1
0045A7CF  81C6 4C000000 add    esi, 0x4C
0045A7D5  01F2         add     edx, esi               Themida_.0040ED7D
0045A7D7  BB 0A000000  mov     ebx, 0xA
0045A7DC  01DA         add     edx, ebx               Themida_.005EAF51
```

Cambió el valor del DUMP a 0x02D100B5h que es justo el punto donde toca copiar la api polimórfica.

Después para aquí:

```
Address   Hex dump     Disassembly                          Comment
0051AF46  FF33         push    dword ptr [ebx]              ntdll.KiFastSystemCallRet
0051AF48  8F07         pop     dword ptr [edi]              Themida_.005FA0DA
0051AF4A  8F03         pop     dword ptr [ebx]              Themida_.005FA0DA
0051AF4C  89EE         mov     esi, ebp                     Themida_.0040ED6F
0051AF4E  81E7 40000000 and    edi, 0x40
0051AF54  29C8         sub     eax, ecx                     Themida_.0040ED73
0051AF56  81E3 00040000 and    ebx, 0x400
0051AF5C  81F2 00080000 xor    edx, 0x800
0051AF62  81C6 4C000000 add    esi, 0x4C
0051AF68  81CA 80000000 or     edx, 0x80
0051AF6E  81CA 1F000000 or     edx, 0x1F
0051AF74  8B36         mov     esi, dword ptr [esi]
Stack [0012FF68]=005FA0DA (Themida_.005FA0DA)
ds:[0040EDA9]=7C91E514 (ntdll.KiFastSystemCallRet)
```

Copiará este valor al dump, RUN nuevamente y colocar 0x02D10000h, valor que compara con la Base del Kernel, <F9> nuevamente y carga de nuevo el campo_IAT en esta posición:

```
Address   Hex dump                                         ASCII
0040EDA9  9A 15 BF F5 68 FF 12 00 13 33 54 00 01 00 00 00  ....h....3T.....
0040EDB9  00 00 BD AF 5E 00 37 6C 40 00 14 50 01 F7 00 F2  ....^.7l@..P....
0040EDC9  77 0F C3 [illegible] 00 80 7C 00 00 00 00 00 00 00 00 00 00  w.....|.........
0040EDD9  00 00 00 00 B5 19 25 3E 00 00 00 00 00 00 00 00  ......%>........
0040EDE9  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040EDF9  00 00 00 00 14 50 01 F7 00 00 00 00 B5 00 D1 02  .....P..........
0040EE09  00 00 00 00 00 00 00 A5 E4 F5 AC 3A 00 14 E5 91  ...........:....
0040EE19  7C 89 EF 81 C7 04 00 00 00 8B 37 89 E8 05 A1 00  |.........7.....
0040EE29  00 00 31 30 BF 00 00 00 00 81 F6 01 00 00 00 89  ..10............
0040EE39  FF BF 00 04 00 00 89 F6 BE 00 00 00 00 B8 0A 00  ................
0040EE49  00 00 89 E9 81 C1 4C 00 00 00 BA 00 02 00 00 81  ......L.........
0040EE59  E7 04 00 00 00 89 C8 8B 09 25 00 04 00 00 B8 00  .........%......
```

Hay que tener paciencia, RUN de nuevo y coloca la Base del Kernel en esta posición, después coloca el valor del campo_IAT una vez más.

Si seguimos pulsando <F9> veremos que toma punteros a la Tabla de la IAT:

```
Address   Hex dump     Disassembly                          Comment
0049BC6E  FF32         push    dword ptr [edx]
0049BC70  FF30         push    dword ptr [eax]              Themida_.005EA533
0049BC72  8F02         pop     dword ptr [edx]              Themida_.005FA0DA
0049BC74  8F00         pop     dword ptr [eax]              Themida_.005FA0DA
0049BC76  89EE         mov     esi, ebp                     Themida_.0040ED6F
0049BC78  81C6 4C000000 add    esi, 0x4C
0049BC7E  8B36         mov     esi, dword ptr [esi]
0049BC80  81C6 10000000 add    esi, 0x10
0049BC86  0FB73E       movzx   edi, word ptr [esi]
0049BC89  89EE         mov     esi, ebp                     Themida_.0040ED6F
0049BC8B  81C6 4C000000 add    esi, 0x4C
ebp=0040ED6F (Themida_.0040ED6F)
esi=0040ED7D (Themida_.0040ED7D)
Address   Hex dump                                         ASCII
005EA533  1C 02 00 C0 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA543  07 06 63 2D 9B 15 BF 15 1B 02 00 40 AA AA AA AA  ..c-.......@....
005EA553  FF FF FF FF DD DD DD DD 6E 46 B7 12 9B 15 BF 35  ........nF.....5
005EA563  14 02 00 80 AA AA AA AA FF FF FF FF DD DD DD DD  ................
005EA573  E7 0B 44 FD 9B 15 BF 55 1A 02 00 80 AA AA AA AA  ..D....U........
005EA583  FF FF FF FF DD DD DD DD 15 CF 29 AB 9B 15 BF 75  ..........)....u
```

Apunta al campo_JMP, unos <F9's> más y paramos aquí:

```
Address   Hex dump     Disassembly                          Comment
004B82A1  293B         sub     dword ptr [ebx], edi         // ** ojo a EDI
004B82A3  9C           pushfd
004B82A4  89FE         mov     esi, edi
004B82A6  89F7         mov     edi, esi                     Themida_.0040EE14
004B82A8  BE 00000000  mov     esi, 0x0
004B82AD  81F3 00080000 xor    ebx, 0x800
004B82B3  89E9         mov     ecx, ebp                     Themida_.0040ED6F
004B82B5  89FA         mov     edx, edi
004B82B7  89CF         mov     edi, ecx                     Themida_.0040ED73
004B82B9  81C1 4C000000 add    ecx, 0x4C
004B82BF  B8 00040000  mov     eax, 0x400
edi=2373EBD9   desencripta Campo_IAT
ds:[0040EDA9]=F5BF159A
Address   Hex dump                                         ASCII
0040EDA9  9A 15 BF F5 68 FF 12 00 13 33 54 00 01 00 00 00  ....h....3T.....
0040EDB9  00 00 7E B4 5E 00 85 6F 40 00 14 50 01 F7 00 00  ..~.^..o@..P....
0040EDC9  00 00 00 33 A5 5E 00 00 00 00 00 00 00 00 00 00  ...3.^..........
0040EDD9  00 00 00 00 CC 9F 12 06 00 00 00 00 00 00 00 00  ................
0040EDE9  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040EDF9  00 00 00 00 14 50 01 F7 00 00 00 00 B5 00 D1 02  .....P..........
```

Utiliza la constante 0x2373EBD9h para desencriptar el campo_IAT, quedando:

XOR F5BF159A, 2373EBD9 == D24B29C1

Después llega un 'rol [eax], 5':

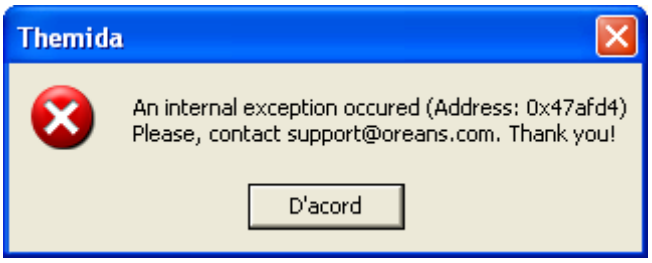
```
Address   Hex dump     Disassembly                          Comment
0047AFD2  D300         rol     dword ptr [eax], cl          // ** rol x, 5
0047AFD4  81E6 80000000 and    esi, 0x80
0047AFDA  FF37         push    dword ptr [edi]
0047AFDC  81C6 01000000 add    esi, 0x1
0047AFE2  01DE         add     esi, ebx
0047AFE4  BF 00000000  mov     edi, 0x0
0047AFE9  81E7 01000000 and    edi, 0x1
0047AFEF  BF 00040000  mov     edi, 0x400
0047AFF4  89FF         mov     edi, edi                     Themida_.0040ED9E
0047AFF6  81C6 FFFFFF7F add    esi, 0x7FFFFFFF
0047AFFC  81CF 00040000 or     edi, 0x400
cl=05 (decimal 5.)
ds:[0040EDA9]=D24B29C1

Address   Hex dump                                          ASCII
0040EDA9  C1 29 4B D2 68 FF 12 00 13 33 54 00 01 00 00 00   .)K.h....3T.....
0040EDB9  00 00 8C B4 5E 00 85 6F 40 00 14 50 01 F7 00 53   ....^..o@..P...S
0040EDC9  41 A0 E9 33 A5 5E 00 00 00 00 00 00 00 00 00 00   A..3.^..........
0040EDD9  00 00 00 00 1F E1 B2 EF 00 00 00 00 00 00 00 00   ................
0040EDE9  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0040EDF9  00 00 00 00 14 50 01 F7 00 00 00 00 B5 00 D1 02   .....P..........
```

Quedando:

ROL D24B29C1, 5 == 4965383A

Si damos RUN termina abruptamente nuestra incursión:



Es lógico, los datos que se la pasan a la función NO están preparados para sus constantes de desencriptación, por lo que esto era de esperar.

No obstante, siguiendo este método se consiguen obtener las direcciones donde realiza las operaciones con los CAMPOS de la Tabla de la IAT y a partir de aquí se obtienen las constantes. A partir de ellas podríamos obtener las direcciones reales de la tabla aplicando las mismas operaciones que Themida.

Entones, para casos en que se utilice la Función Virtual el método consistiría en vigilar los valores que se le pasan anotando las operaciones que va haciendo hasta llegar al valor final, digamos por ejemplo que va a escribir en la dirección 0x401240h de la IAT, anotaríamos todo el proceso desde que toma el valor campo_IAT hasta llegar a la dirección 0x401240h. Ese interesante vigilar constantemente [ESP].

En mis investigaciones he podido descubir que las instrucciones utilizadas en funciones lineales, para desencriptar el campo_IAT habitualmente son estas:

```
1. sub dword [edi], ecx     >> ecx tiene la CONSTANTE1 Virtual
2. rol dword [eax], cl
3. xor dword [edx], esi     >> esi tiene la CONSTANTE2 Virtual
4. add dword [edi], esi
```

El programa de software comercial que tengo por resolver utiliza este mismo método con una función virtual 'lineal' como esta, las instrucciones NO VARÍAN [tengo un pequeño análisis de 12 folios de esta Función] y esto no cambia. Para obtener la dirección de la IAT se utiliza esta fórmula:

- ✓ dirección_iat_virtualizada = ([{campo_IAT SUB CONSTANTE1} ROL 5] XOR CONSTANTE2] ADD ImageBase)

Para obtener la dirección del salto/call a modificar utiliza estas instrucciones:

```
5. sub dword ptr [ebx], edx      >> edx tiene la CONSTANTE3 Virtual
6. rol dword [eax], cl
7. xor dword [edx], esi    >> esi tiene la CONSTANTE4 Virtual
8. add dword [edx], edi
```

Y la fórmula para obtener la dirección a partir del CAMPO es esta:

- ✓ dirección_salto_virtualizada = ([{campo_JMP SUB CONSTANTE3} ROL 5] XOR CONSTANTE4] ADD ImageBase)

Esto para las funciones virtuales lineales, para las cíclicas TODO lo hace en una misma subfunción, toma los operandos y salta a la función, por ejemplo para hacer el 'ROL [registro], x' saltaría a:

```
0076359B    66:8B0C24       mov     cx, word ptr [esp]
0076359F    83C4 02         add     esp, 0x2
007635A2    D30424          rol     dword ptr [esp], cl
007635A5    9C              pushfd
007635A6  ^ E9 0CEEFFFF     jmp     007623B7
```

Para hacer los XOR tiene otra subfunción:

```
007635ED    8B0424          mov     eax, dword ptr [esp]
007635F0    81C4 04000000   add     esp, 0x4
007635F6    310424          xor     dword ptr [esp], eax
007635F9    9C              pushfd
007635FA  ^ E9 B8EDFFFF     jmp     007623B7
```

Lo mismo para la operación ADD, SUB, etc.. trabajan con 4 constantes también.

La ventaja de este método es que encontrando la función que nos interesa podemos poner un BPM (no HBP ni BP pues los detecta) en esta e ir mirando los registros que tiene en un REPORTE especial de donde va leyendo todo el tiempo, tiene los registros actuales, los EFLAGS, etc..

Me gustaría explicar todo esto en detalle si logro que el dumpeado corra.

9. Notas Finales

No tengo palabras para describir lo que me ha gustado este packer y lo que he disfrutado con él, sin duda una de mis mejores experiencias en el mundo del reversing, he disfrutado mucho, sin duda.

Al margen de esto me gustaría decir que no doy por concluido el trabajo, esto es un adelanto de lo que me gustaría hacer ya que Themida da para mucho más. Todo ello depende, por su puesto del avance en el estudio de la protección.

También me he quedado con las ganas de escribir un script para descifrar la Tabla de la IAT en casos más amplios donde se trabaja con más librerías y hay que tener muchísimas más cosas en cuenta, veremos si con el tiempo lo podemos lograr.

Asimismo me gustaría agradecer a otras personas que nos hacen la vida más fácil con su trabajo. GRACIAS Oleh Yuschuk, °£·çôÂó°, Hellsp@wn & Archer, Epsylon, Cozofdeath, RDGMax, gRn Team, Yoda, cybult por vuestras excelentes herramientas sin las cuales el cracking no sería lo mismo. GRACIAS a tuts4you por sus unpackme. GRACIAS a Akira, deroko, +NCR, Ivinson, Karmany por vuestros trabajos en Themida que me han ayudado muchísimo y en los que me he inspirado.

Y como no GRACIAS a CrackSLatinoS por su gran trabajo y sus excelentes aportaciones. Grandes crackers en una gran familia.

Un fuerte abrazo a TODOS !!

Hasta la próxima
Snat.~